pouwchain

Блокчейн-протокол з консенсусом для розподіленого навчання нейронних мереж. Покроковий гайд до дипломного проєкту.

Гайд до бакалаврської кваліфікаційної роботи, спеціальність 122, кафедра систем штучного інтелекту, НУ «Львівська політехніка». Термін захисту — червень 2027.

Кроки 0–11 доведені до рівня контрактів, скелетів і підказок: за ними прототип має бути готовий до 1 листопада 2026. Фази 2–4 подані планом і завданнями без скелетів — їхні розділи дописуються, коли прототип працює і відомі перші цифри.

Готового коду тут немає. Розв'язки — в окремому файлі **pouwchain-solutions.pdf**. Відкривати лише після власної спроби.

Зміст

Як користуватися цим гайдом 7
Головне правило 7
Цикл на кожен крок 7
Позначки 8
Темп 8
Словник незнайомих слів 8
Що потрібно перед стартом 8
Що ми будуємо 10
Проблема одним абзацом 10
Життєвий цикл раунду 10
Чотири механізми верифікації 10
П'ять моделей атак 11
Що саме міряємо 11
Дві частини системи 12
Правила протоколу, які вже прийняті 12
Структура проєкту 13
Гілки й коміти 13
Календар до 1 листопада 15
Крок 0. Середовище і скелет репозиторію 16
Ідея 16
Нові слова 16
Контракт 16
Твоє завдання 17
Перевірка 18
Крок 1. Контракт між C++ і Python 19
Ідея 19
Нові слова 19
Контракт: канонічний хеш ваг 20
Контракт: заголовок блоку 20
Контракт: обмін вузол ↔ воркер 21
Контракт: сховище ваг 22
Скелет 22
Твоє завдання 24
Перевірка 25
Крок 2. PyTorch рівно стільки, скільки треба 26
Ідея 26
Нові слова 27
Контракт 27

Скелет 29
Твоє завдання 31
Перевірка 32
Зламай навмисне 33
Крок 3. Навчальний блок 34
Ідея 34
Нові слова 34
Контракт 35
Скелет 36
Твоє завдання 37
Перевірка 39
Зламай навмисне 39
Крок 4. Верифікація A і B 41
Ідея 41
Нові слова 42
Контракт 42
Скелет 44
Твоє завдання 46
Перевірка 47
Зламай навмисне 48
Крок 5. Перші атаки: лінивий і халтурник 50
Ідея 50
Нові слова 52
Контракт: модуль атак 52
Контракт: поле `attack` у `task.json` 53
Контракт: скрипт вимірювання 53
Скелет 54
Твоє завдання 55
Перевірка 56
Зламай навмисне 57
Крок 6. Симулятор раундів 58
Ідея 58
Нові слова 59
Контракт: конфіг і блок 59
Контракт: CSV, похідні метрики і базова лінія 60
Скелет 61
Твоє завдання 62
Перевірка 63
Зламай навмисне 63
Вправи 64
Крок 7. Блок і канонічна серіалізація 66

Ідея ... 66
Нові слова ... 66
Контракт ... 67
Скелет ... 69
Тести ... 71
Твоє завдання ... 73
Перевірка ... 73
Зламай навмисне ... 74
Крок 8. Ланцюг і правила валідації ... 76
Ідея ... 76
Нові слова ... 76
Контракт ... 77
Скелет ... 80
Тести ... 83
Твоє завдання ... 83
Перевірка ... 83
Зламай навмисне ... 85
Крок 9. Мережа: три вузли по TCP ... 86
Ідея ... 86
Нові слова ... 86
Контракт ... 87
Скелет ... 90
Твоє завдання ... 93
Перевірка ... 93
Зламай навмисне ... 94
Крок 10. Інтеграція: вузол запускає воркер ... 96
Ідея ... 96
Нові слова ... 96
Контракт ... 97
Скелет ... 98
Скелет: цикл майнінгу ... 100
Скелет: шлях перевірки вхідного блоку ... 100
Контракт: три вузли однією командою ... 101
Контракт: формат логу ... 102
Твоє завдання ... 102
Перевірка ... 103
Зламай навмисне ... 103
Крок 11. Перші цифри і звіт керівнику ... 105
Ідея ... 105
Нові слова ... 105
Контракт: які прогони робимо ... 105
Контракт: що записуємо ... 106

Контракт: порожні таблиці ... 107
Скелет: графіки ... 108
Контракт: звіт керівнику ... 108
Правила чесної звітності ... 109
Твоє завдання ... 109
Перевірка ... 110
Зламай навмисне ... 111
Фаза 2. Решта механізмів і атак (2 листопада — 13 грудня) ... 112
Що має з'явитися за шість тижнів ... 112
Тиждень 8. Механізм C — реплікація ... 112
Тиждень 9. Механізм D — градієнтна узгодженість, і варіант A+ ... 113
Тиждень 10. Атаки 2 (плагіат) і 4 (отруєння міток) ... 114
Тиждень 11. Атака 5 — підробка траєкторії як абляція ... 115
Тиждень 12. Конфіг, CSV, графіки і fraud proof ... 116
Тиждень 13. Запас, розділ 1, погодження з керівником ... 117
Фаза 3. Експерименти (14 грудня — 28 лютого) ... 119
План експериментів ... 119
Пастка в кожному експерименті ... 119
Правила, спільні для всіх експериментів ... 121
Що ріжемо, якщо не встигаю ... 121
Фаза 4. Текст роботи (листопад — червень) ... 123
Обсяг і графік ... 123
Розділ 1. Аналіз предметної області ... 123
Розділ 2. Проєктування протоколу ... 124
Розділ 3. Програмна реалізація ... 124
Розділ 4. Експериментальне дослідження ... 124
Вступ, висновки, новизна ... 124
Формальні вимоги, які треба тримати в полі зору ... 125
Що робити, коли щось іде не так ... 126
Коли просити допомоги і що приносити ... 127
Правила, яких варто триматися до захисту ... 127
Додаток А. Шпаргалка команд ... 129
Середовище Python ... 129
Збірка C++ ... 129
Python: тести, воркер, симулятор ... 129
git ... 130
Запуск трьох вузлів ... 130
Прибирання ... 130
Додаток Б. Типові помилки та як їх читати ... 132
CMake і лінкування ... 132
C++ під час виконання ... 133
Python і PyTorch ... 135

Мовчазна помилка: C++ і Python дають різні хеші 136
Додаток В. Словник понять 138
Блокчейн і протокол 138
Навчання нейронних мереж 139
Додаток Г. Чек-листи 141
Чек-лист сеансу 141
Чек-лист тижня 141
Чек-лист перед контрольною точкою 1 листопада 142
Чек-лист перед кожним експериментом 142
Чек-лист перед показом керівнику 142
Додаток Д. Як користуватися розв'язками 144
Додаток Е. Питання, до яких треба бути готовим на захисті 145

Як користуватися цим гайдом

Цей гайд — не опис готової системи, а робочий план на вісім місяців. Він влаштований так само, як `toychain`: спершу ідея, потім контракт, потім скелет, а тіла функцій пишеш ти. Різниця одна: тут результатом є не навчальна іграшка, а дипломна робота з експериментами, які треба буде захищати перед комісією.

Головне правило

> **УВАГА**
>
> **Не набирай рядок, який не можеш сказати вголос своїми словами.**
>
> У дипломі це правило жорсткіше, ніж у навчальному проєкті. На захисті питають не «що робить ця функція», а «чому ти обрав саме такий поріг» і «що станеться, якщо вузол збреше ось тут». Код, якого ти не розумієш, на захисті стає мовчанкою.

Цикл на кожен крок

1. **Прочитай «Ідею» до кінця, перш ніж дивитися на код.** Спершу навіщо, потім як.
2. **Пройди «Нові слова».** Кожен крок вводить кілька нових понять; вони зібрані в таблицю.
3. **Набери «Контракт».** Заголовки, схеми файлів і сигнатури даються повністю: це домовленість, а не робота. Їх набирай як є.
4. **Візьми «Скелет» і напиши тіла сам.** Тіла замінені на покроковий опис того, що має статись.
5. **Застряг — читай «Підказки» по одній.** Рівень 1 — куди дивитись. Рівень 2 — яка саме функція чи формула. Рівень 3 — майже готовий рядок (там, де рівнів вистачає двох, третього немає). Не читай рівень 3, поки не спробував рівень 2.
6. **Заповни «Передбач» до запуску.** Запиши, що саме виведе програма. Не збіглося — ти щойно знайшов місце, де твоя модель системи хибна. Це найцінніший сигнал у навчанні.
7. **Перевір себе по «Еталону».** Там, де результат залежить від випадковості чи заліза, еталон дає не число, а властивість, яка має виконуватись.
8. **Дай відповідь на «Самоперевірку» вголос.** Мовчазне «ну я ж розумію» — це ілюзія засвоєння.
9. **Зроби «Зламай навмисне».** Зрозуміти код = знати, що станеться, коли його зіпсувати. У цьому проєкті «Зламай» особливо важливий: половина результатів роботи — це саме те, як система поводиться, коли її ламають.
10. **Перенеси в текст роботи те, що позначено рамкою «У текст роботи».** Це економить у квітні тижні: розділи 2 і 3 диплома складаються з рішень, які ти приймаєш зараз.

Позначки

позначка	що означає
ідея	Навіщо це потрібно. Читати першим, до коду.
нові слова	Поняття, які вводить крок.
твоє завдання	Що саме написати. Основна робота розділу.
підказки	Три рівні допомоги. Читати по одній зверху вниз.
передбач до запуску	Записати очікуваний результат **до** запуску.
еталон для самоперевірки	Точні значення або властивості, які мають виконуватись.
самоперевірка	Питання, на які треба відповісти вголос.
зламай навмисне	Навмисно зіпсувати код і пояснити наслідок.
чому саме так	Пояснення рішення, яке могло бути іншим.
увага	Пастка, безпека або незворотна дія.
у текст роботи	Матеріал, який прямо піде в пояснювальну записку.
контрольна точка	Дата і критерій готовності етапу.

Темп

Крок за сеанс, іноді за два. Кроки 0 і 1 — короткі, це один вечір разом. Кроки 2–6 (Python) — по одному сеансу. Кроки 7–9 (C++) — по одному-два. Крок 10 (інтеграція) майже напевно займе два.

Пропускати кроки не можна: Крок 4 не має сенсу без Кроку 3, а Крок 10 без Кроків 7–9.

Словник незнайомих слів

Заведи файл `UNKNOWN.md` у корені проєкту. Кожне слово, конструкція чи математичний знак, який ти не можеш пояснити своїми словами, іде туди — не в голову, а у файл.

UNKNOWN.md

```
# Словник незнайомих слів

- [ ] tensor.detach()   — навіщо, якщо я і так не викликаю backward?
- [x] seed              — число, з якого генератор випадкових чисел відтворює ту саму
                          послідовність (Крок 2)
```

Цей файл має скорочуватись, а не рости.

Що потрібно перед стартом

Робота ведеться на Linux-ПК. Потрібні:

що	навіщо	перевірка
`git`	історія проєкту, гілки	`git --version`

що	навіщо	перевірка
`g++` 12+ або `clang++` 16+	C++-вузол	`g++ --version`
`cmake` 3.20+	збірка C++	`cmake --version`
Python 3.12	навчання, верифікація, атаки	`python3.12 --version`
`pip` / `venv`	ізольоване середовище	`python3.12 -m venv --help`

Відеокарта не обов'язкова. Еталонним пристроєм у всіх експериментах буде CPU; якщо в ПК є NVIDIA-карта, вона стане в пригоді пізніше як «інше залізо» для експерименту з хибними відхиленнями. Про це — у Кроці 4.

Що ми будуємо

Проблема одним абзацом

У Bitcoin вузли перебирають хеші. Робота захищає мережу, але поза мережею не має цінності. Ідея корисної роботи (Proof of Useful Work) — замінити перебір на задачу, розв'язок якої потрібен сам по собі. Навчання нейромережі підходить: воно важке, ділиться на порції, результат має цінність.

Але заміна ламає те, на чому тримався Bitcoin:

властивість	хешування	навчання
вартість виконання	висока	висока
вартість перевірки	одна хеш-операція	порівнянна з виконанням
детермінованість	повна	залежить від seed, порядку даних, заліза
унікальність результату	так	ні, валідних відповідей багато

Асиметрія «дорого виконати — дешево перевірити» зникає. Як її повернути — і є предметом роботи.

Життєвий цикл раунду

1. У ланцюгу лежить поточний стан моделі (точніше, його хеш; самі ваги — у сховищі).
2. Майнер бере цей стан і робить K кроків навчання на своїй порції даних, зберігаючи проміжний стан кожні k кроків.
3. Майнер формує блок: хеш нових ваг, хеші проміжних станів, досягнута метрика.
4. Валідатори перевіряють блок одним із механізмів A–D.
5. Блок прийнято — він іде в ланцюг, стан моделі оновлюється, майнер отримує нагороду. Відхилено — блок відкидається.
6. Наступний раунд починається з нового стану.

Через сотні раундів у ланцюгу накопичується історія навчання, а модель поступово покращується, не належачи жодному учаснику.

Чотири механізми верифікації

Це центральна частина дослідження. Аналогія — вчитель перевіряє розв'язану задачу.

	механізм	аналогія	ціна	що пропускає
A	метрика на випадковій підвибірці	подивитись лише на відповідь	дуже дешево	плагіат, чужі валідні ваги

	механізм	аналогія	ціна	що пропускає
B	перерахунок 1–2 переходів між контрольними точками	перевірити пару рядків розв'язку	$q \cdot k/K$ від роботи	підроблену траєкторію, якщо поріг невдалий
C	реплікація на n вузлів і порівняння	дати задачу трьом учням	n-кратна	змову реплік
D	узгодженість напрямку зміни ваг із градієнтом	чи логіка йде в правильний бік	дешево	точні значення ваг не перевіряє

Механізм B — це Proof of Learning з наукової літератури, і саме на нього є відомі атаки. Механізм D цікавий тим, що майже не залежить від заліза: він оцінює ефект оновлення, а не побітову рівність.

П'ять моделей атак

	атака	що робить недобросовісний вузол
1	ліниве повторення	не навчає; повертає попередній стан із малим шумом
2	плагіат	копіює результат іншого вузла і подає як свій
3	часткова робота	виконує 10% кроків, решту домальовує інтерполяцією
4	отруєння	навчає чесно, але на зіпсованих мітках, щоб погіршити спільну модель
5	підробка траєкторії	конструює послідовність контрольних точок, що проходить B без навчання

Без реалізованих атак робота не має що міряти. Атаки — не «додаток» до проєкту, це половина його наукової частини.

Що саме міряємо

метрика	означення	навіщо
detection rate	частка спійманих спроб шахрайства, окремо по кожній атаці	головна таблиця роботи
false rejection rate	частка чесних блоків, відхилених помилково	чесний вузол на іншому залізі дає інші ваги — це ціна перевірки
накладні витрати	час верифікації / час навчання	дорога перевірка знецінює корисну роботу
частка корисної роботи	яка частина витрачених обчислень дала прогрес моделі	головний аргумент «за» або «проти» підходу
точність моделі	точність за раундами проти централізованого навчання	чи працює система взагалі

Дві частини системи

частина	мова	що робить
вузол	C++	блок, хеш-ланцюг, P2P по TCP, правила консенсусу, серіалізація
воркер	Python + PyTorch	навчання, верифікація, атаки, метрики, графіки

ЧОМУ САМЕ ТАК

Чому не все на Python? Тому що мережева частина — це твоя сильна сторона і окремий розділ диплома; на C++ вона виглядає як справжній вузол, а не як скрипт.

Чому не все на C++? Тому що навчання нейромереж на C++ — це місяці роботи без наукової віддачі. PyTorch тут інструмент, а не предмет дослідження.

Зв'язка навмисно найпростіша: вузол запускає воркер як окремий процес і обмінюється з ним JSON-файлами. Ніяких вбудованих інтерпретаторів.

Правила протоколу, які вже прийняті

Ці рішення прийняті до написання коду, кожне закриває конкретну атаку. Повне обґрунтування — у `docs/design-decisions.md`; у гайді вони зустрінуться там, де їх треба реалізувати.

	правило	що закриває
P1	стартовий стан задає ланцюг, а не майнер	підміну ініціалізації
P2	`seed = SHA256(prev_hash ‖ miner_id ‖ round)`, з нього — порядок даних	плагіат, підбір вигідних даних
P3	усі гіперпараметри фіксовані в завданні; SGD без моментуму	атаку «нескінченно малих оновлень»
P4	у блоці лише хеші; ваги — у сховищі за хешем	роздування блоків
P5	перевірювані переходи валідатор обирає приватно	підбір блоку під передбачувану перевірку
P6	поріг відносний і відкалібрований на чесних вузлах	експлуатацію фіксованого порогу
P7	канонічний хеш ваг і канонічний заголовок блоку	розбіжність C++ і Python
P8	для реплікації — спільний seed і схема «спершу зобов'язання, потім розкриття»	копіювання між репліками
P11	підписів у прототипі немає; ID вузлів довірені	записується в обмеження роботи
P12	датасет публічний, канонічний, зафіксований хешем; порція — з seed	робить механізм B можливим взагалі

Структура проєкту

Один репозиторій, дві частини, спільна тека для ваг.

дерево проєкту

```
pouwchain/
├── CMakeLists.txt          # збірка C++-вузла
├── README.md
├── UNKNOWN.md              # словник незнайомих слів
├── .gitignore
├── node/                   # C++: вузол
│   ├── include/            # заголовки (.h)
│   ├── src/                # реалізація (.cpp)
│   └── tests/              # тести на золотих векторах
├── worker/                 # Python: навчання і верифікація
│   ├── worker.py           # точка входу: train | verify | attack
│   ├── model.py            # модель і дані
│   ├── training.py         # крок навчання, контрольні точки
│   ├── verify.py           # механізми A, B, C, D
│   ├── attacks.py          # атаки 1–5
│   ├── canonical.py        # канонічний хеш ваг, seed
│   └── tests/
├── sim/                    # Python: симулятор раундів без мережі
│   ├── simulate.py
│   └── configs/            # конфіги експериментів (JSON)
├── experiments/
│   ├── results/            # CSV з результатами (у git не кладемо)
│   └── plots/              # графіки для роботи
├── store/                  # ваги за хешем (у git не кладемо)
├── docs/                   # план, рішення, дослідження, контракт
└── guide/                  # цей гайд
```

ЧОМУ САМЕ ТАК

Чому `worker/` і `sim/` окремо? Воркер — це процес, який запускає C++-вузол, він нічого не знає про раунди й ланцюг. Симулятор — навпаки: він грає роль усієї мережі, але в одному процесі Python, без TCP. Саме симулятор дасть таблиці для розділу 4, і саме тому він не має залежати від того, чи працює мережа.

УВАГА

У `git` ніколи не потрапляють: `store/`, `experiments/results/`, `data/` (датасети), `build/`, `.venv/`. Ваги однієї моделі — це сотні мегабайтів; репозиторій із ними стає непридатним за тиждень.

Гілки й коміти

1. Гілка `main` завжди збирається і проходить тести.
2. Кожен крок гайда — окрема гілка: `step-03-train-block`. Завершив крок — зливаєш у `main`.
3. Коміти у форматі `тип(область): що зроблено`, наприклад `feat(worker): canonical weight hash`. Типи: `feat`, `fix`, `test`, `docs`, `refactor`, `exp`.

4. У повідомленні комміта пиши **чому**, а не **що**: «що» видно з коду.

Календар до 1 листопада

Сьогодні 16 вересня 2026. До прототипу — шість з половиною тижнів.

тиждень	дати	кроки гайда	год
1	16–20.09	Крок 0, Крок 1	8
2	21–27.09	Крок 2, Крок 3	10
3	28.09–04.10	Крок 4, Крок 5	10
4	05–11.10	Крок 6, Крок 7	10
5	12–18.10	Крок 8, Крок 9	10
6	19–25.10	Крок 10	10
7	26.10–01.11	Крок 11 і запас	7

КОНТРОЛЬНА ТОЧКА · 1 ЛИСТОПАДА 2026

Прототип вважається готовим, якщо одночасно виконано:

1. три вузли на C++ обмінюються блоками по TCP локально;
2. вузли навчають модель на MNIST і додають блоки в ланцюг;
3. працює механізм B (і як базова лінія — A);
4. реалізовано щонайменше одну атаку, і її видно в цифрах;
5. є перші числа: detection rate, FRR, накладні витрати, точність за раундами.

Якщо станом на 1 листопада мережі немає — прототипом вважається симулятор (Кроки 2–6) плюс C++-ланцюг без P2P, а мережа переїжджає у фазу 2. Обсяг ріжемо, терміни не рухаємо.

Крок 0. Середовище і скелет репозиторію

Мета кроку: порожній, але робочий проєкт: збирається C++-частина, запускається Python-частина, перший коміт у git.

Ідея

Цей крок здається формальністю, але він вирішує проблему, яка інакше з'їсть тиждень у березні: **відтворюваність**. Через пів року ти повернешся до експерименту і маєш отримати ті самі числа. Для цього версії інструментів фіксуються один раз і записуються у файл.

Друга причина — межі. Проєкт складається з двох частин різними мовами, і найгірше, що може статися, — це коли вони «трохи знають» одна про одну: Python пише файл, C++ його якось читає, а формат живе в головах. Тому структура тек задається зараз, а формат обміну — наступним кроком, ще до першого рядка логіки.

Нові слова

слово	що означає
`venv`	Ізольоване середовище Python: свої пакети, які не конфліктують із системними.
`requirements.txt`	Список пакетів із точними версіями. Головний файл відтворюваності.
`CMake`	Генератор збірки: описуєш проєкт один раз, а він створює правила компіляції.
`target`	Одиниця збірки в CMake: виконуваний файл або бібліотека.
`CMAKE_CXX_STANDARD`	Який стандарт мови вимагати. У нас 17.
`-Wall -Wextra`	Увімкнути попередження компілятора. Попередження тут — це майбутні баги.
`.gitignore`	Список того, що git має ігнорувати: збірка, дані, ваги.

Контракт

Набирай як є. Це інфраструктура, а не робота.

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.20)
project(pouwchain LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_EXPORT_COMPILE_COMMANDS ON)

add_compile_options(-Wall -Wextra -Wpedantic)

add_executable(node
    node/src/main.cpp
)
target_include_directories(node PRIVATE node/include)
```

node/src/main.cpp

```cpp
#include <iostream>

int main() {
    std::cout << "pouwchain node: alive\n";
    return 0;
}
```

requirements.txt

```
torch==2.14.0
torchvision==0.19.0
safetensors==0.4.5
numpy==2.1.3
matplotlib==3.9.2
```

> Точні номери версій підстав ті, що встановились у тебе (`pip freeze`). Важлива не конкретна версія, а те, що вона записана.

worker/worker.py

```python
"""Точка входу воркера. Поки що лише перевірка середовища."""
import sys
import torch


def main() -> int:
    print("torch", torch.__version__)
    print("cuda", torch.cuda.is_available())
    print("threads", torch.get_num_threads())
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

.gitignore

```
.venv/
build/
__pycache__/
data/
store/
experiments/results/
*.safetensors
compile_commands.json
```

Твоє завдання

ТВОЄ ЗАВДАННЯ

1. Створи теки за деревом із розділу «Структура проєкту» (порожні теки git не зберігає — постав у кожній файл `.gitkeep`).
2. Створи середовище і встанови пакети: `python3.12 -m venv .venv && source .venv/bin/activate && pip insta`

   Якщо в ПК немає NVIDIA-карти, став CPU-збірку torch — інструкція на pytorch.org.

3. Збери C++-частину: `cmake -B build -DCMAKE_BUILD_TYPE=Release && cmake --build build -j`.
4. Запусти обидві частини: `./build/node` і `python worker/worker.py`.
5. `git init`, перший коміт: `chore: project skeleton`.
6. Створи `UNKNOWN.md` і запиши туди все, чого не зміг пояснити в цьому кроці.

Перевірка

ПЕРЕДБАЧ ДО ЗАПУСКУ

Перед запуском `worker.py` запиши: що покаже `torch.cuda.is_available()` на твоєму ПК і скільки потоків покаже `torch.get_num_threads()`? Друге число з чимось пов'яжи — з чим саме?

ЕТАЛОН ДЛЯ САМОПЕРЕВІРКИ

```
pouwchain node: alive
torch 2.14.0
cuda False        # або True, якщо є NVIDIA-карта і CUDA-збірка torch
threads 8         # зазвичай = кількість фізичних ядер
```

Головне: обидві команди відпрацювали без помилок, а `git status` показує чисте дерево (усе зайве ігнорується).

САМОПЕРЕВІРКА

1. Навіщо `venv`, якщо пакети можна поставити системно?
2. Чому `build/` і `store/` не потрапляють у git, а `requirements.txt` — потрапляє?
3. Що станеться з експериментом у березні, якщо сьогодні не зафіксувати версію torch?

ЗЛАМАЙ НАВМИСНЕ

Експеримент 1. Прибери `-Wall -Wextra` з `CMakeLists.txt`, додай у `main.cpp` рядок `int x;` і використай `x` у виводі. Зберись із попередженнями і без них. Що компілятор сказав в одному випадку і промовчав у другому?

Експеримент 2. Деактивуй `venv` (`deactivate`) і запусти `python worker/worker.py`. Яку помилку побачив? Поясни, чому вона доводить, що середовище ізольоване.

Крок 1. Контракт між C++ і Python

Мета кроку: документ `docs/contract.md` і модуль `worker/canonical.py`, які однозначно визначають, що саме дві частини одна одній передають і як рахують хеші.

Ідея

Дві частини системи пишуться окремо і майже напевно в різні тижні. Якщо не домовитись про формат заздалегідь, ти отримаєш класичний баг інтеграції: C++ рахує хеш блоку від JSON-тексту, Python — від байтів ваг, обидва «працюють», а разом дають різні числа. Знайти це потім важко, бо помилки немає — є розбіжність.

Тому спершу контракт, і аж потім обидві половини.

Контракт складається з чотирьох частин:

1. **Канонічний хеш ваг.** Як з набору тензорів отримати 32 байти так, щоб будь-яка реалізація дала те саме.
2. **Канонічний заголовок блоку.** Точний бінарний розклад полів, від якого береться `block_hash`.
3. **Формат завдання й результату** (`task.json`, `result.json`) — те, чим обмінюються вузол і воркер.
4. **Сховище ваг** — тека, де файл називається своїм хешем.

> ЧОМУ САМЕ ТАК
>
> Чому хеш блоку рахується від бінарного розкладу, а не від JSON? Бо JSON не має канонічної форми: порядок ключів, пробіли, `1.0` проти `1`, екранування — усе це змінює байти, не змінюючи змісту. Хеш від такого тексту не відтворюється в іншій мові. Бінарний розклад із фіксованим порядком і фіксованим порядком байтів відтворюється завжди.
>
> Чому хеш ваг рахується не від файлу `.safetensors`? Бо в цьому форматі є поле метаданих, яке серіалізується без фіксованого порядку: ті самі ваги з тими самими метаданими можуть дати різні байти файлу. Перевірено експериментально, див. `docs/research/technical.md`.

Нові слова

слово	що означає
`канонічна серіалізація`	Одне-єдине представлення даних у байтах, однакове в будь-якій мові й на будь-якій машині.
`little-endian`	Порядок байтів, у якому молодший байт числа йде першим. Стандарт на x86 і arm.
`fixed-point`	Дробове число, збережене як ціле: точність 0.9123 → 912300 (множник 10^6). Уникаємо float у хешованих полях.

слово	що означає
`content-addressed store`	Сховище, де ім'я файлу — це хеш його вмісту. Той самий вміст завжди має ту саму адресу.
`golden vectors`	Кілька заздалегідь порахованих прикладів «вхід → очікуваний хеш», на яких обидві мови звіряються.
`struct.pack`	Модуль Python для перетворення чисел у байти із заданим порядком (`<` — little-endian).
`hashlib.sha256`	SHA-256 у Python; `.digest()` дає 32 байти, `.hexdigest()` — 64 символи.

Контракт: канонічний хеш ваг

Ваги моделі — це словник «ім'я тензора → масив чисел». Хеш береться так:

правило канонічного хешу ваг

```
Для кожного тензора, у порядку зростання імені (байтове порівняння ASCII):
    u32  len(name)          довжина імені в байтах, little-endian
    ...  name               саме ім'я, ASCII
    u8   dtype_tag          1 = float32 (інших поки немає)
    u8   ndim               кількість вимірів
    u32  dim[i]             кожен вимір, little-endian
    ...  raw                сирі байти тензора, little-endian, C-порядок

Усе це підряд подається в SHA-256. Результат — 32 байти, у JSON записуємо як 64 hex-символи.
```

> **УВАГА**
> Тензор має бути приведений до CPU і до безперервного розкладу в пам'яті (`.cpu().contiguous()`) перед тим, як брати його байти. Інакше «сирі байти» залежатимуть від того, на якому пристрої лежить тензор і чи робив ти перед цим `transpose`.

Контракт: заголовок блоку

Рівно 236 байтів, усі числа little-endian, у цьому порядку:

поле	тип	байтів	що це
`version`	u32	4	версія формату, поки 1
`height`	u64	8	номер блоку в ланцюгу, генезис = 0
`prev_hash`	32B	32	хеш заголовка попереднього блоку
`task_id`	32B	32	хеш опису задачі навчання
`round`	u64	8	номер раунду
`miner_id`	32B	32	SHA-256 від імені вузла
`timestamp`	u64	8	час у секундах Unix
`w_start`	32B	32	канонічний хеш ваг на початку
`w_end`	32B	32	канонічний хеш ваг у кінці
`ckpt_root`	32B	32	хеш списку хешів контрольних точок

поле	тип	байтів	що це
`K`	u32	4	скільки кроків навчання в блоці
`k`	u32	4	через скільки кроків зберігається точка
`metric_micro`	i64	8	заявлена точність $\times 10^6$

`block_hash` = SHA-256 від цих 236 байтів.

`ckpt_root` = SHA-256 від конкатенації всіх хешів контрольних точок підряд (C_0 … C_m, по 32 байти кожен).

> ЧОМУ САМЕ ТАК
>
> Чому точність зберігається як ціле число $\times 10^6$? Бо `float` у хешованому полі — це джерело розбіжностей: 0.9123 у різних мовах може дати різні байти, а округлення при друку приховує різницю. Ціле число таких сюрпризів не має.
>
> Чому `miner_id` — це хеш імені, а не саме ім'я? Бо поле фіксованої довжини не потребує окремого поля довжини, і заголовок лишається рівно 236 байтів завжди.

Контракт: обмін вузол ↔ воркер

Вузол пише `task.json`, запускає `python worker/worker.py <шлях до task.json>` і читає `result.json` із теки, вказаної в завданні.

task.json — навчання

```
{
  "op": "train",
  "task_id": "9f2c...",
  "round": 17,
  "miner_id": "4ab1...",
  "prev_block_hash": "00c8...",
  "w_start": "7d31...",
  "K": 500,
  "k": 25,
  "batch_size": 64,
  "lr": 0.01,
  "dataset": "mnist",
  "store_dir": "store",
  "out": "run/result.json"
}
```

result.json — навчання

```
{
  "ok": true,
  "w_end": "b0e4...",
  "ckpt_hashes": ["7d31...", "...", "b0e4..."],
  "metric_micro": 912300,
  "steps": 500,
  "seconds": 6.42
}
```

task.json — перевірка B

```
{
  "op": "verify_b",
```

```
  "task_id": "9f2c...",
  "round": 17,
  "miner_id": "4ab1...",
  "prev_block_hash": "00c8...",
  "ckpt_from": "7d31...",
  "ckpt_to": "a91f...",
  "step_from": 125,
  "k": 25,
  "batch_size": 64,
  "lr": 0.01,
  "dataset": "mnist",
  "threshold_rule": {"kind": "relative", "alpha": 0.3},
  "store_dir": "store",
  "out": "run/verify.json"
}
```

verify.json — результат перевірки

```
{
  "ok": true,
  "verdict": "accept",
  "distance": 0.0000016,
  "threshold": 0.0021,
  "seconds": 0.31
}
```

> Поле `ok` означає «воркер відпрацював без збою», а `verdict` — «блок правильний чи ні». Це різні речі: якщо воркер упав, вузол не має права вважати блок відхиленим.
>
> Значення `alpha` тут — заповнювач формату, а не готове число. Поріг калібрується вимірюванням у Кроці 4; якщо взяти `alpha` завеликим (скажімо, 1.0), перевірка починає приймати будь-що.

Контракт: сховище ваг

сховище

```
store/<канонічний хеш ваг у hex>.safetensors
```

Ваги через мережу не передаються. У блоці лише хеші; у прототипі всі вузли працюють на одній машині й бачать одну теку `store/`.

> У ТЕКСТ РОБОТИ
>
> Ці чотири контракти майже без змін підуть у розділ 3 роботи («Програмна реалізація»), підрозділ «Формат обміну між компонентами». Таблиця заголовка блоку — готовий рисунок.

Скелет

worker/canonical.py — сигнатури

```
from typing import Dict
import torch

def canonical_weight_hash(state: Dict[str, torch.Tensor]) -> bytes:
```

```
    """32 байти SHA-256 за правилом канонічного хешу ваг."""

def derive_seed(prev_block_hash: bytes, miner_id: bytes, round_no: int) -> int:
    """seed = перші 8 байтів SHA-256(prev ‖ miner_id ‖ round) як ціле."""

def block_header_bytes(header: dict) -> bytes:
    """Рівно 236 байтів за таблицею заголовка."""
```

worker/canonical.py · тіла написати самому

```
import hashlib
import struct
from typing import Dict

import torch


def canonical_weight_hash(state: Dict[str, torch.Tensor]) -> bytes:
    # 1. Заведи об'єкт hashlib.sha256().
    # 2. Пройдись по sorted(state.keys()) — саме sorted, порядок словника не
    #     гарантований.
    # 3. Для кожного тензора:
    #    - t = state[name].detach().cpu().contiguous()
    #    - перевір, що t.dtype == torch.float32, інакше кинь ValueError
    #    - згодуй у хеш: довжину імені (struct.pack("<I", ...)), саме ім'я
    #     (name.encode()),
    #      байт dtype_tag = 1, байт ndim, кожен вимір як "<I"
    #    - згодуй сирі байти: t.numpy().tobytes()
    # 4. Поверни .digest() — 32 байти, не hexdigest.


def derive_seed(prev_block_hash: bytes, miner_id: bytes, round_no: int) -> int:
    # 1. Перевір довжини: обидва хеші мають бути по 32 байти.
    # 2. Порахуй SHA-256 від конкатенації prev ‖ miner_id ‖ struct.pack("<Q", round_no).
    # 3. Візьми перші 8 байтів результату і перетвори на ціле: struct.unpack("<Q",
    #     ...)[0].


def block_header_bytes(header: dict) -> bytes:
    # 1. Склади список частин у порядку таблиці заголовка.
    # 2. Числа — через struct.pack: "<I" для u32, "<Q" для u64, "<q" для i64.
    # 3. Хеші — через bytes.fromhex(...), кожен рівно 32 байти (перевіряй!).
    # 4. Склей усе і переконайся, що довжина == 236, інакше кинь ValueError.
    # 5. Поверни байти (не хеш — хеш візьме той, хто викликає).
```

ПІДКАЗКИ

Рівень 1. Почни з `derive_seed` — вона найкоротша і перевіряється одразу. `canonical_weight_hash` роби другою, `block_header_bytes` третьою.

Рівень 2. `struct.pack("<I", n)` — чотири байти little-endian; `"<Q"` — вісім беззнакових; `"<q"` — вісім знакових. `h.update(b)` додає байти в хеш, `h.digest()` завершує. Для тензора: `t.numpy().tobytes()` дає сирі байти в C-порядку.

Рівень 3. Каркас першої функції:

```
h = hashlib.sha256()
for name in sorted(state):
    t = state[name].detach().cpu().contiguous()
    nb = name.encode("ascii")
    h.update(struct.pack("<I", len(nb)) + nb)
    h.update(bytes([1, t.dim()]))
    for d in t.shape:
```

```
        h.update(struct.pack("<I", d))
    h.update(t.numpy().tobytes())
return h.digest()
```

worker/tests/test_canonical.py

```python
import hashlib
import torch
from worker.canonical import canonical_weight_hash, derive_seed, block_header_bytes


def test_sha256_sanity():
    assert hashlib.sha256(b"abc").hexdigest().startswith("ba7816bf")


def test_order_independent():
    a = {"w": torch.zeros(2, 3), "b": torch.ones(3)}
    b = {"b": torch.ones(3), "w": torch.zeros(2, 3)}
    assert canonical_weight_hash(a) == canonical_weight_hash(b)


def test_value_sensitive():
    a = {"w": torch.zeros(2, 2)}
    b = {"w": torch.zeros(2, 2)}
    b["w"][0, 0] = 1e-7
    assert canonical_weight_hash(a) != canonical_weight_hash(b)


def test_seed_binds_to_miner():
    prev, m1, m2 = bytes(32), b"\x01" * 32, b"\x02" * 32
    assert derive_seed(prev, m1, 7) != derive_seed(prev, m2, 7)
    assert derive_seed(prev, m1, 7) != derive_seed(prev, m1, 8)
    assert derive_seed(prev, m1, 7) == derive_seed(prev, m1, 7)


def test_header_length():
    header = dict(
        version=1, height=0, prev_hash="00" * 32, task_id="11" * 32, round=0,
        miner_id="22" * 32, timestamp=0, w_start="33" * 32, w_end="44" * 32,
        ckpt_root="55" * 32, K=500, k=25, metric_micro=912300,
    )
    assert len(block_header_bytes(header)) == 236
```

Твоє завдання

ТВОЄ ЗАВДАННЯ

1. Перенеси всі чотири контракти в `docs/contract.md` — це твій документ, на нього посилатимуться обидві частини.
2. Напиши тіла трьох функцій у `worker/canonical.py`.
3. Набери тести як є і запусти: `python -m pytest worker/tests -q` (`pip install pytest`, додай у `requirements.txt`).
4. Створи файл золотих векторів `docs/golden.json`: візьми заголовок із тесту, порахуй `SHA-256` від його 236 байтів і запиши hex. Це число потім має отримати C++.
5. Коміт: `feat(worker): canonical hashing and contract`.

Перевірка

ПЕРЕДБАЧ ДО ЗАПУСКУ

Перед запуском тестів дай відповіді:

1. Чи зміниться хеш ваг, якщо перейменувати тензор `w` на `a`, не чіпаючи чисел?
2. Чи зміниться хеш, якщо той самий тензор порахувати на GPU замість CPU (припустимо, числа побітово ті самі)?
3. У тесті `test_value_sensitive` змінюється одне число на 10^{-7}. Скільки приблизно символів хеша зміниться?

ЕТАЛОН ДЛЯ САМОПЕРЕВІРКИ

Усі п'ять тестів проходять. Крім того, мають виконуватись властивості:

1. `sha256(b"abc")` починається з `ba7816bf` — це всесвітньо відома константа, будь-яка правильна реалізація дає її;
2. хеш ваг не залежить від порядку ключів у словнику;
3. зміна одного числа на 10^{-7} змінює приблизно половину бітів хеша;
4. `len(block_header_bytes(...)) == 236` рівно, без «приблизно».

САМОПЕРЕВІРКА

1. Чому `sorted(state)`, а не просто `for name in state`? Що станеться, якщо в C++ порядок виявиться інший?
2. Навіщо `.contiguous()` перед `.numpy().tobytes()`?
3. `derive_seed` прив'язує seed до `miner_id`. Поясни в двох реченнях, як саме це ламає атаку плагіату.
4. Чому в заголовку немає поля «список хешів контрольних точок», а є лише `ckpt_root`?
5. Що зламається, якщо хтось у C++ запише `K` як u64 замість u32?

ЗЛАМАЙ НАВМИСНЕ

Експеримент 1. Прибери `sorted` у `canonical_weight_hash` і запусти `test_order_independent` кілька разів на різних словниках. Іноді проходить, іноді ні. Чому «іноді»?

Експеримент 2. Заміни `struct.pack("<I", ...)` на `">I"` (big-endian) лише в одному місці. Які тести впадуть, а які ні? Поясни, чому невідповідність порядку байтів — найпідступніший різновид помилки інтеграції.

Експеримент 3. Зроби `metric` звичайним float у заголовку замість `metric_micro`. Порахуй заголовок для 0.1 + 0.2 і для 0.3. Хеші однакові?

У ТЕКСТ РОБОТИ

Результат цього кроку — підрозділ «Канонічне подання даних і формат обміну» розділу 3, разом із таблицею заголовка і правилом хешування ваг. Запиши також **чому** JSON не годиться для хешування: на захисті це хороша ілюстрація того, що ти розумієш різницю між форматом і його канонічною формою.

Крок 2. PyTorch рівно стільки, скільки треба

Мета кроку: модуль `worker/model.py` — модель, дані, детерміновані батчі, точність — і доведення, що два запуски з тим самим seed на твоїй машині дають побітово однакові ваги.

Ідея

PyTorch тут інструмент, а не предмет дослідження, тому вчимо вузьку смугу бібліотеки: рівно те, без чого не можна написати навчальний блок і перевірку. Регуляризація, планувальники, аугментація, `DataLoader` не використовуються, і кожне таке «ні» має причину.

Причина одна: **детермінованість**. Звичайний ML-код має бути точним, наш — відтворюваним. Якщо валідатор повторює той самий крок від тієї самої точки на тій самій машині, він мусить отримати ті самі байти. Точність моделі другорядна: 90 % на MNIST достатньо, щоб бачити прогрес по раундах. А кожне джерело прихованої випадковості — це майбутній хибно відхилений чесний блок, тобто зіпсована метрика FRR у розділі 4 роботи.

Що таке градієнтний спуск, одним абзацом

Модель — це функція з великою кількістю ручок (ваг). Подаєш картинку — отримуєш десять чисел, по одному на цифру. Функція втрат бере ці десять чисел і правильну відповідь і повертає одне число: наскільки сильно модель помилилась. Далі працює механічна процедура: для кожної ручки рахується, у який бік її крутнути, щоб втрати **зросли** (це і є градієнт), після чого кожна ручка крутиться в протилежний бік на маленький крок:

$$W \leftarrow W - \eta \cdot \nabla L$$

Тут η («ета», вона ж `lr`) — розмір кроку. Повторюєш тисячі разів на різних порціях даних, і втрати падають. Магії немає: є функція, є її похідна, є маленький крок проти неї. Важливо інше: процедура **повністю детермінована**, якщо зафіксувати початкові ваги, порядок даних і арифметику. Перше й друге фіксує протокол (P1 і P2), третє — налаштування PyTorch і однакове залізо. Саме на цьому тримається механізм верифікації B.

шар	вихід	параметрів
вхід	$1 \times 28 \times 28$	—
`Conv2d(1, 16, 3)`	$16 \times 26 \times 26$	160
`ReLU` + `MaxPool2d(2)`	$16 \times 13 \times 13$	0
`Conv2d(16, 32, 3)`	$32 \times 11 \times 11$	4 640
`ReLU` + `MaxPool2d(2)`	$32 \times 5 \times 5$	0
`Flatten` → `Linear(800, 100)` + `ReLU`	100	80 100
`Linear(100, 10)`	10	1 010
разом		**85 910**

ЧОМУ САМЕ ТАК

Знизу розмір моделі обмежує змістовність: вона має реально вчитись, щоб прогрес по раундах було видно на графіку. Зверху — три речі: файл ваг має бути малим (точок на блок буде 20+), крок має займати мілісекунди (експеримент — це тисячі вузло-раундів), перевірка B має бути дешевою (її вартість теж результат роботи). Масштаб моделі не є предметом дослідження: велика модель нічого не додасть до висновків про верифікацію, але з'їсть час.

Нові слова

слово	що означає
`tensor`	Багатовимірний масив чисел. Знає свій `dtype` (`float32` — рахуємо, `uint8` — зберігаємо, `int64` — мітки), `shape` і `device`.
`.detach()`	Копія тих самих чисел, відчеплена від графа обчислень: її не тягне за собою backward.
`nn.Module`	Базовий клас моделі: зберігає параметри, вміє `state_dict()`, викликається як функція.
`forward / logits`	Прямий прохід. На виході десять сирих чисел (логіти), ще не ймовірності.
`F.cross_entropy`	Функція втрат: бере логіти й мітки, повертає одне число.
`loss.backward()`	Порахувати градієнт втрат по всіх параметрах і покласти в `.grad` кожного.
`zero_grad / step`	Обнулити градієнти (PyTorch їх **додає**) і зробити крок оптимізатора.
`torch.no_grad()`	Блок, у якому граф не будується: швидше і без пам'яті. Для вимірювання точності.
`seed`	Число, з якого генератор випадкових чисел відтворює ту саму послідовність.
`state_dict()`	Словник «ім'я параметра → тензор». Саме його ми хешуємо і зберігаємо.

Контракт

`worker/model.py` — п'ять функцій. Імена й сигнатури фіксовані: на них спираються Кроки 3 і 4.

`worker/model.py — сигнатури`

```python
def build_model(seed: int) -> torch.nn.Module:
    """CNN за таблицею шарів. Ініціалізація детермінована від seed."""

def load_mnist(root: str) -> dict:
    """Повертає {"X_train": uint8 N x 1 x 28 x 28, "y_train": int64 N,
                 "X_test": ..., "y_test": ...}. Завантажує раз, далі — з кешу."""

def to_float(X: torch.Tensor) -> torch.Tensor:
    """uint8 -> float32, нормалізація за фіксованими константами."""

def batch_indices(seed: int, step: int, batch_size: int, n: int) -> torch.Tensor:
    """Індекси батча номер `step` — детерміновано від seed."""

def accuracy(model, X, y, idx=None) -> float:
    """Частка правильних відповідей на X[idx] (або на всьому X)."""
```

правило нормалізації і правило батчів

```
x_float = (x_uint8 / 255.0 - 0.1307) / 0.3081

perm  = randperm(n, generator=Generator().manual_seed(seed))
start = (step * batch_size) % n
батч  = perm[start : start + batch_size],  з обгортанням через кінець
```

`0.1307` і `0.3081` — стандартні середнє і відхилення MNIST. Обгортання: якщо `start + batch_size > n`, беруться `perm[start:]` і початок `perm[...]` до потрібної довжини; довжина батча завжди рівно `batch_size`.

ЧОМУ САМЕ ТАК

Чому нормалізація — частина контракту, а не деталь реалізації. Перевірка B звіряє байти: якщо валідатор поділить на 255, а майнер ні, ваги розійдуться на порядки і чесний блок буде відхилено. Усе, що впливає на числа, має бути в контракті.

Чому порядок даних — чиста функція від seed, а не звичайне перемішування. Перше: валідатор мусить повторити крок номер 137 точно так, як його зробив майнер, а для цього знати, які саме 64 картинки були в батчі 137 — не «приблизно ті самі», а саме ті. Якби порядок був випадковим, його довелося б передавати в блоці: 500 кроків по 64 індекси це 32 тисячі чисел, а як функція від seed він займає нуль байтів. Друге і важливіше: майнер не має права обирати свої дані, інакше з'являється атака — перебрати перестановки й узяти ту, де метрика краща. Правило P2 закриває це тим, що seed виводиться з `prev_block_hash`, `miner_id` і номера раунду: майнер не впливає на жоден із трьох доданків.

Звідси технічне правило: у `batch_indices` заборонено викликати `torch.rand*` без явного генератора. Глобальний генератор — спільний стан, а спільний стан ламає відтворюваність мовчки.

УВАГА

`DataLoader` не використовується ніде: він тасує дані власним генератором, тягне робочі процеси, а порядок батчів залежить від `num_workers` — рівно те джерело недетермінованості, яке ми ловимо як помилку. Увесь MNIST у `uint8` — це 47 МБ: він просто лежить у тензорі, а батч є зрізом.

Шапка `worker/model.py` набирається як є, **до** решти коду:

worker/model.py — шапка

```
import os
os.environ.setdefault("CUBLAS_WORKSPACE_CONFIG", ":4096:8")

import torch

torch.use_deterministic_algorithms(True)
torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False

METRIC_SAMPLE = 2000        # розмір підвибірки для заявленої метрики
MNIST_MEAN, MNIST_STD = 0.1307, 0.3081
```

прапорець	що буде без нього
`use_deterministic_algorithms`	той самий вхід дає різні останні біти: порядок додавання не фіксований
`cudnn.deterministic`	згортка на CUDA може обрати недетермінований варіант
`cudnn.benchmark = False`	cuDNN бере найшвидшу реалізацію **на цьому запуску**; при іншому завантаженні GPU вибір інший, а отже інші біти
`CUBLAS_WORKSPACE_CONFIG`	паралельні потоки CUDA дають різні суми

УВАГА

Змінна `CUBLAS_WORKSPACE_CONFIG` читається при завантаженні CUDA, тому `os.environ` треба чіпати **до** `import torch` — рядки місцями не міняти. Це рідкісний випадок, коли побічний ефект при імпорті виправданий: детермінованість не можна забути увімкнути в одному місці з трьох.

УВАГА

Оптимізатор — звичайний SGD без моментуму. Не Adam, не `momentum=0.9`. Причина не в якості навчання: моментум і Adam мають **власний стан** (накопичену швидкість, ковзні середні), який впливає на наступний крок так само, як ваги. Щоб валідатор міг повторити перехід, цей стан довелося б класти в кожну контрольну точку, хешувати й зберігати — точка виросла б утричі, а контракт на цілий розділ. Зі звичайним SGD стан порожній: наступні ваги залежать лише від поточних ваг, батча і η. Це рішення P3, і воно ж закриває атаку «нескінченно малих оновлень».

Скелет

`worker/model.py · тіла написати самому`

```python
import torch
import torch.nn as nn
import torch.nn.functional as F


class SmallCNN(nn.Module):
    def __init__(self):
        # 1. Виклич super().__init__().
        # 2. Заведи шари як поля: conv1, conv2, fc1, fc2 за таблицею шарів.
        #    Пулінг і ReLU стану не мають — бери їх з F у forward.

    def forward(self, x):
        # 1. conv1 -> relu -> max_pool2d(2), потім conv2 -> relu -> max_pool2d(2)
        # 2. розгорни в плаский вигляд: x.flatten(1)  (перший вимір — батч!)
        # 3. fc1 -> relu -> fc2
        # 4. Поверни логіти, без softmax: його зробить cross_entropy.


def build_model(seed: int) -> nn.Module:
    # 1. torch.manual_seed(seed) — саме перед створенням моделі.
    # 2. Створи SmallCNN() і поверни. Модель лишається на CPU: пристрій обирає той,
    #    хто викликає.
```

```python
def load_mnist(root: str) -> dict:
    # 1. Шлях до кешу <root>/mnist_cache.pt; якщо є — torch.load і повернути.
    # 2. Інакше torchvision.datasets.MNIST(root, train=True/False, download=True).
    # 3. З кожного набору взяти .data (uint8 N x 28 x 28) і .targets (int64),
    #    додати вимір каналу через .unsqueeze(1).
    # 4. Скласти словник із чотирьох тензорів, torch.save і повернути.


def to_float(X: torch.Tensor) -> torch.Tensor:
    # 1. Перевір, що dtype == torch.uint8, інакше ValueError.
    # 2. Переведи у float32 і застосуй правило нормалізації з контракту.


def batch_indices(seed: int, step: int, batch_size: int, n: int) -> torch.Tensor:
    # 1. perm = torch.randperm(n, generator=...) з ЯВНИМ генератором від seed.
    # 2. start = (step * batch_size) % n.
    # 3. Якщо батч не виходить за кінець — поверни зріз.
    # 4. Інакше склей perm[start:] з perm[:скільки бракує] через torch.cat.
    # 5. Перевір, що довжина результату рівно batch_size.


def accuracy(model, X, y, idx=None) -> float:
    # 1. Якщо idx не None — візьми X[idx], y[idx].
    # 2. Під torch.no_grad(): прогони to_float(X) через модель, візьми argmax по
    #     вимірі 1.
    # 3. Поверни частку правильних як звичайний float (не тензор!).
```

ПІДКАЗКИ

Рівень 1. Порядок: `to_float` → `batch_indices` → `SmallCNN` → `accuracy` → `load_mnist`. Форму виходу згортки перевіряй не в голові, а `print(x.shape)` усередині `forward`.

Рівень 2. `nn.Conv2d(1, 16, 3)`, `nn.Linear(800, 100)`; у `forward` — `F.relu(...)`, `F.max_pool2d(x, 2)`. Генератор: `g = torch.Generator(); g.manual_seed(seed)`. Частка правильних: `(pred == y).float().mean().item()`. У `torchvision.datasets.MNIST` потрібні поля `.data` і `.targets`, а не індексація датасету: вона віддає PIL-картинки.

Рівень 3. Розгортання перед першим `Linear` — місце, де найчастіше плутають виміри:

```
x = F.max_pool2d(F.relu(self.conv2(x)), 2)   # N x 32 x 5 x 5
x = x.flatten(1)                             # N x 800
```

`flatten(1)` зберігає вимір батча; `flatten()` без аргументу склеїть увесь батч в один вектор, і модель мовчки почне вчити нісенітницю.

Тепер сам цикл навчання. У Кроці 3 він переїде в `train_block`, але спершу напиши його руками — один раз, щоб побачити всі п'ять рядків.

experiments/smoke.py · тіла написати самому

```
"""Разовий скрипт: 500 кроків SGD, друк втрат кожні 50 кроків.
У протоколі участі не бере."""

# 1. Завантаж дані (load_mnist) і побудуй модель (build_model) з seed = 12345.
# 2. Оптимізатор: torch.optim.SGD(model.parameters(), lr=0.01).
#    Аргумент momentum не передавати взагалі.
# 3. Цикл по step від 0 до 499:
#    - idx = batch_indices(seed, step, 64, n_train)
#    - xb = to_float(X_train[idx]), yb = y_train[idx]
#    - прямий прохід, втрати F.cross_entropy(logits, yb)
```

```
#    - обнулити градієнти, backward, step  (саме в цьому порядку!)
#    - кожні 50 кроків друкувати step і loss.item()
# 4. Після циклу надрукуй accuracy на перших 2000 прикладах тесту
#    і канонічний хеш ваг (Крок 1), перші 16 символів.
```

ПІДКАЗКИ

Рівень 1. Помилка про розміри в `cross_entropy` майже завжди означає, що логіти мають форму не `(N, 10)`. Друкуй `logits.shape` перед викликом. `loss.item()` дістає число з тензора; без `.item()` ти тримаєш живим граф обчислень.

Рівень 2. Потрібні рівно п'ять дій на крок: прямий прохід, втрати, обнулення градієнтів, `backward`, `step`. `opt.zero_grad(set_to_none=True)` трохи швидший за звичайний.

Рівень 3. Ядро кроку, і порядок у ньому не довільний:

```
logits = model(xb)
loss = F.cross_entropy(logits, yb)
opt.zero_grad(set_to_none=True)
loss.backward()
opt.step()
```

`zero_grad` — до `backward`, `step` — після. Переставиш — отримаєш експеримент 4 з розділу «Зламай навмисне».

Твоє завдання

ТВОЄ ЗАВДАННЯ

1. Набери шапку `worker/model.py` як є (порядок рядків важливий) і напиши тіла п'яти функцій.
2. Запусти `load_mnist` і надрукуй `int(X_train.sum())`. Запиши число в `docs/contract.md`: воно однакове на будь-якій машині, і це найдешевша перевірка «у нас однакові дані».
3. Напиши `experiments/smoke.py`, прожени 500 кроків, збережи вивід у файл. Запусти двічі поспіль і порівняй надруковані хеші ваг.
4. Набери тести як є і запусти: `python -m pytest worker/tests -q`. Додай від себе два: на обгортання батча через кінець і на те, що `to_float` відхиляє не-`uint8`.
5. Додай у `docs/contract.md` підрозділ «Детермінованість»: чотири прапорці, правило батчів, правило нормалізації, «SGD без моментуму».
6. Коміт: `feat(worker): model, deterministic batching, metrics`.

worker/tests/test_model.py

```
import torch
from worker.model import build_model, to_float, batch_indices
from worker.canonical import canonical_weight_hash


def test_param_count_and_shape():
    m = build_model(1)
    assert sum(p.numel() for p in m.parameters()) == 85910
    assert m(torch.zeros(7, 1, 28, 28)).shape == (7, 10)
```

```
def test_same_seed_same_init():
    a = build_model(42).state_dict()
    b = build_model(42).state_dict()
    c = build_model(43).state_dict()
    assert canonical_weight_hash(a) == canonical_weight_hash(b)
    assert canonical_weight_hash(a) != canonical_weight_hash(c)


def test_batch_is_pure_function():
    i1 = batch_indices(7, 3, 64, 1000)
    i2 = batch_indices(7, 3, 64, 1000)
    i3 = batch_indices(8, 3, 64, 1000)
    assert torch.equal(i1, i2)
    assert not torch.equal(i1, i3)
    assert i1.shape == (64,) and len(set(i1.tolist())) == 64
    assert set(i1.tolist()).isdisjoint(batch_indices(7, 4, 64, 1000).tolist())
```

Перевірка

ПЕРЕДБАЧ ДО ЗАПУСКУ

1. Яке значення втрат буде на кроці 0? Підказка: модель ще нічого не знає, тому дає всім десяти класам приблизно рівні шанси. Чому з цього випливає конкретне число?
2. Два запуски `smoke.py` підряд дадуть однакові хеші ваг? А якщо між ними перезавантажити комп'ютер? А якщо запустити на іншому ПК?
3. `batch_indices(7, 0, 64, 60000)` і `batch_indices(7, 1, 64, 60000)` — чи можуть мати спільні індекси?

ЕТАЛОН ДЛЯ САМОПЕРЕВІРКИ

Числа залежать від машини, тому звіряйся з властивостями.

1. **Старт втрат.** На кроці 0 втрати близькі до $\ln 10 \approx 2.30$: модель рівноймовірно «вгадує» один із 10 класів, а перехресна ентропія рівномірного розподілу по 10 класах дорівнює $\ln 10$. Бачиш 0.2 або 15 — щось не так із нормалізацією або з мітками.
2. **Спад.** За кілька сотень кроків втрати опускаються нижче за 0.5. Падіння нерівне: окремі батчі дають сплески вгору — нормально, кожен крок бачить лише 64 приклади.
3. **Точність.** Після 500 кроків (приблизно пів проходу по MNIST) точність на тесті десь 85–95 %. Менше 70 % — шукай помилку, а не «недонавчання».
4. **Детермінованість.** Два запуски на одній машині дають **побітово** однакові ваги: хеші збігаються символ у символ, `torch.equal(w1, w2)` дає `True`. Не «майже» — однакові.
5. **Час.** 500 кроків на CPU — одиниці секунд. Для орієнтиру: на іншій машині (Apple M5, CPU) тисяча кроків цієї моделі зайняла близько 5 секунд. Важливо, щоб твоє число не стрибало вдвічі між запусками.

САМОПЕРЕВІРКА

1. Чому початкові втрати дорівнюють саме $\ln 10$, а не, скажімо, 1.0?
2. Що конкретно накопичиться, якщо прибрати `zero_grad`, і чому це не видно одразу?

3. Чому `batch_indices` не має права використати `torch.randperm(n)` без генератора, хоча результат виглядатиме так само випадково?
4. Ти прибрав `torch.use_deterministic_algorithms(True)`, і тести все одно проходять. Чи означає це, що прапорець не потрібен?
5. Чому в моделі немає жодного `Dropout` і жодної `BatchNorm`? Назви дві різні причини.

Зламай навмисне

ЗЛАМАЙ НАВМИСНЕ

Експеримент 1. Прибери seed. Закоментуй `torch.manual_seed(seed)` у `build_model`, запусти `smoke.py` двічі. Чи збігаються хеші ваг? Чи збігається **точність**? Друга відповідь важливіша: поясни, чому метрика виглядає однаково, а ваги різні, і що це означає для механізму A.

Експеримент 2. Зміни кількість потоків. Додай `torch.set_num_threads(1)`, запиши хеш; потім `torch.set_num_threads(4)` і порівняй. На частині машин хеші збігаються, на частині ні (`pytorch#88718`). Твій результат — факт про **твоє** залізо, і його треба записати в `docs/research/technical.md`: якщо потоки впливають, у тебе безкоштовно з'явився другий «пристрій» для експерименту з FRR у Кроці 4.

Експеримент 3. Один і той самий батч. Заміни `batch_indices(seed, step, ...)` на `batch_indices(seed, 0, ...)`: усі 500 кроків бачать одні й ті самі 64 картинки. Що станеться з втратами? А з точністю на тесті?

Експеримент 4. Забудь `zero_grad`. Прибери рядок, прожени 100 кроків, опиши, як розходяться втрати, і поясни через $W \leftarrow W - \eta \cdot \nabla L$: чим фактично стає η, коли градієнти сумуються від кроку до кроку.

Крок 3. Навчальний блок

Мета кроку: `train_block` — функція, що з одного стану моделі робить блок роботи: K кроків, контрольні точки кожні k кроків, метрика. Плюс сховище ваг і робоча команда `python worker/worker.py task.json`.

Ідея

Досі був просто цикл навчання. Тепер він стає **одиницею роботи протоколу** — тим, що майнер пред'являє мережі, а валідатор перевіряє. Змінюється не математика, а обов'язки.

звичайне навчання	навчальний блок
важливий кінцевий результат	важлива вся траєкторія: її треба буде довести
проміжні стани викидаються	проміжні стани — це і є доказ роботи
гіперпараметри обирає автор	гіперпараметри приходять у завданні (P3)
порядок даних випадковий	порядок даних виведений із seed (P2)
результат — число точності	результат — хеші, метрика і час

Контрольні точки $C_0, C_1, \ldots, C_m$ — знімки ваг через кожні k кроків. C_0 — стан **до** першого кроку, тобто рівно `w_start` із завдання; C_m — стан після останнього кроку, тобто `w_end`. Разом їх $K/k + 1$.

ЧОМУ САМЕ ТАК

Навіщо точки, якщо можна перевірити кінцевий результат? Не можна — у цьому вся проблема роботи. З кінцевих ваг не видно, чи їх отримано навчанням: це просто набір чисел. Валідатор може або повторити весь шлях (дорого, і — як побачимо в Кроці 4 — ще й марно на іншому залізі), або не перевіряти нічого. Контрольні точки розрізають одну дорогу перевірку на K/k дешевих: валідатор бере пару сусідніх точок, робить k кроків і звіряє. Вартість — k/K від роботи майнера замість 100 %. Це і є Proof of Learning. Ціна: майнер зобов'язаний зберегти $K/k + 1$ файлів на блок (при $K = 500$, $k = 25$ — 21 файл по 344 КБ).

Чому C_0 дублює `w_start`? Щоб перевірка першого переходу нічим не відрізнялася від решти: валідатор бере пару (C_j, C_{j+1}) і не має знати, чи $j = 0$. Заодно це безкоштовна перевірка цілісності: якщо `ckpt_hashes[0]` не дорівнює `w_start` із заголовка, блок відхиляється за одне порівняння рядків, ще до обчислень.

Нові слова

слово	що означає
`контрольна точка`	Знімок ваг через кожні k кроків. У блок іде лише хеш, ваги — у сховище.
`траєкторія`	Уся послідовність станів моделі від C_0 до C_m.

слово	що означає
`safetensors`	Формат зберігання тензорів: без виконання коду при читанні, на відміну від pickle.
`content-addressed`	Ім'я файлу — хеш його вмісту; той самий вміст завжди має ту саму адресу. Звідси ідемпотентність: зберегти ті самі ваги двічі — це один файл.
`perf_counter`	Монотонний час високої роздільності. Для вимірів, не `time.time()`.

Контракт

worker/training.py і worker/storage.py — сигнатури

```
def train_block(state: dict, seed: int, K: int, k: int, lr: float,
                batch_size: int, data: dict) -> tuple[dict, list[dict], float]:
    """K кроків SGD від state. Повертає (кінцевий стан, [C_0..C_m], метрика)."""

def save_state(state: dict, store_dir: str) -> str:
    """Зберігає у store_dir/<hex канонічного хешу>.safetensors, повертає hex.
       Якщо файл уже є — нічого не переписує."""

def load_state(hash_hex: str, store_dir: str) -> dict:
    """Читає ваги за хешем. Перевіряє, що хеш прочитаного збігається з іменем файлу."""
```

властивість	правило
подільність	K ділиться на k без остачі, інакше `ValueError`
перша точка	C_0 — копія `state` до першого кроку
крок s	батч `batch_indices(seed, s, batch_size, n_train)`, $s = 0 \ldots K-1$
точка C_j	стан після $j \cdot k$ кроків; остання C_m збігається з кінцевим станом
оптимізатор	`torch.optim.SGD(params, lr=lr)`, без моменту
метрика	точність на підвибірці тесту, детермінованій від seed

правило заявленої метрики

```
perm_test = randperm(n_test, generator=Generator().manual_seed(seed))
idx       = perm_test[:METRIC_SAMPLE]        # METRIC_SAMPLE = 2000
metric_micro = round(accuracy(model, X_test, y_test, idx) * 1e6)
```

ЧОМУ САМЕ ТАК

Чому підвибірка для метрики виводиться з seed, а не обирається майнером? Бо інакше майнер перебирає підвибірки по 2000 прикладів, доки не знайде ту, де його модель дає 96 % замість 91 %, і заявляє це число. Жодного навчання, чиста вигода. Той самий принцип, що й у P2: усе, на що майнер може вплинути безкоштовно, він рано чи пізно почне підбирати. Наслідок приємний: метрика стає точно відтворюваною. Механізм A при цьому не зайвий — він міряє точність на **своїй** випадковій підвибірці й порівнює із заявленою.

УВАГА

`load_state` **зобов'язана** перерахувати канонічний хеш прочитаного і порівняти з іменем файлу. Це два рядки, які рятують від найгіршого класу помилок: мовчки зіпсований файл у сховищі. Без них помилка вилізе за три кроки звідси — як «чесний блок відхилено» у Кроці 4.

Обмін із вузлом (повторення контракту з Кроку 1):

task.json (op = train) → result.json

```
вхід:  op task_id round miner_id prev_block_hash w_start
       K k batch_size lr dataset store_dir out
вихід: ok w_end ckpt_hashes[] metric_micro steps seconds
```

поле результату	що саме туди кладеться
`ok`	`true`, якщо воркер відпрацював без винятку. Це **не** вердикт про блок
`w_end`	hex канонічного хешу кінцевих ваг = останній елемент `ckpt_hashes`
`ckpt_hashes`	список із $K/k + 1$ hex-рядків, перший = `w_start`
`metric_micro`	ціле, точність $\times 10^6$; `steps` — це K, дублюється навмисно
`seconds`	час самого навчання за `perf_counter` — без старту Python і без читання даних

ЧОМУ САМЕ ТАК

Чому `seconds` міряє лише навчання? Бо це число піде в знаменник метрики «накладні витрати верифікації». Якщо туди потрапить 1–3 секунди старту інтерпретатора, відношення стане властивістю способу запуску, а не механізму. Старт теж міряй, але окремо.

Скелет

worker/storage.py · тіла написати самому

```python
def save_state(state: dict, store_dir: str) -> str:
    # 1. Порахуй канонічний хеш ваг (Крок 1), переведи в hex.
    # 2. Склади шлях store_dir/<hex>.safetensors, створи теку, якщо немає.
    # 3. Якщо файл уже існує — одразу поверни hex (ідемпотентність).
    # 4. Інакше збережи через safetensors.torch.save_file. Тензори мають бути
    #    .detach().cpu().contiguous() — інакше формат відмовить.


def load_state(hash_hex: str, store_dir: str) -> dict:
    # 1. Прочитай через safetensors.torch.load_file, перерахуй канонічний хеш.
    # 2. Не збігається з hash_hex -> ValueError з обома значеннями в тексті.
```

worker/training.py · тіла написати самому

```python
def train_block(state, seed, K, k, lr, batch_size, data):
    # 1. Перевір K % k == 0, інакше ValueError.
    # 2. Побудуй build_model(seed) і завантаж у неї state (load_state_dict).
    # 3. Заведи SGD(lr=lr) без моментуму.
    # 4. ckpts = [знімок поточного стану]  <- це C_0, і це має бути КОПІЯ.
    # 5. Для step від 0 до K-1:
    #    - idx = batch_indices(seed, step, batch_size, кількість тренувальних прикладів)
    #    - xb = to_float(X_train[idx]), yb = y_train[idx]
    #    - п'ять рядків кроку (як у smoke.py)
    #    - якщо (step + 1) % k == 0 — додай знімок стану в ckpts
```

```
    # 6. Порахуй метрику за правилом заявленої метрики (підвибірка від seed).
    # 7. Перевір: len(ckpts) == K // k + 1. Не сходиться — виправляй п.5, а не тест.
    # 8. Поверни (останній знімок, ckpts, метрика).
```

ПІДКАЗКИ

Рівень 1. Найперше налагодь не навчання, а **кількість** точок: постав `K=4, k=2` і подивись, скільки елементів у `ckpts`. Має бути 3.

Рівень 2. Знімок — новий словник із копіями тензорів, інакше всі точки вкажуть на ту саму пам'ять і після навчання стануть однаковими. Перетворення `uint8` роби **побатчево** (`to_float(X_train[idx])`): увесь MNIST у float32 — це 188 МБ без жодної користі.

Рівень 3. Знімок:

```
ckpts.append({n: t.detach().clone()
              for n, t in model.state_dict().items()})
```

`.clone()` обов'язковий: `state_dict()` віддає **посилання** на живі тензори моделі, тож без нього ти збережеш 21 посилання на один і той самий набір ваг, що весь час оновлюється.

worker/worker.py — доповнення · тіла написати самому

```
def op_train(task: dict) -> dict:
    # 1. seed = derive_seed(bytes.fromhex(prev_block_hash),
    #                       bytes.fromhex(miner_id), round).
    # 2. Завантаж дані load_mnist("data").
    # 3. Початковий стан: якщо w_start == "00"*32, це генезис — побудуй модель від
    #    seed0 = перші 8 байтів task_id як ціле і збережи в сховище;
    #    інакше storage.load_state(w_start, store_dir).
    # 4. t0 = time.perf_counter() перед train_block, t1 після.
    # 5. Збережи КОЖНУ контрольну точку в сховище, збери список hex-хешів.
    # 6. Перевір, що ckpt_hashes[0] == task["w_start"] (для генезису — щойно
    #     збережений).
    # 7. Склади словник результату за контрактом і поверни.


def main(argv):
    # 1. Шлях до task.json з argv[1]; немає аргументу -> помилка і код 2.
    # 2. Розбери JSON, візьми поле "op", диспетчеризуй: {"train": op_train}
    #    (у Кроці 4 додасться verify_a, verify_b). Невідомий op -> ok: false.
    # 3. Обгорни виклик у try/except: будь-який виняток -> {"ok": false, "error":
    #     str(e)}.
    #    Воркер НЕ МАЄ права падати з трасуванням: вузол чекає JSON.
    # 4. Запиши результат у файл task["out"] (створи теку), поверни 0.
```

УВАГА

Пункт 3 у `main` — не косметика. Якщо воркер упаде зі стеком у `stderr`, вузол не зможе відрізнити «блок поганий» від «воркер зламався». Перше означає «відхилити блок», друге — «моя машина несправна, мовчати». Переплутати їх означає відхиляти чесні блоки через власні баги, тобто зіпсувати FRR.

Твоє завдання

ТВОЄ ЗАВДАННЯ

1. Напиши `worker/storage.py` і `worker/training.py`, доповни `worker/worker.py`.

2. Набери тести як є, запусти.
3. Зроби руками `run/task_genesis.json` (`w_start` = 64 нулі, `K` = 500, `k` = 25, `batch_size` = 64, `lr` = 0.01) і запусти `python worker/worker.py run/task_genesis.json`. Подивись на `result.json` і на теку `store/`: порахуй файли й обсяг.
4. Зроби другий блок: `w_start` = `w_end` першого, `round` + 1. Метрика має зрости.
5. Запиши в `docs/plan.md` три числа для своєї машини: секунд, файлів і мегабайтів на блок.
6. Коміт: `feat(worker): training block with checkpoints and store`.

worker/tests/test_training.py

```
import torch
from worker.model import build_model, load_mnist
from worker.training import train_block
from worker.canonical import canonical_weight_hash
from worker.storage import save_state, load_state


def _tiny():
    """Малий набір, щоб тест був швидкий."""
    d = load_mnist("data")
    return {"X_train": d["X_train"][:256], "y_train": d["y_train"][:256],
            "X_test": d["X_test"][:256], "y_test": d["y_test"][:256]}


def test_checkpoints_count_anchor_and_distinct():
    data = _tiny()
    st = build_model(1).state_dict()
    h0 = canonical_weight_hash(st)
    end, ckpts, metric = train_block(st, seed=5, K=20, k=5, lr=0.01,
                                     batch_size=32, data=data)
    assert len(ckpts) == 20 // 5 + 1
    assert canonical_weight_hash(ckpts[0]) == h0
    assert canonical_weight_hash(ckpts[-1]) == canonical_weight_hash(end)
    hashes = [canonical_weight_hash(c) for c in ckpts]
    assert len(set(hashes)) == len(hashes)
    assert 0.0 <= metric <= 1.0


def test_recompute_is_bitwise_equal():
    data = _tiny()
    st = build_model(1).state_dict()
    e1, _, _ = train_block(st, 5, 20, 5, 0.01, 32, data)
    e2, _, _ = train_block(st, 5, 20, 5, 0.01, 32, data)
    assert canonical_weight_hash(e1) == canonical_weight_hash(e2)
    for name in e1:
        assert torch.equal(e1[name], e2[name])


def test_store_roundtrip(tmp_path):
    st = build_model(3).state_dict()
    h = save_state(st, str(tmp_path))
    again = load_state(h, str(tmp_path))
    assert canonical_weight_hash(again) == canonical_weight_hash(st)
    assert save_state(again, str(tmp_path)) == h
```

`test_recompute_is_bitwise_equal` — найважливіший тест у проєкті. Якщо він падає, механізм В не працює в принципі, і всі числа розділу 4 нічого не варті. Ганяй його після кожної зміни.

Перевірка

ПЕРЕДБАЧ ДО ЗАПУСКУ

1. Скільки файлів з'явиться в `store/` після блоку з $K = 500$, $k = 25$, і скільки це мегабайтів, якщо один файл — близько 344 КБ?
2. Другий блок стартує з `w_end` першого. Скільки **нових** файлів додасться — рівно стільки ж, більше чи менше? Чому?
3. Метрика першого блоку і другого: яка більша і на скільки приблизно?

ЕТАЛОН ДЛЯ САМОПЕРЕВІРКИ

1. `ok: true`, у `ckpt_hashes` рівно 21 елемент ($500/25 + 1$), перший збігається з `w_start`, останній — з `w_end`; усі 21 хеш різні. Два однакові поспіль означають, що загубився `.clone()` або що модель не вчиться взагалі.
2. Файлів у сховищі на один менше, ніж хешів у другому блоці: C_0 другого — це C_m першого, він уже там. У цьому й сенс content-addressed сховища.
3. Метрика другого блоку вища, але приріст менший: перші сотні кроків дають найбільший стрибок. Якщо метрика **впала** — перевір, чи справді другий блок стартував з `w_end` першого.
4. `seconds` для $K = 500$ на CPU — одиниці секунд, і між двома запусками того самого завдання відрізняються на відсотки, а не в рази. Сам `w_end` при цьому збігається символ у символ.

САМОПЕРЕВІРКА

1. Чому C_0 зберігається окремо, якщо дорівнює `w_start` із заголовка блоку?
2. Майнер збільшив k з 25 до 250, щоб зменшити кількість файлів. Що це дає йому і що відбирає у валідатора?
3. `ok: false` і `verdict: "reject"` — поясни різницю на прикладі, коли в сховищі бракує файлу.
4. Чому `save_state` нічого не переписує, якщо файл уже існує?
5. Що зміниться в контракті, якщо завтра ми вирішимо додати моментум?

Зламай навмисне

ЗЛАМАЙ НАВМИСНЕ

Експеримент 1. Прибери `.clone()` у знімку контрольної точки. Скільки різних хешів у списку? Поясни, чим `state_dict()` відрізняється від копії ваг і чому цей баг не проявився б у звичайному навчальному коді.

Експеримент 2. Зміни `lr` **на** 10^{-9} (наприклад, 0.01 на 0.010000001). Порівняй `w_end` двох запусків: скільки символів хеша збіглося? Тепер порівняй **метрику**. Це найкоротша демонстрація того, чому перевірка за метрикою не ловить підміну траєкторії.

Експеримент 3. Зміни seed, лишивши все інше. Метрика приблизно та сама, ваги зовсім інші. Запиши це: воно піде в роботу як обґрунтування того, чому «правильна відповідь» у навчанні не єдина, а отже реплікація (механізм C) вимагає порівняння з допуском, а не на рівність.

Експеримент 4. Постав $K = 500$, $k = 30$. Має бути `ValueError`. Прибери перевірку подільності й подивись, що станеться з довжиною списку точок і чи збігається остання точка з кінцевим станом. Поясни, чому мовчазне «майже працює» тут гірше за виняток.

У ТЕКСТ РОБОТИ

Крок дає підрозділ «Життєвий цикл блоку» розділу 3. У ньому мають бути: схема $C_0 \to C_1 \to \ldots \to C_m$ зі стрілками по k кроків SGD; таблиця «звичайне навчання проти навчального блоку» з початку цього кроку; правила детермінованості (seed із P2, гіперпараметри з P3, SGD без моментуму); твої власні три числа — секунд, файлів і мегабайтів на блок, з назвою процесора і версією PyTorch поруч; оцінка обсягу сховища для експерименту. Пиши зараз, доки рішення свіжі: у квітні ти пам’ятатимеш «що», але не «чому».

Крок 4. Верифікація A і B

Мета кроку: `worker/verify.py` з механізмами A і B, дві нові операції воркера і — головне — **відкалібрований** поріг δ, отриманий вимірюванням, а не вибраний на око.

Ідея

Це центральний крок роботи: досі ми будували те, що **робить** роботу, тепер — те, що вирішує, чи роботу справді зроблено.

Механізм A дивиться лише на відповідь: валідатор бере випадкову підвибірку тесту, проганяє через неї надіслану модель і звіряє з заявленою метрикою. Дешево і майже сліпо: чужі валідні ваги пройдуть A ідеально, бо вони справді дають заявлену точність.

Механізм B дивиться на шлях: валідатор бере пару сусідніх контрольних точок, повторює k кроків від першої і порівнює з другою. Ловить те, чого A не бачить, але має власну проблему: результат перерахунку **ніколи** не збігається побітово, якщо валідатор рахує не на тій самій машині. Тому потрібен поріг — і весь Крок 4 насправді про те, як його вибрати.

Чому не можна перевірити всю траєкторію

Перша ідея кожного: нехай валідатор повторить усі K кроків від `w_start` і порівняє з `w_end`. Не працює, і причина не в ціні, а в арифметиці. Числа з рухомою комою не асоціативні: $(a + b) + c$ і $a + (b + c)$ дають різні останні біти. Порядок додавання всередині згортки залежить від реалізації, реалізація — від пристрою, версії бібліотеки і кількості потоків. Різниця в останньому біті на кроці 1 стає різницею в передостанньому на кроці 10: похибка накопичується разом із самим навчанням.

Порядки величин, виміряні на **іншій** машині — Apple M5, CPU проти GPU, синтетичні дані, невелика CNN для MNIST, **один** запуск на комірку. Це чужі числа: вони показують напрямок, а не значення для твоєї роботи.

що порівнюється	відносна відстань L_2
уся траєкторія, 100 кроків, CPU проти GPU	$3.6 \cdot 10^{-4}$
уся траєкторія, 500 кроків, CPU проти GPU	$3.3 \cdot 10^{-3}$
уся траєкторія, 1000 кроків, CPU проти GPU	$5.3 \cdot 10^{-3}$
для масштабу: чесне оновлення за 100 кроків	$5.7 \cdot 10^{-3}$

Останній рядок — вирок. Після тисячі кроків похибка чесного повтору на іншому пристрої ($5.3 \cdot 10^{-3}$) того самого порядку, що й **увесь корисний прогрес** за сто кроків ($5.7 \cdot 10^{-3}$). Поріг, який пропустить чесного, пропустить і того, хто не робив нічого.

Тепер перерахунок **одного переходу** від контрольної точки майнера: майнер на CPU, валідатор на GPU, старт із кроку 500. «Ліниве повторення» означає, що майнер не рахував нічого й видав попередній стан.

k	похибка чесного вузла	відстань лінивого повторення	запас
1	$1.5 \cdot 10^{-6}$	$6.8 \cdot 10^{-4}$	$\approx 460\times$
10	$1.6 \cdot 10^{-6}$	$2.0 \cdot 10^{-3}$	$\approx 1200\times$
50	$6.9 \cdot 10^{-5}$	$3.9 \cdot 10^{-3}$	$\approx 56\times$
100	$5.9 \cdot 10^{-4}$	$4.8 \cdot 10^{-3}$	$\approx 8\times$
200	$8.1 \cdot 10^{-4}$	$7.2 \cdot 10^{-3}$	$\approx 9\times$

Різниця з першою таблицею не в обчисленнях, а в **точці старту**: коли валідатор стартує з контрольної точки майнера, похибці нема де накопичитись — вона встигає вирости лише за k кроків. Звідси два висновки, які визначають увесь дизайн: перевіряти треба **короткі переходи від точок майнера**, ніколи не всю траєкторію; k має бути малим, 10–50, бо при $k = 100$ запас падає до восьми разів, а це вже не інженерний запас, а тонка межа.

> **УВАГА**
> Наведені числа отримані на чужому залізі, на синтетичних даних і в один запуск. У роботу вони йдуть лише як посилання на попереднє дослідження здійсненності. У таблиці розділу 4 мають стояти **твої** виміри, на справжньому MNIST, із зазначенням процесора, пристрою, версії PyTorch і кількості повторів.

Нові слова

слово	що означає
`detection rate`	Частка спійманих спроб шахрайства. Окремо для кожної атаки.
`FRR`	False rejection rate — частка чесних блоків, відхилених помилково. Ніколи не нуль.
`ROC-крива`	Графік «виявлення проти хибних відхилень» при різних порогах. Замінює одне число.
`ε і δ`	Допуск механізму A (метрика) і поріг механізму B (відстань між вагами).
`відносний поріг`	$\delta = \alpha \cdot \|\Delta W \text{ перерахований}\|$ — масштабується разом із оновленням; абсолютний поріг — просто фіксоване число.
`grinding`	Атака перебором: майнер крутить поле блоку, доки перевірка не стане вигідною.

Контракт

worker/verify.py — сигнатури

```python
def rel_distance(a: dict, b: dict) -> float:
    """‖a − b‖₂ / ‖b‖₂ по всіх тензорах, склеєних в один вектор."""

def threshold(rule: dict, recomputed_update_norm: float) -> float:
    """{"kind":"absolute","delta":x} -> x
       {"kind":"relative","alpha":a} -> a * recomputed_update_norm"""

def verify_a(state, data, sample_size, claimed_metric, eps, rng) -> tuple[str, float]:
    """Метрика на ВИПАДКОВІЙ підвибірці валідатора. ("accept"|"reject", метрика)"""
```

```python
def verify_b(state_from, state_to, step_from, k, seed, lr, batch_size,
             data, threshold_rule) -> tuple[str, float, float]:
    """Перерахунок k кроків від state_from. ("accept"|"reject", відстань, поріг)"""
```

правило перевірки B

```
1. W_r  = k кроків SGD від state_from, кроки step_from .. step_from + k − 1,
          батчі batch_indices(seed, s, batch_size, n) — seed МАЙНЕРА, не валідатора
2. d    = rel_distance(W_r, state_to)      наскільки перерахунок розійшовся з заявою
3. u    = rel_distance(W_r, state_from)    наскільки великий сам перерахований крок
4. δ    = threshold(rule, u)
5. вердикт = "accept", якщо d ≤ δ, інакше "reject"
```

ЧОМУ САМЕ ТАК

Чому і відстань, і норма оновлення відносні. Величини по різні боки порівняння мають бути в одних одиницях: `rel_distance` ділить на норму ваг, тобто дає безрозмірне число, і якби поріг рахувався від **абсолютної** норми оновлення, ти порівнював би відсотки з кілограмами.

Чому валідатор бере seed майнера. Бо він відтворює чужу роботу. На віру він його не приймає: seed виводиться формулою P2 з `prev_block_hash`, `miner_id` і номера раунду, тож валідатор рахує його сам із полів заголовка. Майнер не може ні обрати його, ні збрехати про нього.

Правил порога два. **Абсолютне** ($\delta = x$) фіксує похибку в абсолютних одиницях і служить базовою лінією для порівняння. **Відносне** ($\delta = \alpha \cdot u$) задає її як частку зробленого кроку і є основним варіантом (P6). Очікуваний результат експерименту записаний у P6 як гіпотеза: наприкінці навчання, коли справжні оновлення стають малими, фіксований δ починає накривати і «нічого не робив» — тобто ліниве повторення проходить саме тоді, коли модель уже цінна.

ЧОМУ САМЕ ТАК

Звідки береться діапазон розумних α. Лінивий майнер видав `state_to` $\approx$ `state_from`. Валідатор рахує чесно й отримує W_r, що відрізняється від `state_from` рівно на оновлення, тобто на u; отже виміряна відстань $d \approx u$. Щоб такого майнера відхилити, потрібно $\alpha < 1$: інакше $\delta = \alpha \cdot u \geq d$ і ліниве повторення проходить. З іншого боку, чесний валідатор на іншому залізі має похибку e, тож потрібно $\alpha > e/u$: за чужими числами з таблиці при $k = 10$ це $1.6 \cdot 10^{-6} / 2.0 \cdot 10^{-3} \approx 0.0008$, а при $k = 100$ — вже ≈ 0.12. Отже вікно для α при $k = 10$ — приблизно від 0.001 до 0.5, а при $k = 100$ — від 0.12 до 0.5: перше широке, друге вузьке, і будь-яка неточність калібрування виштовхує або у високий FRR, або у пропущену атаку.

У прикладі `task.json` з Кроку 1 стоїть `"alpha": 1.0` — це заглушка формату, а не значення: з $\alpha = 1.0$ механізм B не ловить навіть найпримітивнішу атаку.

Механізм A і чому ε не може бути маленьким

Валідатор бере випадкову підвибірку розміру s **власним** генератором, міряє точність і приймає блок, якщо $|\text{виміряно} - \text{заявлено}| \leq \varepsilon$. Навіть при чесному майнері виміряне число не дорівнює заявленому: це різні підвибірки. Точність на вибірці з s прикладів — випадкова величина зі стандартним відхиленням приблизно $\sqrt{p(1-p)/s}$.

s	σ при $p \approx 0.9$	розумний мінімум ε
500	≈ 0.013	≈ 0.04
2000	≈ 0.0067	≈ 0.02
10000	≈ 0.003	≈ 0.01

ЧОМУ САМЕ ТАК

Це пояснює, чому A не може бути головним механізмом. Щоб не відхиляти чесних, ε мусить бути близько 0.02, а різниця між чесно натренованою моделлю і чужою вкраденою — нуль: вкрадена дає рівно ту саму точність. А ловить лише грубу брехню про метрику, і зменшення ε цього не змінить — воно лише підніме FRR.

УВАГА

Правило P5: звідки валідатор бере випадковість. Здається зручним виводити номери перевірюваних переходів із хешу блоку: $j = H(\text{block_hash}) \bmod m$ — тоді всі валідатори перевіряють те саме. Але блок складає майнер: він крутить `timestamp` (чи будь-яке інше поле, що не впливає на роботу), перераховує `block_hash` і дивиться, який перехід випаде. Нехай чесно зроблено лише 10 % переходів, а валідатор перевіряє $q = 2$: імовірність, що обидва номери впадуть на чесні, близько 0.01, отже 100 спроб — і блок пройде перевірку. Сто хешувань займають частки мілісекунди, атака безкоштовна.

Тому кожен валідатор обирає переходи **власним приватним генератором**, і майнер не може знати вибір заздалегідь. Ціна — різні валідатори можуть дійти різних висновків; у MVP кожен вузол вирішує сам, fraud proof переїжджає у фазу 2.

При q незалежно обраних переходах і частці чесних переходів f імовірність пропустити атаку дорівнює f^q: при $f = 0.1$ і $q = 2$ це 1 %, при $q = 3$ — 0.1 %. Формула — готовий абзац у розділ 4; в експерименті її треба підтвердити емпірично.

Скелет

worker/verify.py · тіла написати самому

```
def rel_distance(a: dict, b: dict) -> float:
    # 1. Перевір, що набори ключів однакові (інакше порівнюєш різні моделі).
    # 2. Склей тензори кожного словника в один довгий вектор, обходячи ключі
    #    в sorted(...) — порядок має бути однаковий з обох боків.
    # 3. Рахуй у float64: різниці тут бувають близько 1e-6, і float32 почне
    #    міряти власний шум замість відстані.
    # 4. Поверни ‖a−b‖ / ‖b‖ як float. Якщо ‖b‖ == 0 — ValueError.


def threshold(rule: dict, recomputed_update_norm: float) -> float:
    # 1. "absolute" -> rule["delta"];  "relative" -> alpha * recomputed_update_norm.
    # 2. Інше -> ValueError із назвою невідомого правила.


def verify_a(state, data, sample_size, claimed_metric, eps, rng):
    # 1. Обери sample_size індексів тесту генератором rng — це ВЛАСНА випадковість
    #    валідатора, не seed майнера і не хеш блоку.
    # 2. Побудуй модель, завантаж у неї state, поміряй accuracy на цих індексах.
```

```
    # 3. Поверни ("accept" якщо abs(виміряно − claimed_metric) <= eps, виміряна
    #     метрика).


def verify_b(state_from, state_to, step_from, k, seed, lr, batch_size,
             data, threshold_rule):
    # 1. Перевір, що step_from кратний k.
    # 2. Побудуй модель, завантаж у неї state_from.
    # 3. Зроби k кроків SGD для s = step_from .. step_from + k − 1, батчі −
    #    batch_indices(seed, s, batch_size, n_train). Той самий крок, що в train_block:
    #    винеси його в спільну функцію, щоб перевірка не розійшлася з навчанням.
    # 4. d = rel_distance(перерахунок, state_to), u = rel_distance(перерахунок,
    #     state_from).
    # 5. delta = threshold(threshold_rule, u).
    # 6. Поверни ("accept" якщо d <= delta інакше "reject", d, delta).
```

ПІДКАЗКИ

Рівень 1. Почни з `rel_distance` на штучних даних: відстань між станом і ним самим має бути рівно 0, а між `W` і `2W` — рівно 0.5 (порахуй на папері, чому саме 0.5).

Рівень 2. Склеювання: `torch.cat([...])` по списку `t.flatten()`; норма — `torch.linalg.vector_norm(v)`. Для `rng` у `verify_a` зручний `torch.Generator()` із seed від `secrets.randbits(63)` — саме **випадковим**, це і є приватна випадковість валідатора.

Рівень 3. Склеювання з переходом у float64 — один рядок, і саме в ньому найчастіше плутають порядок:

```
va = torch.cat([a[n].flatten().double() for n in sorted(a)])
```

Другий вектор будується так само, обов'язково через `sorted(b)`. Далі — дві норми й ділення; це вже пиши сам.

УВАГА

Пункт 3 у `verify_b`: крок навчання має бути **буквально тим самим кодом**, що й у `train_block` — спільна функція, яку викликають обидва. Якщо це дві копії, вони рано чи пізно розійдуться на один рядок, і ти отримаєш хибні відхилення, причину яких шукатимеш у залізі, а не в собі.

worker/worker.py — нові операції · тіла написати самому

```
def op_verify_a(task: dict) -> dict:
    # 1. Завантаж state зі сховища за task["w_end"].
    # 2. Заведи ВЛАСНИЙ генератор із випадковим seed (не з завдання!).
    # 3. verify_a із sample_size, claimed_metric = metric_micro / 1e6, eps із завдання.
    # 4. Поверни {ok, verdict, measured_metric, seconds}.


def op_verify_b(task: dict) -> dict:
    # 1. Виведи seed майнера так само, як у op_train.
    # 2. Завантаж дві точки: ckpt_from і ckpt_to.
    # 3. verify_b із step_from, k, lr, batch_size, threshold_rule із завдання.
    # 4. Поверни {ok, verdict, distance, threshold, seconds} за контрактом Кроку 1.
```

Твоє завдання

ТВОЄ ЗАВДАННЯ

1. Напиши `worker/verify.py` і додай дві операції у диспетчер воркера.
2. Набери тести як є, запусти. Додай від себе ще один: той самий блок, але `seed` іншого майнера — перевірка має відхилити. Саме так P2 ловить плагіат.
3. Напиши `experiments/calibrate_delta.py` (скелет нижче) і проведи калібрування.
4. За результатами обери α для кожного k і запиши в `docs/contract.md` з обґрунтуванням у два речення.
5. Побудуй **свою** версію таблиці запасу: k, похибка чесного, відстань лінивого повторення, у скільки разів запас. Це головна таблиця розділу 4.
6. Коміт: `feat(worker): verification A and B`, окремо `exp: delta calibration`.

experiments/calibrate_delta.py · тіла написати самому

```
"""Міряє, наскільки розходяться ЧЕСНІ перерахунки на різному «залізі»
і наскільки далеко від них ліниве повторення. З цього вибирається alpha."""

# ПАРИ ПРИСТРОЇВ (бери ті, що є на ПК; мінімум дві пари):
#   - CPU float32 (майнер) проти CPU float64 (валідатор)
#   - torch.set_num_threads(1) проти torch.set_num_threads(N)
#   - CPU проти CUDA, якщо є NVIDIA-карта
#   - дві різні версії PyTorch у двох venv (найчесніша пара, але найдорожча)
#
# 1. Натренуй еталонний прогін на 1000 кроків, зберігаючи стан кожні 50 кроків. Це
#     «майнер».
# 2. Візьми ~20 стартових точок із прогону, і з початку, і з кінця: похибка залежить
#    від того, наскільки модель уже навчена.
# 3. Для кожного k у {1, 10, 50, 100} і кожної стартової точки:
#    - ЧЕСНО: перерахуй k кроків у ролі валідатора (інший пристрій/точність/потоки),
#      d_honest = rel_distance(перерахунок, точка майнера через k кроків),
#      u = rel_distance(перерахунок, стартова точка);
#    - ЛІНИВО: підстав замість результату саму стартову точку (+ малий шум), порахуй
#     d_lazy;
#    - запиши рядок: k, номер точки, d_honest, d_lazy, u, d_honest/u, d_lazy/u.
# 4. Збережи у experiments/results/calib_<пара>.csv — сирі рядки, не агрегати.
# 5. Надрукуй таблицю: для кожного k — максимум d_honest/u, мінімум d_lazy/u
#    і відношення між ними (це і є запас).
# 6. Запропонуй alpha: геометричне середнє між цими двома. Відправна точка, не істина.
```

ПІДКАЗКИ

Рівень 1. Не пиши одразу всі пари: спершу CPU float32 проти CPU float64 — вона працює на будь-якій машині й не вимагає нічого, крім `model.double()` і `xb.double()`, а при порівнянні — зворотного `t.float()`. Для варіанта з потоками запускай **два окремі процеси**: `set_num_threads` усередині процесу може не подіяти на вже створені пули.

Рівень 2. Структура зовнішнього циклу — три вкладені `for`:

```
for k in (1, 10, 50, 100):
    for j, start in enumerate(starts):
        d_h, u = honest_pair(start, k)
        d_l, _ = lazy_pair(start, k)
        rows.append((k, j, d_h, d_l, u))
```

Тіла `honest_pair` і `lazy_pair` — твої; обидві всередині викликають `verify_b` з різними аргументами.

worker/tests/test_verify.py

```
import torch
from worker.model import build_model, load_mnist
from worker.training import train_block
from worker.verify import rel_distance, threshold, verify_b

RULE = {"kind": "relative", "alpha": 0.1}


def _block():
    d = load_mnist("data")
    data = {"X_train": d["X_train"][:256], "y_train": d["y_train"][:256],
            "X_test": d["X_test"][:256], "y_test": d["y_test"][:256]}
    st = build_model(1).state_dict()
    _, ckpts, _ = train_block(st, seed=5, K=20, k=10, lr=0.01,
                              batch_size=32, data=data)
    return data, ckpts


def test_distance_and_threshold_basics():
    a, b = {"w": torch.ones(4)}, {"w": torch.ones(4)}
    assert rel_distance(a, b) == 0.0
    assert abs(rel_distance(a, {"w": torch.ones(4) * 2}) - 0.5) < 1e-12
    assert threshold({"kind": "absolute", "delta": 0.01}, 999.0) == 0.01
    assert abs(threshold(RULE, 0.02) - 0.002) < 1e-15


def test_honest_transition_accepted():
    data, ckpts = _block()
    v, d, t = verify_b(ckpts[0], ckpts[1], 0, 10, 5, 0.01, 32, data, RULE)
    assert v == "accept"
    assert d == 0.0          # та сама машина, той самий код: побітово


def test_lazy_replay_rejected():
    data, ckpts = _block()
    fake = {n: t.clone() for n, t in ckpts[0].items()}   # «нічого не робив»
    v, d, t = verify_b(ckpts[0], fake, 0, 10, 5, 0.01, 32, data, RULE)
    assert v == "reject"
```

`test_honest_transition_accepted` вимагає `d == 0.0` рівно — навмисно. На одній машині тим самим кодом перерахунок зобов'язаний бути побітовим. Ненульове значення означає приховану недетермінованість, і шукати її треба зараз, а не після того, як зіпсується вся таблиця FRR.

Перевірка

ПЕРЕДБАЧ ДО ЗАПУСКУ

1. Похибка чесного перерахунку при $k = 1$ і при $k = 100$: яка більша і приблизно у скільки разів?
2. Запас (відношення лінивого до чесного) зі зростанням k зростає чи падає? Чому?

3. Чи зміниться похибка чесного вузла залежно від того, на якому етапі навчання взята стартова точка — на 50-му кроці чи на 950-му?

ЕТАЛОН ДЛЯ САМОПЕРЕВІРКИ

Конкретні числа будуть твої. Мають виконуватись властивості:

1. **Побітовість на місці.** На тій самій машині, без зміни точності й потоків, `verify_b` дає рівно 0.0. Не 10^{-12} — нуль.
2. **Похибка чесного зростає з k**: при $k = 100$ вона на порядки більша, ніж при $k = 1$. Не зростає — ти випадково порівнюєш щось із самим собою.
3. **Ліниве повторення на порядки далі.** При малих k його відстань більша за похибку чесного у сотні-тисячі разів. Це і є той запас, на якому тримається механізм B.
4. **Запас скорочується зі зростанням k** — саме тому k малий. Якщо в тебе при $k = 100$ запас менший за десять разів, це нормальний результат, і його треба написати в роботі як є.
5. **Залежність від етапу навчання.** Похибка чесного зазвичай більша там, де більші градієнти. Побачив це в даних — маєш ще один аргумент на користь відносного порога проти абсолютного.
6. **Механізм A.** Виміряна метрика гуляє навколо заявленої в межах кількох σ з таблиці. Гуляє більше — підвибірка не випадкова або модель завантажена не та.

САМОПЕРЕВІРКА

1. Чому перевірка всієї траєкторії не працює? Відповідь у двох реченнях, не вживаючи слова «дорого».
2. Що таке FRR і чому він не дорівнює нулю навіть тоді, коли всі вузли чесні?
3. Чому валідатор не може брати випадковість із хешу блоку? Опиши атаку і порахуй, скільки приблизно спроб вона коштує при $f = 0.1$ і $q = 2$.
4. Чому при $\alpha = 1.0$ механізм B пропускає ліниве повторення? Покажи через $d \approx u$.
5. У чому різниця між ε і δ — не за формулою, а за природою величини, яку кожен із них допускає? Валідатор перевірив 2 переходи з 20 і прийняв блок: яку гарантію він має, а якої не має?

Зламай навмисне

ЗЛАМАЙ НАВМИСНЕ

Експеримент 1. $\alpha = 1.0$. Постав у `threshold_rule` те саме значення, що в прикладі `task.json` з Кроку 1, і прожени `test_lazy_replay_rejected`. Тест має впасти. Поясни одним реченням, чому це не баг тесту.

Експеримент 2. Абсолютний поріг, відкалібрований рано. Візьми δ, виміряний на стартових точках з початку навчання, і застосуй до переходів із кінця довгого прогону (крок

900+). Скільки лінивих повторень тепер проходить? Це очікуваний результат, прямо записаний у P6 як гіпотеза — тепер у тебе є для неї числа.

Експеримент 3. Grinding. Напиши двадцять рядків: підбирай `timestamp` у заголовку, рахуй $j = H(\text{block_hash}) \bmod 20$ і зупинись, коли j впаде на один із двох «чесних» переходів. Порахуй, скільки спроб знадобилось і скільки це зайняло часу. Найпереконливіша ілюстрація правила P5 для захисту.

Експеримент 4. Перевірка всієї траєкторії. Замість переходу між сусідніми точками перерахуй усі 500 кроків від C_0 і порівняй із `w_end` — на іншій точності (float64) або на іншому пристрої. Порівняй отриману відстань із відстанню лінивого повторення. Обидва числа поруч — це твоя власна версія першої таблиці цього кроку, і саме вона обґрунтовує весь дизайн механізму B.

У ТЕКСТ РОБОТИ

Крок 4 дає найбільшу частину розділу 4. Готуй одразу: таблицю запасу (твої числа, з описом заліза і кількістю повторів); ROC-криву — для двадцяти значень α у логарифмічній сітці порахуй пару (detection rate лінивого повторення, FRR чесних) і побудуй графік для відносного й абсолютного порогів на одних осях; виміряні накладні витрати — `seconds` перевірки B при $k = 10, 50, 100$, поділені на `seconds` блоку, звірені з теоретичним $q \cdot k/K$; абзац про непридатність перевірки всієї траєкторії з обома таблицями; абзац про P5 з підрахунком вартості grinding-атаки; чесний перелік обмежень — один датасет, одна модель, локальні процеси, без підписів. Останній пункт не пропускай: робота, яка сама називає свої межі, на захисті виглядає сильніше за ту, яка робить вигляд, що їх немає.

КОНТРОЛЬНА ТОЧКА · 4 ЖОВТНЯ 2026

Кроки 2–4 закриті, якщо одночасно виконано:

1. `python worker/worker.py task.json` навчає блок: K кроків, $K/k + 1$ контрольних точок у сховищі, коректний `result.json`;
2. повторний запуск того самого завдання дає той самий `w_end` — побітово;
3. працюють обидві операції перевірки, і всі тести з трьох файлів проходять;
4. є відкалібрований α для щонайменше двох значень k, записаний у `docs/contract.md` разом із тим, як саме його отримано;
5. є **твоя** таблиця запасу з назвою процесора, парою пристроїв і кількістю повторів.

Немає калібрування — не рухайся далі. Крок 5 (атаки) без відкаліброваного порога не має що міряти: будь-яка атака або проходить усе, або не проходить нічого.

Крок 5. Перші атаки: лінивий і халтурник

Мета кроку: модуль `worker/attacks.py` з двома атаками, поле `"attack"` у `task.json` і скрипт `experiments/detect_matrix.py`, який дає перші рядки таблиці «атака × механізм».

Ідея

До цього кроку всі вузли чесні, і механізми A та B не ловлять нічого — ловити нема чого. Механізм верифікації без атаки — це замок, який ніхто не пробував відкрити. Перша атака потрібна не «для повноти роботи», а щоб перевірка вперше сказала `reject` і щоб ти побачив, за яким числом вона це сказала.

Головне, що варто винести з кроку: **атакувальник — не окрема програма й не зіпсований вузол**. Це звичайний вузол, який реалізує той самий інтерфейс іншим тілом функції. `train_block` повертає трійку `(state, ckpts, metric)`; `attack_lazy` і `attack_partial` повертають рівно таку саму трійку — ті самі типи, та сама довжина списку точок, той самий `ckpts[0]`. Ні воркер, ні валідатор не відрізнять одне від іншого за формою результату: `result.json` виходить однаковий за схемою. Відрізнити можна лише перерахунком, тобто роботою. Це і є та сама втрачена асиметрія: у хешуванні підробку видно з форми відповіді, у навчанні — ні.

> ЧОМУ САМЕ ТАК
> Атака тут не «злом», а вимірювальний прилад, і код у неї має бути таким самим охайним. Якщо вона написана недбало, ти не знатимеш, чому механізм її не спіймав: бо механізм слабкий чи бо атака зламана. Звідси правило: **кожна атака мусить мати контрольне значення параметра, при якому вона перетворюється на чесну роботу** (для `attack_partial` це `honest_fraction = 1.0`). Якщо при ньому механізм відхиляє блок, ти спіймав не атаку, а власний баг.

Лінивий вузол

Лінивий вузол не навчає взагалі: бере стан із вершини ланцюга, додає маленький випадковий шум і подає це як свій блок. Шум потрібен рівно для одного — щоб `w_end` не збігався з `w_start` побітово: блок, у якому кінець дорівнює початку, відкидається безкоштовно, структурною перевіркою. Заявлена метрика — метрика попереднього блока: вона публічна, лежить у полі `metric_micro` вершини, і заявити її найтихіше, бо число не суперечить нічому, що бачить мережа.

Спостереження, заради якого ця атака стоїть першою: **після виходу навчання на плато механізм A перестає її бачити**. А дивиться лише на відповідь — точність на підвибірці. Стара модель плюс малий шум відрізняється за точністю від нової менше, ніж поріг ε, з яким A мусить працювати, щоб не відхиляти чесних. Там, де A терпимий настільки, щоб не бити своїх, він

терпимий настільки, щоб пропустити чужого. Це не хиба реалізації, а межа механізму: аркуш із правильною відповіддю не доводить, що задачу розв'язували.

> Ранній раунд — інша справа: поки модель швидко вчиться, різниця точностей більша за ε, і А ловить ліниве повторення легко. Тому цифри для цієї атаки бери **не з першого раунду**, а з ділянки плато. Інакше висновок буде «А працює», хоча насправді він працює лише доти, доки модель швидко росте.

Халтурник

Халтурник виконує частину роботи чесно, а решту домальовує так, щоб траєкторія виглядала неперервною:

1. Блок містить $m = K/k$ переходів між сусідніми контрольними точками.
2. Вузол чесно проганяє перші $j = f \cdot m$ переходів справжнім SGD, з правильним seed і правильними батчами: виходять справжні точки $C_0 \dots C_j$.
3. Оцінює середнє зміщення за перехід $d = (C_j - C_0)/j$ і домальовує решту по прямій: $C_i = C_j + (i - j) \cdot d$ для $i > j$.
4. Оголошує C_m своїм `w_end` і чесно міряє точність цієї моделі — модель існує, тут обманювати нема потреби.

Навіщо пряма? Бо довжини кроків між сусідніми точками валідатор порівнює безкоштовно: якщо хвіст траєкторії стоїть на місці або стрибає на порядок далі за початок, це видно без перерахунку. Домальовка по прямій дає кроки правдоподібної довжини і боїться лише перерахунку.

Чому халтурник ловиться з імовірністю $1 - f^q$

Валідатор за правилом P5 обирає q переходів **власним приватним генератором**; майнер не знає, які саме. Валідатор пропустить підробку лише тоді, коли всі q виборів впадуть у чесні переходи. Імовірність одного влучання $\approx f$, вибори незалежні, отже:

$$P_{\text{пропуск}} \approx f^q, \quad P_{\text{виявлення}} \approx 1 - f^q$$

1. Це формула **випадковості вибору**, а не властивість B: вона справджується, лише якщо кожен фальшивий перехід дає відстань більшу за поріг (сила B), а кожен чесний — меншу (калібрування, P6). З завеликим порогом формула покаже 0.99, а виявлення буде нульове.
2. Якщо валідаторів V і одного `reject` досить, показник множиться: $1 - f^{q \cdot V}$ (P5).
3. Вибір без повернення дає трохи вище виявлення. Чому — питання самоперевірки.

Головний якісний висновок, який піде просто в текст роботи: **формула працює в обидва боки**. Чим менше роботи робить халтурник, тим дешевша його підробка — і тим легше його спіймати; щоб стати непомітним, він мусить виконати майже всю роботу чесно, а тоді економія зникає. Механізм B не робить підробку неможливою — він робить її невигідною, і для блокчейну це саме потрібне твердження.

Нові слова

слово	що означає
`sigma (відносний шум)`	Норма шуму, поділена на норму ваг. Абсолютний шум безглуздий: шари мають різний масштаб.
`інтерполяція`	Точка між двома відомими: $(1-t)\cdot a + t\cdot b$. Нею халтурник домальовує хвіст траєкторії.
`detection rate`	Частка спроб шахрайства, які механізм відхилив, окремо на кожну пару «атака, механізм».
`false rejection rate`	Частка чесних блоків, відхилених помилково. Без неї detection rate нічого не вартий: «відхиляти все» дає 100%.
`контрольне значення`	Значення параметра, при якому атака стає чесною роботою. Відрізняє спійману атаку від власного бага.
`довгий формат CSV`	Один рядок — одна подія, без усереднень. Агрегати рахуються потім, з файлу.

Контракт: модуль атак

`worker/attacks.py — сигнатури`

```
State  = Dict[str, torch.Tensor]
Result = Tuple[State, List[State], Optional[float]]

def attack_lazy(state: State, sigma: float, K: int, k: int) -> Result:
    """Не навчає: попередній стан плюс відносний шум sigma.
       Точки — інтерполяція між стартом і кінцем. Метрика: None (не вимірював)."""

def attack_partial(state: State, seed: int, K: int, k: int, lr: float,
                   batch_size: int, data, honest_fraction: float) -> Result:
    """Чесно рахує частку honest_fraction переходів, решту домальовує по прямій.
       Метрика: справжня точність побудованої моделі."""
```

1. `ckpts[0]` — **точна копія** вхідного `state`, без шуму: стартовий стан задає ланцюг (P1), і валідатор звіряє його хеш із `w_end` попереднього блока.
2. Довжина списку точок така сама, як у `train_block`: $m+1$, де $m = K/k$. Інша довжина — безкоштовне відхилення.
3. `attack_lazy` повертає `None` замість метрики: вузол справді нічого не вимірював. Яке число він заявить — вирішує політика заяви у воркері, а не атака.
4. `sigma` відносна: $\|\text{шум}\| / \|\text{ваги}\| \approx \sigma$ по всьому набору тензорів разом, а не по кожному окремо.
5. Аргумент `data` ніде не розбирається — лише передається далі: його форма зафіксована в Кроці 2, і цей модуль про неї нічого не знає.

> ЧОМУ САМЕ ТАК
>
> Чому `attack_lazy` не міряє метрику сама? Бо в її сигнатурі немає даних, і це навмисно. Заява про метрику — рішення **вузла**, а не властивість траєкторії: з одного лінивого стану можна подати попередню метрику, завищену або чесно виміряну. Це три різні атаки на A з однієї траєкторії, і перебирати їх треба конфігом, а не правками коду. При цьому `None` бачить лише

твій воркер: у `result.json`, тобто там, де починається протокол, метрика завжди число, і схема результату однакова для чесного й нечесного вузла.

Контракт: поле `attack` у `task.json`

task.json — атака замість чесної роботи

```
{
  "op": "train", "round": 17, "miner_id": "4ab1...", "w_start": "7d31...",
  "K": 200, "k": 20, "batch_size": 64, "lr": 0.01, "store_dir": "store",
  "prev_metric_micro": 912300,
  "attack": {"kind": "lazy", "sigma": 0.001, "claim": "prev"}
}
                        ... або ...
  "attack": {"kind": "partial", "honest_fraction": 0.1}
```

Відсутнє поле `attack` означає чесну роботу. `sigma` — орієнтир 10^{-3}; `honest_fraction` $= 1$ — контрольне значення. Політика заяви `claim` має три значення: `"prev"` (метрика попереднього блока: чи бачить A відсутність прогресу), `"inflate"` (попередня плюс `inflate_by`: чи ловить A завищення) і `"measure"` (чесно виміряна метрика зашумленої моделі: число справжнє, робота — ні). `prev_metric_micro` новим знанням не є — вузол і так бачить вершину ланцюга.

УВАГА

`result.json` не змінюється **жодним полем**. Ніякого `"attack": ...` у відповідь, навіть «для налагодження». Щойно слід атаки з'явиться в повідомленні протоколу, валідатор зможе його прочитати — і експеримент міритиме не механізм верифікації, а чит-код. Правда про те, хто атакував, живе в конфігу експерименту, а не в повідомленні.

Контракт: скрипт вимірювання

колонки experiments/results/detect.csv

```
attack,mechanism,k,q,threshold_rule,trial,verdict,distance,threshold,
seconds_verify,seconds_train
```

`attack` приймає й значення `honest` — це знаменник FRR, без нього таблиця неповна. Один рядок — один механізм, ніяких «A+B»; для A колонка `q` порожня, а не нуль. `threshold_rule` пишеться рядком на кшталт `relative:1.0`, інакше через місяць не згадаєш, з яким порогом міряв. Для B з $q > 1$ правило зведення таке: `verdict` — `reject`, якщо хоч один із q переходів не пройшов; `distance` — найбільша з q відстаней; `threshold` — поріг **того самого** переходу. Без цього правила колонка `distance` не має означення.

ЧОМУ САМЕ ТАК

Чому в CSV ідуть сирі рядки, а не готові середні? Із рядків завжди можна порахувати середнє, із середнього рядки — ніколи. У розділі 4 потрібні ще й розкид за кількома seed та ROC-криві: усе це рахується з `distance` і `threshold` постфактум, без перезапуску. А якщо прогін упаде

на 37-му trial, 36 рядків уже лежатимуть на диску — тому файл відкривається на дозапис, а після кожного рядка йде `flush()`.

УВАГА

Накладні витрати рахуй як «час верифікації поділити на час **чесного** навчання того самого блока». `seconds_train` лінивого вузла — майже нуль; поділивши на нього, отримаєш нескінченність у таблиці диплома.

Скелет

worker/attacks.py · тіла написати самому

```
def _add_relative_noise(state: State, sigma: float) -> State:
    # 1. Склей усі тензори в один вектор і візьми його норму -> nw.
    # 2. Згенеруй шум тієї ж форми для кожного тензора (torch.randn_like),
    #    склей так само, візьми норму -> nn.
    # 3. Поверни новий словник: state[name] + noise[name] * (sigma * nw / nn).
    # 4. Вхідний state не чіпай — клонуй. Мовчазна зміна чужого словника
    #    у симуляторі з Кроку 6 дасть баг, який шукається годинами.


def _lerp(a: State, b: State, t: float) -> State:
    # 1. Перевір, що набори імен збігаються, інакше ValueError.
    # 2. Для кожного імені поверни (1 - t) * a[name] + t * b[name].


def attack_lazy(state: State, sigma: float, K: int, k: int) -> Result:
    # 1. m = K // k; перевір, що K % k == 0 (контракт Кроку 3).
    # 2. end = _add_relative_noise(state, sigma).
    # 3. Точки: ckpts[0] = копія state, ckpts[m] = end,
    #    проміжні ckpts[i] = _lerp(state, end, i / m).
    # 4. Поверни (end, ckpts, None): метрику цей вузол не вимірював.


def attack_partial(state, seed, K, k, lr, batch_size, data,
                   honest_fraction) -> Result:
    # 1. m = K // k; j = max(1, round(honest_fraction * m)).
    # 2. Чесна частина: train_block(state, seed, j * k, k, lr, batch_size, data).
    #    Той самий seed — інакше батчі не збігатимуться з перерахунком валідатора.
    # 3. Середнє зміщення за перехід: d[name] = (C_j[name] - C_0[name]) / j.
    # 4. Домалюй хвіст: C_i = C_j + (i - j) * d для i = j+1..m; w_end = C_m.
    # 5. Виміряй метрику чесно: build_model + load_state_dict(w_end) + accuracy.
    # 6. Поверни (w_end, ckpts, metric) — трійка тієї ж форми, що й у train_block.
```

worker/worker.py — доповнення до диспетчера · тіла написати самому

```
def run_train(task: dict) -> dict:
    # 1. atk = task.get("attack")   # немає поля -> train_block, чесний шлях
    # 2. "lazy"    -> attack_lazy(state, atk["sigma"], K, k)
    #    "partial" -> attack_partial(..., atk["honest_fraction"])
    #    невідомий kind -> ValueError: мовчазне «вважати чесним» зіпсує таблицю.
    # 3. Якщо metric is None, застосуй atk.get("claim", "prev"):
    #    "prev" -> task["prev_metric_micro"]; "inflate" -> plus atk["inflate_by"];
    #    "measure" -> виміряй accuracy на тестовій частині.
    # 4. save_state для всіх точок; result.json рівно за контрактом Кроку 1.
    #    Схема результату однакова для всіх гілок.
```

experiments/detect_matrix.py · тіла написати самому

```
COLUMNS = ["attack", "mechanism", "k", "q", "threshold_rule", "trial",
           "verdict", "distance", "threshold", "seconds_verify", "seconds_train"]


def one_trial(attack: str, mechanism: str, cfg: dict, trial: int) -> dict:
    # 1. Стартовий стан: build_model(seed) і кілька чесних блоків — вийди на плато.
    #    Інакше висновок про A буде про ранній раунд, а не про систему.
    # 2. seed раунду — derive_seed з фіктивними prev_hash і miner_id, РІЗНИМИ
    #    для кожного trial: однакові seed зроблять з N прогонів одну подію.
    # 3. Побудуй блок (train_block / attack_lazy / attack_partial), заміряй час.
    # 4. Перевір: "A" -> verify_a за контрактом Кроку 4; "B" -> обери q переходів
    #    приватним генератором ВАЛІДАТОРА (random.Random зі своїм seed, не seed
    #    майнера!) і виклич verify_b на кожному.
    # 5. Зведи q результатів за правилом контракту; поверни dict із ключами COLUMNS.


def main():
    # 1. argparse: --trials, --k, --q, --alpha, --out.
    # 2. CSV на дозапис; csv.DictWriter(f, fieldnames=COLUMNS); заголовок — лише
    #    для нового файлу. Три цикли: атака x механізм x trial.
    # 3. Після кожного прогону writer.writerow(row) і f.flush().
    # 4. Нічого не усереднюй і не друкуй, крім шляху до файлу.
```

ПІДКАЗКИ

Рівень 1. Почни з `attack_lazy` — їй не потрібні ні дані, ні навчання. Перевір її окремо, ще до верифікації: `rel_distance(state, end)` має дати приблизно твій `sigma`.

Рівень 2. Склеювання у вектор: `torch.cat([t.reshape(-1) for t in state.values()])`, норма — `.norm()`. Копія стану: `{n: t.clone() for n, t in state.items()}`. У `attack_partial` номер кроку рахується від початку блока, тому чесна частина — це просто `train_block` із `K = j * k` і тим самим `seed`.

Рівень 3. Каркас масштабування шуму:

```
flat  = torch.cat([t.reshape(-1) for t in state.values()])
noise = {n: torch.randn_like(t) for n, t in state.items()}
nn    = torch.cat([t.reshape(-1) for t in noise.values()]).norm()
scale = sigma * flat.norm() / nn
end   = {n: state[n] + noise[n] * scale for n in state}
```

Твоє завдання

ТВОЄ ЗАВДАННЯ

1. Напиши `worker/attacks.py` і додай розбір поля `"attack"` у `worker/worker.py`. Чесний шлях не має змінитись жодним рядком — тести Кроку 3 мусять проходити.
2. **Контрольний тест перед усім іншим:** `attack_partial(..., honest_fraction=1.0)` дає ті самі точки, що й `train_block` із тими самими аргументами, побітово. Не проходить — далі не йди.
3. Порахуй руками $1 - f^q$ для $f = 0.1$ і $q = 1, 2, 3$, потім для $f = 0.9$ з тими самими q. Шість чисел запиши в `docs/`: це колонка «очікувано» в таблиці розділу 4.

4. Напиши `experiments/detect_matrix.py` і прожени щонайменше 20 прогонів на кожну пару (атака × механізм) при $k = 20$, $q = 2$. Не забудь рядки з `attack = honest`.
5. Порахуй із CSV три числа на пару: частку `reject`, медіану `distance / threshold` і середній `seconds_verify` — у pandas, але не всередині скрипта вимірювання.
6. Коміт: `feat(worker): lazy and partial-work attacks + detection matrix`.

Перевірка

ПЕРЕДБАЧ ДО ЗАПУСКУ

1. Лінива атака з малим `sigma`, механізм B, $k = 20$: що більше — `distance` чи `threshold` і на скільки порядків? Назви оцінку, а не «більше».
2. Механізм A на лінивій атаці в раунді 2 і в раунді 40: вердикти мають бути різні. Які і чому?
3. `honest_fraction = 0.1`, $q = 2$, 20 прогонів: скільки з них B має відхилити?

ЕТАЛОН ДЛЯ САМОПЕРЕВІРКИ

Числа залежать від заліза, тому еталон — це властивості:

1. **В проти лінивого повторення при малих k**: `distance` більша за `threshold` у **кожному** прогоні, з великим запасом. Запас залежить від k: у пробних вимірюваннях проєкту (`docs/research/technical.md`) при $k = 10…50$ він був у десятки й тисячі разів, а при $k = 100…200$ стискався до одиниць. Властивість, яку маєш побачити: відношення `distance / threshold` спадає зі зростанням k.
2. **А проти лінивого повторення на плато**: приймає. Якщо відхиляє — перевір ε: швидше за все, цей поріг відхилятиме й чесні вузли.
3. **В проти часткової роботи**: блок відхиляється лише тоді, коли серед перевірених переходів трапився фальшивий, тож частка `reject` має бути близька до $1 - f^q$ і не дорівнювати ні нулю, ні одиниці. Вийшло рівно 1.0 у всіх прогонах — перевір, чи вибір справді випадковий, а не «завжди останній».
4. **Контроль**: `honest_fraction = 1.0` дає той самий набір вердиктів, що й чесний вузол.

САМОПЕРЕВІРКА

1. Чому `ckpts[0]` у лінивій атаці — точна копія вхідного стану, а не «стан плюс трішки шуму»?
2. Лінивий вузол «навчався» нуль секунд. Які два похідні числа це зіпсує, якщо рахувати їх необережно?
3. Вибір q переходів без повернення дає трохи вище виявлення, ніж $1 - f^q$. Чому?
4. У формулі $1 - f^q$ немає ні k, ні порога. Що вона тоді описує і чого не описує?
5. Чому в `result.json` немає жодного поля про атаку, хоча обидві половини системи пишеш ти сам?

6. Халтурник із `honest_fraction = 0.9` майже невидимий для B. Чому це не катастрофа для протоколу — і яким одним реченням ти поясниш це на захисті?

Зламай навмисне

ЗЛАМАЙ НАВМИСНЕ

Експеримент 1. Збільшуй `sigma`: $10^{-3} \to 10^{-2} \to 10^{-1} \to 1$. На якому значенні A починає відхиляти ліниве повторення? Знайди межу з точністю до порядку і поясни, чому атакувальникові вигідно тримати `sigma` малим.

Експеримент 2. Порівняй `claim: "prev"` і `claim: "inflate"` з `inflate_by` у пів відсотка, потім у п'ять. Який варіант A ловить? Сформулюй, яку саме брехню бачить A: про роботу чи про число?

Експеримент 3. Постав `honest_fraction = 1.0` — контроль. «Атакувальник» має стати чесним вузлом побітово, і обидва механізми мають його прийняти. Проганяй це щоразу після зміни `attacks.py`. Окремо спробуй зробити лінивому вузлові точки копіями стартового стану замість інтерполяції: який механізм відхилить блок тепер і скільки обчислень це коштувало?

У ТЕКСТ РОБОТИ

Крок дає підрозділ «Моделі атак» розділу 2 (опис двох атак, їхні параметри й припущення про можливості атакувальника) і першу половину головної таблиці розділу 4. Окремо винеси формулу $1 - f^q$ разом із двома умовами, за яких вона справджується: це теоретична частина на пів сторінки і рідкісний для бакалаврської роботи випадок, коли є передбачення, яке експеримент може не підтвердити. Таблицю «очікувано за формулою / виміряно» став у текст обов'язково — розбіжність цікавіша за збіг.

Крок 6. Симулятор раундів

Мета кроку: `sim/simulate.py` — один процес Python, який за конфігом проганяє R раундів із N вузлами, веде ланцюг, роздає нагороди і пише CSV, з якого рахуються всі цифри розділу 4.

Ідея

У плані роботи є принцип, з якого починається цей крок: **спочатку симулятор, потім мережа**. Таблиці розділу 4 — те, заради чого робота існує; мережа на C++ — це розділ 3 і демонстрація на захисті. Якщо цифри залежатимуть від того, чи не гальмує TCP-синхронізація на третьому вузлі, будь-який мережевий баг у жовтні з'їдає експеримент разом із контрольною точкою 1 листопада. Симулятор розриває цю залежність: він грає всю мережу в одному процесі, без сокетів і без C++.

Звідси жорстке правило: `sim/` не імпортує нічого з `node/` і нічого не знає про TCP. Він імпортує лише `worker/` — ті самі `train_block`, `verify_a`, `verify_b`, `attacks`, що їх викликає справжній вузол. Спільним є те, що **вимірюється**; різним — те, що **доставляє**.

1. **Стан.** Ланцюг — звичайний список словників; кожен має рівно поля заголовка з Кроку 1.
2. **Раунд.** Кожен вузол бере вершину, виводить свій seed за P2 і будує кандидата — чесно або за своєю роллю в конфігу.
3. **Перегони** (B1). Кандидати впорядковуються, і перший, хто пройшов перевірку, стає блоком раунду.
4. **Перевірка.** Кожен інший вузол перевіряє кандидата ввімкненими механізмами, обираючи переходи власною приватною випадковістю (P5). Один `reject` — і кандидат відкинутий.
5. **Нагорода.** Переможець отримує одиницю; накопичені нагороди показують, чи вигідно шахраювати.

Перегони, корисна робота і чому це чесно показують

Правило B1 обрано найпростіше: перемагає перший **валідний** блок, як у PoW. Але в симуляторі «перший» треба означити явно — вузли рахуються послідовно в одному процесі, і ніякого «раніше» там немає. Означення: кандидати впорядковуються за виміряним часом побудови блока, при однаковому часі — за іменем вузла.

Наслідки два. Перший: лінивий вузол будує блок майже за нуль секунд і тому **завжди** подає першим. Це не артефакт симулятора, а правда про перегони: хто не працює, той встигає раніше. Саме тому без механізму B частка нагород атакувальника не просто «не падає», а зростає вище за його частку серед вузлів. Другий: робота всіх, хто програв раунд, викидається — при N чесних вузлах у ланцюг потрапляє один блок із N, тобто частка корисної роботи $\approx 1/N$.

Саму частку корисної роботи можна означити щонайменше трьома способами: (а) K за кожен прийнятий блок; (б) кроки, які переможець **справді** виконав (лінивий дає нуль, халтурник $f \cdot$

K); (в) те саме, але з часом верифікації у знаменнику. Беремо (б), знаменник — сума кроків, реально виконаних **усіма** вузлами за всі раунди, і записуємо це в методику розділу 4.

> ЧОМУ САМЕ ТАК
>
> Цифру $1/N$ треба показати, а не сховати: вона і є головним обмеженням варіанта «перегони», і на ній тримається аргумент на користь варіанта з дедлайном і агрегацією (`docs/design-decisions.md`, B1), який ти згадаєш у висновках як напрям розвитку. Питання «а куди подівалась робота решти вузлів?» поставлять у будь-якому разі.
>
> І головне не сам вибір означення, а те, що вибір названо: метрика без означення в тексті — це число, яке розсиплеться від першого питання на захисті.

Нові слова

слово	що означає
`tip`	Вершина ланцюга — останній прийнятий блок. Від нього рахується наступний раунд.
`кандидат`	Блок, поданий вузлом, але ще не прийнятий. У раунді їх стільки, скільки вузлів.
`перегони (B1)`	Правило раунду: перемагає перший валідний блок. Прямий аналог PoW.
`run seed`	Seed усього прогону. Один прогін — один seed — один файл CSV.
`baseline`	Централізоване навчання тієї самої моделі на тій самій кількості кроків. Без нього крива точності ні про що не говорить.
`ground truth`	Правда, відома експериментаторові (хто атакував), але не доступна протоколу.

Контракт: конфіг і блок

`sim/configs/e2-lazy1.json`

```
{
  "name": "e2-lazy1",
  "rounds": 40, "dataset": "mnist", "genesis_seed": 12345,
  "K": 200, "k": 20, "q": 2, "batch_size": 64, "lr": 0.01,
  "mechanisms": ["A", "B"],
  "sample_size": 2000,
  "threshold_rule": {"kind": "relative", "alpha": 1.0},
  "seeds": [1, 2, 3, 4, 5],
  "nodes": [
    {"name": "n1", "role": "honest"},
    {"name": "n2", "role": "honest"},
    {"name": "n3", "role": "lazy", "sigma": 0.001, "claim": "prev"}
  ]
}
```

У конфігу, а не в коді, живе все, що змінюється від експерименту до експерименту: `nodes` — єдине місце, де записано, хто атакує; `mechanisms` перетворює `["A"]` і `["A","B"]` на два різні експерименти з тим самим кодом; `q` і `sample_size` — ручки компромісу «виявлення проти витрат»; `threshold_rule` дозволяє порівняти відносний поріг з абсолютним (P6); `seeds` задає повні незалежні прогони, і менше п'яти їх не беруть.

> **УВАГА**
> Роль вузла з конфіга — ground truth. Вона потрапляє в CSV і **ніколи** — у код перевірки. Якщо `validate` десь прочитає `role`, усі detection rate стануть одиницями, а робота — недійсною. Найпростіший захист: функція перевірки приймає **блок**, а не вузол-кандидат разом зі службовими даними.

блок як словник Python

```
{
  "version": 1, "height": 12, "prev_hash": "00c8...", "task_id": "9f2c...",
  "round": 12, "miner_id": "4ab1...", "timestamp": 1761400000,
  "w_start": "7d31...", "w_end": "b0e4...", "ckpt_root": "5c02...",
  "K": 200, "k": 20, "metric_micro": 912300,
  "block_hash": "0a77...",          # SHA-256 від 236 байтів заголовка
  "ckpt_hashes": ["7d31...", "...", "b0e4..."]
}
```

> **ЧОМУ САМЕ ТАК**
> Жодного додаткового поля тут бути не може: службові дані симулятора (роль вузла, час побудови, справжня кількість кроків) живуть у паралельному словнику `meta`, який у ланцюг не входить. У Кроці 7 C++-вузол читатиме такі самі структури, і якщо в блоці з'явиться поле `role` чи `seconds`, у C++ доведеться або відтворювати поле, якого в протоколі немає, або писати окремий розбір для «блоків із симулятора».

Контракт: CSV, похідні метрики і базова лінія

Файлів два, бо питань два: «як ішли раунди» і «як спрацьовували перевірки».

experiments/results/<config>-<seed>.csv — один рядок на раунд

```
run_id,seed,round,winner,winner_role,was_attack,n_candidates,n_rejected,
verdicts,accuracy,rewards,steps_real,steps_accepted,seconds_train,seconds_verify
```

experiments/results/<config>-<seed>-checks.csv — один рядок на перевірку

```
run_id,round,candidate,candidate_role,validator,mechanism,k,q,
verdict,distance,threshold,seconds_verify
```

`run_id` — це `<ім'я конфіга>-<seed>`, той самий рядок, що й у назві файлу. `was_attack` — ground truth з конфіга, а не висновок протоколу. `verdicts` і `rewards` пишуться компактно: `n1:accept;n2:reject` і `n1:7;n2:5;n3:0`. `steps_real` — скільки кроків реально виконали всі вузли разом, `steps_accepted` — скільки з них виконав переможець насправді (означення (б)). Порожній `winner` означає раунд, у якому не пройшов жоден кандидат: такі раунди теж результат і теж дають рядок.

Імена колонок у `checks.csv` навмисне ті самі, що в `detect_matrix.py` із Кроку 5: там ти міряєш механізми в пробірці, тут — у мережі, а числа лишаються порівнюваними. Два файли потрібні через різну зернистість: у раунді з трьома вузлами перевірок буде до шести, і запхати їх у колонки раундового файлу означало б рядки змінної ширини.

похідна метрика	як рахується після прогонів
detection rate	серед кандидатів із `candidate_role != honest` — частка тих, кого хоч один валідатор відхилив; рахувати по кандидатах, а не по рядках
false rejection rate	те саме серед `candidate_role == honest`, на тому самому k і тому самому порозі
накладні витрати	сума `seconds_verify` до суми `seconds_train` **чесних** кандидатів
частка нагород	нагороди ролей $\neq$ `honest` до всіх виданих нагород
корисна робота	сума `steps_accepted` до суми `steps_real`

Базова лінія — `sim/baseline.py`, файл `baseline-<dataset>-<seed>.csv` з колонками `block,steps,accuracy,seconds`: централізоване навчання тієї самої моделі, той самий старт (`build_model(genesis_seed)`), ті самі `lr` і `batch_size`, усього `rounds * K` кроків, точність — кожні K кроків, щоб точки кривих збігалися з раундами.

> Чесне застереження в текст роботи: порівняння йде за **кількістю кроків**, а не за даними. У симуляторі порядок даних міняється з кожним блоком, бо seed прив'язаний до майнера (P2); у базовій лінії він один на весь прогін. Тому невелика різниця кривих очікувана, і видавати її за ефект протоколу не можна.

Скелет

`sim/simulate.py` · тіла написати самому

```
def build_candidate(node, tip, cfg, data, round_no):
    # 1. seed = derive_seed(bytes.fromhex(tip["block_hash"]),
    #    miner_id(node), round_no).
    # 2. За node["role"]: honest -> train_block, lazy -> attack_lazy,
    #    partial -> attack_partial. Час — time.perf_counter().
    # 3. save_state для всіх точок; збери ckpt_hashes і ckpt_root.
    # 4. Склади словник заголовка рівно за контрактом Кроку 1, порахуй block_hash.
    # 5. Поверни (block, meta): meta — роль, час, справді виконані кроки.
    #    У блок meta не входить.


def validate(block, validator, cfg, data, rng):
    # 1. Безкоштовні структурні перевірки: prev_hash == tip.block_hash,
    #    w_start == tip.w_end, довжина ckpt_hashes, ckpt_root, K і k як у завданні.
    #    Не пройшло — reject без жодного перерахунку.
    # 2. Для кожного механізму з cfg["mechanisms"]:
    #    "A" -> verify_a на підвибірці розміру cfg["sample_size"];
    #    "B" -> rng.sample(range(m), cfg["q"]) і verify_b на кожному з них.
    # 3. rng — приватний генератор ЦЬОГО валідатора. Ніколи не виводь його
    #    з block_hash: інакше ти сам реалізуєш вразливість, описану в P5.
    # 4. Поверни (verdict, records): verdict = reject, якщо хоч одна reject.


def run_round(chain, nodes, cfg, data, rngs, rewards, round_no):
    # 1. Побудуй кандидата від кожного вузла.
    # 2. Впорядкуй за (seconds_train, name) — це і є «перегони».
    # 3. Іди по черзі: кожного кандидата перевіряють усі інші вузли. Перший,
    #    кого ніхто не відхилив, стає блоком раунду; далі не перевіряй.
    # 4. Жоден не пройшов -> раунд порожній: ланцюг не росте, нагороди не
    #    видаються, але рядок у CSV пишеться.
```

```
    # 5. Нагорода переможцю += 1; chain.append(block); порахуй точність
    #    нової вершини на тестовому наборі.
    # 6. Поверни рядок раунду і список рядків для checks.csv.


def main():
    # 1. argparse: шлях до конфіга, --out-dir, --seed (перекриває список).
    # 2. Для кожного seed — окремий прогін з нуля: свій ланцюг, свої нагороди,
    #    свої генератори валідаторів, свій файл.
    # 3. Генезис: build_model(cfg["genesis_seed"]) -> save_state -> блок висоти 0.
    # 4. Цикл раундів; після кожного — writerow і flush в обидва файли.
    # 5. Нічого не усереднюй і не малюй. Друкуй лише шляхи до файлів.
```

ПІДКАЗКИ

Рівень 1. Почни з найпростішого світу: усі вузли чесні, `mechanisms` порожній, ніхто нікого не перевіряє. Домагайся, щоб ланцюг ріс, а точність у CSV зростала. Механізми і ролі додавай у робочий цикл — інакше не зрозумієш, що саме зламалось: раунд, перевірка чи навчання.

Рівень 2. Ланцюг — список словників, вершина `chain[-1]`. Приватна випадковість валідатора: `random.Random(f"{seed}-{round_no}-{name}")` — свій генератор на вузол, створений раз на прогін; вибір переходів — `rng.sample(range(m), q)`.

Рівень 3. Каркас перегонів:

```
order = sorted(cands, key=lambda c: (c["seconds_train"], c["name"]))
for cand in order:
    peers = [n for n in names if n != cand["name"]]
    vs = [validate(cand["block"], p, cfg, data, rngs[p])[0] for p in peers]
    if all(v == "accept" for v in vs):
        winner = cand; break
```

Твоє завдання

ТВОЄ ЗАВДАННЯ

1. Напиши `sim/simulate.py` і `sim/baseline.py`.
2. Зроби три конфіги: `e0-honest3.json`, `e2-lazy1-a.json` (два чесні + лінивий, `mechanisms: ["A"]`) і `e2-lazy1-ab.json` (те саме з `["A","B"]`). Різниця між двома останніми — рівно один рядок конфіга.
3. Прожени `e0` на трьох seed і побудуй графік точності за раундами разом із кривою baseline.
4. Прожени обидва `e2` на п’яти seed і порахуй частку нагород лінивого вузла в кожному випадку.
5. Додай конфіг із халтурником (`honest_fraction: 0.1`) і порівняй виміряну частку `reject` із передбаченням $1 - f^{q \cdot V}$ з Кроку 5. Тут $V = 2$ валідатори — число буде інше.
6. Напиши короткий `experiments/summarize.py`, який із CSV дає таблицю з п’ятьма похідними метриками. Жодних обчислень метрик усередині симулятора.
7. Запиши в `docs/` обране означення частки корисної роботи — це прямий текст у методику.
8. Коміт: `feat(sim): round simulator, configs and baseline`.

Перевірка

ПЕРЕДБАЧ ДО ЗАПУСКУ

1. Три чесні вузли, 40 раундів, $K = 200$: скільки кроків виконає мережа разом і скільки з них потрапить у ланцюг?
2. Крива симулятора проти baseline на тій самій кількості кроків: яка буде вище і чому?
3. Один лінивий вузол із трьох, увімкнено лише A. Яку частку нагород він візьме: приблизно третину, більше чи менше?

ЕТАЛОН ДЛЯ САМОПЕРЕВІРКИ

1. **Усі чесні**: крива точності росте за раундами і йде приблизно поряд із кривою централізованого навчання на тій самій кількості кроків. «Приблизно» — бо порядок даних інший. Якщо крива симулятора помітно нижча, перше, що перевіряй, — чи справді старт спільний.
2. **Один лінивий + механізм B**: частка нагород лінивого падає практично до нуля; у CSV це видно як `n_rejected >= 1` майже в кожному раунді при чесному `winner`.
3. **Той самий склад, лише A**: частка нагород лінивого не падає. Оскільки він «навчається» за нуль секунд і подає першим, вона цілком може виявитись найбільшою з трьох, а точність виходить на плато і стоїть.
4. **Частка корисної роботи** при N чесних вузлах — близько $1/N$, і вона не залежить від кількості раундів. Залежить — шукай помилку в знаменнику.

САМОПЕРЕВІРКА

1. Чому `sim/` не імпортує нічого з `node/`? Назви два наслідки: один для 1 листопада, другий для розділу 4.
2. Чотири вузли навчали, у ланцюг увійшов один блок. Що таке «корисна робота» в цьому раунді? Дай означення, яке зможеш захистити, і назви альтернативне.
3. Чому `was_attack` не можна давати коду перевірки, хоча обидві половини системи пишеш ти?
4. Чому seed лежить у кожному рядку CSV, хоча він уже є в назві файлу, — і чому рядок пишеться на кожен раунд, а не один на прогін?
5. Лінивий вузол подає блок першим, бо будує його за нуль секунд. Це баг симулятора чи властивість правила B1?
6. Чому baseline тренується на $R \cdot K$ кроках, а не на $R \cdot K \cdot N$?

Зламай навмисне

ЗЛАМАЙ НАВМИСНЕ

Експеримент 1. Заміни приватний генератор валідатора на вибір переходів із хеша блока (`random.Random(block["block_hash"])`), а потім дай халтурникові перебирати `timestamp`, доки

обидва «вибрані» переходи не потраплять у його чесну частину. Скільки спроб це займає? Порівняй із P5 — це готовий аргумент у розділ 2, підкріплений власним вимірюванням.

Експеримент 2. Зроби всі вузли лінивими, лиши механізм A. Ланцюг росте, блоки приймаються, нагороди видаються — а точність стоїть. Сформулюй одним реченням, що саме доводить твердження «ланцюг працює» і чого воно не доводить.

Експеримент 3. Прибери сортування за часом і бери кандидатів у порядку конфіга. Як змінюється частка нагород лінивого вузла? Що це говорить про те, наскільки детально треба описувати правило раунду в методиці?

Експеримент 4. Постав $q = 1$ і `honest_fraction = 0.5`, прожени п'ять seed. Розкид між прогонами має бути великим — поясни, чому одна крива тут нічого не доводить і звідки в плані взялася вимога «щонайменше п'ять seed».

Вправи

1. Прожени $N = 2, 3, 5$ чесних вузлів і побудуй графік частки корисної роботи від N. Чи лягає він на $1/N$? Якщо ні — що ще з'їдає роботу?
2. Навчи симулятор писати ще й `experiments/results/<run_id>-chain.json` — весь ланцюг. У Кроці 7 C++-вузол має прочитати ці блоки без жодного поля-сироти. П'ятихвилинна вправа, яка в листопаді економить вечір.

У ТЕКСТ РОБОТИ

Симулятор — це розділ 4 цілком: методика (конфіг, seed, правило раунду, означення метрик), таблиці E0 і E2 з плану і рисунок «точність за раундами проти централізованого навчання». Опис правила B1 і чесне обговорення частки корисної роботи винеси окремим підрозділом в обмеження: комісія оцінює не «у мене все вийшло», а «я знаю, де в мене слабко, і можу пояснити, чому це прийнятно для поставленої задачі».

КОНТРОЛЬНА ТОЧКА · 25 ЖОВТНЯ 2026

Після Кроків 5–6 у тебе мають бути перші справжні цифри — навіть якщо C++-мережа ще не працює:

1. detection rate для двох атак (лінива, часткова) на механізмах A і B, щонайменше з 20 прогонів кожна;
2. false rejection rate чесних вузлів на тому самому k і тому самому порозі;
3. накладні витрати: час верифікації до часу чесного навчання;
4. крива точності за раундами для трьох чесних вузлів поряд із централізованим навчанням на тій самій кількості кроків, щонайменше на трьох seed;
5. частка нагород лінивого вузла окремо з механізмом A і з механізмом B.

Це п'ять чисел і один графік. Вони закривають пункт «перші цифри» контрольної точки 1 листопада незалежно від стану мережі, і саме їх показуй керівникові першими: розмова про роботу, у якій є цифри, відрізняється від розмови про роботу, у якій є плани.

> Якщо станом на 25 жовтня цифр немає — ріж не експеримент, а обсяг: один механізм (B), одна атака (лінива), три seed. Маленька таблиця в жовтні краща за порожню в листопаді.

Крок 7. Блок і канонічна серіалізація

Мета кроку: C++-частина вміє скласти 236 байтів заголовка і порахувати від них `block_hash`, і цей хеш побайтово збігається з тим, що порахував Python у Кроці 1.

Ідея

Досі C++-вузол умів лише виводити рядок у термінал. Тепер він починає працювати з тим самим об'єктом, що й Python, — із заголовком блоку. Уся складність кроку в одному реченні: **дві мови мають отримати те саме число**.

Здавалося б, тривіально: структура з тринадцятьма полями, SHA-256 від неї, готово. Насправді саме тут живе найдорожчий баг проєкту. Він не падає, не дає попереджень і не видно в логах — просто у Python виходить один `block_hash`, а в C++ інший, і вузол мовчки відкидає всі чесні блоки. Шукати таке через три тижні, коли вже є мережа й воркер, — це втрачений тиждень.

Тому крок влаштовано так: спершу серіалізація і хеш, одразу тести, і останнім тестом — звірка із золотим вектором із `docs/golden.json`. Поки цей тест не зелений, далі йти не можна.

> ЧОМУ САМЕ ТАК
>
> Чому серіалізація пишеться руками, байт за байтом, а не одним `memcpy` структури?
>
> **Заповнення (padding).** Компілятор має право вставляти невидимі байти між полями, щоб кожне лежало за адресою, кратною його вирівнюванню: `version` — це 4 байти, а наступне поле `height` вимагає кратності 8, тож між ними з'являється діра. Вміст цих байтів стандарт не визначає. Хеш від структури з такими дірами — це хеш від сміття.
>
> **Порядок байтів і свобода компілятора.** `memcpy` копіює те подання, яке має ця машина, і ту розкладку, яку обрав цей компілятор. Розкладка структури — частина ABI, а не мови: інший прапорець чи інша версія можуть змінити її мовчки.
>
> Ручна серіалізація прибирає обидві проблеми: ти сам кажеш, який байт куди йде. Ціна — двадцять рядків, які пишуться один раз. І та сама причина, з якої в Кроці 1 ми заборонили хешувати JSON: хеш береться лише від канонічних байтів, ніколи від тексту.

Нові слова

слово	що означає
`std::array<T, N>`	Масив фіксованої довжини, який поводиться як нормальний тип: копіюється, порівнюється, знає розмір. Наш `Hash` — це він.
`вирівнювання (alignment)`	Вимога, щоб адреса об'єкта ділилась на певне число. `std::uint64_t` зазвичай вимагає кратності 8.

слово	що означає
заповнення (padding)	Невидимі байти, які компілятор вставляє між полями заради вирівнювання. Їхній вміст не визначений.
sizeof	Розмір типу в пам'яті разом із заповненням. Це **не** розмір канонічної серіалізації.
static_assert	Перевірка під час компіляції. Бачить лише те, що може порахувати компілятор.
reinterpret_cast	Погляд на ті самі байти як на інший тип. У серіалізації протоколу — заборонений прийом.
EVP_Digest	Функція OpenSSL, яка рахує дайджест за один виклик; сучасна заміна застарілого `SHA256()`.
libssl-dev	Пакет із заголовками OpenSSL. Без нього `find_package(OpenSSL)` не знайде нічого.

Контракт

Три заголовки. Набирай як є: це домовленість, а не робота.

node/include/hex.h

```
#pragma once
#include <array>
#include <cstdint>
#include <string>

namespace pouw {

using Hash = std::array<std::uint8_t, 32>;

// 32 байти -> 64 шістнадцяткові символи нижнього регістру.
std::string to_hex(const Hash& h);

// 64 символи (будь-який регістр) -> 32 байти.
// Кидає std::invalid_argument, якщо довжина не 64 або трапився не-hex символ.
Hash from_hex(const std::string& s);

// Хеш із самих нулів: prev_hash генезис-блоку.
Hash zero_hash();

}  // namespace pouw
```

node/include/sha256.h

```
#pragma once
#include <cstddef>
#include <cstdint>
#include <string>
#include <vector>
#include "hex.h"

namespace pouw {

// SHA-256 від len байтів за адресою data. Основна форма.
Hash sha256(const std::uint8_t* data, std::size_t len);
// Обгортки для зручності; тіла — у sha256.cpp, обидва по одному рядку.
```

```
Hash sha256(const std::vector<std::uint8_t>& v);
Hash sha256(const std::string& s);

}  // namespace pouw
```

node/include/block.h

```
#pragma once
#include <cstdint>
#include <vector>
#include "hex.h"

namespace pouw {

// Заголовок блоку. Рівно 236 байтів у канонічному вигляді.
// Увага: sizeof(BlockHeader) НЕ дорівнює 236 і дорівнювати не зобов'язаний.
struct BlockHeader {
    std::uint32_t version = 1;
    std::uint64_t height = 0;
    Hash prev_hash{};
    Hash task_id{};
    std::uint64_t round = 0;
    Hash miner_id{};
    std::uint64_t timestamp = 0;
    Hash w_start{};
    Hash w_end{};
    Hash ckpt_root{};
    std::uint32_t K = 0;
    std::uint32_t k = 0;
    std::int64_t metric_micro = 0;
};

inline constexpr std::size_t HEADER_BYTES = 236;

// Канонічне подання: рівно HEADER_BYTES байтів, усі числа little-endian,
// поля в порядку оголошення. Це єдине, від чого рахується хеш блоку.
std::vector<std::uint8_t> serialize(const BlockHeader& h);
// SHA-256 від serialize(h).
Hash block_hash(const BlockHeader& h);
// SHA-256 від конкатенації 32-байтових хешів контрольних точок C_0 .. C_m.
Hash ckpt_root_of(const std::vector<Hash>& ckpts);

}  // namespace pouw
```

УВАГА

Порядок полів у структурі збігається з канонічним порядком **навмисно**: так серіалізацію легше звіряти з таблицею Кроку 1. Але це зручність для людини, а не механізм. Якщо колись переставиш поля у структурі, канонічний порядок має лишитись тим самим — інакше всі раніше створені блоки стануть невалідними.

CMakeLists.txt — після змін

```
cmake_minimum_required(VERSION 3.20)
project(pouwchain LANGUAGES CXX)
set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_EXPORT_COMPILE_COMMANDS ON)
add_compile_options(-Wall -Wextra -Wpedantic)

find_package(OpenSSL REQUIRED)
```

```
add_library(pouw_core STATIC
    node/src/hex.cpp
    node/src/sha256.cpp
    node/src/block.cpp
)
target_include_directories(pouw_core PUBLIC node/include)
target_link_libraries(pouw_core PUBLIC OpenSSL::Crypto)

add_executable(node node/src/main.cpp)
target_link_libraries(node PRIVATE pouw_core OpenSSL::Crypto)

enable_testing()
add_executable(test_block node/tests/test_block.cpp)
target_link_libraries(test_block PRIVATE pouw_core)
add_test(NAME block COMMAND test_block ${CMAKE_SOURCE_DIR}/docs/golden.json)
```

ЧОМУ САМЕ ТАК

Чому з'явилась бібліотека `pouw_core`, а не просто список файлів у цілі `node`? Бо той самий код потрібен трьом цілям: вузлу і двом тестовим програмам (у Кроці 8 буде третя). Якщо перелічувати файли в кожній цілі, рано чи пізно вони розійдуться, і тест збиратиме не той код, що вузол. `PUBLIC` у `target_include_directories` означає «усі, хто лінкується до `pouw_core`, теж бачать теку `node/include`» — саме тому тестова ціль цього рядка не повторює.

Якщо `find_package(OpenSSL REQUIRED)` нічого не знаходить: на Debian/Ubuntu це `sudo apt install libssl-dev`, на Fedora — `openssl-devel`. Потрібні саме заголовки: без пакета `-dev` у системі є `libcrypto.so`, але немає `openssl/evp.h`.

Скелет

node/src/hex.cpp · тіла написати самому

```
#include "hex.h"
#include <stdexcept>

namespace pouw {

std::string to_hex(const Hash& h) {
    // 1. Заведи рядок, зарезервуй 64 символи.
    // 2. Для кожного байта додай два символи: старший півбайт (b >> 4), молодший (b &
    //     0x0F).
    // 3. Цифру 0..15 перетворюй через таблицю "0123456789abcdef" — коротше й швидше
    //     за printf.
}

Hash from_hex(const std::string& s) {
    // 1. s.size() != 64 -> кинь std::invalid_argument із зрозумілим текстом.
    // 2. Локальна функція nibble(char) -> int: '0'..'9', 'a'..'f', 'A'..'F', інакше
    //     виняток.
    //    Регістр приймаємо обидва, віддаємо завжди нижній.
    // 3. Збери байт із двох півбайтів: (hi << 4) | lo.
}

Hash zero_hash() {
    // 1. Один рядок: Hash{} уже заповнений нулями.
}
```

```
}  // namespace pouw
```

node/src/sha256.cpp · тіла написати самому

```
#include "sha256.h"
#include <openssl/evp.h>
#include <stdexcept>

namespace pouw {

Hash sha256(const std::uint8_t* data, std::size_t len) {
    // 1. Hash out{}; unsigned int out_len = 0;
    // 2. Один виклик: EVP_Digest(data, len, out.data(), &out_len, EVP_sha256(),
    //     nullptr).
    // 3. Повернув не 1 або out_len != 32 -> кинь std::runtime_error. Мовчки віддавати
    //     нулі
    //    не можна: нульовий хеш виглядає як цілком валідне значення.
    // 4. Поверни out.
}

Hash sha256(const std::vector<std::uint8_t>& v) {
    // 1. Делегуй основній формі. Порожній вектор: v.data() може бути nullptr, а len =
    //     0 —
    //    це коректний випадок, SHA-256 від порожнього входу існує.
}

Hash sha256(const std::string& s) {
    // 1. Те саме, вказівник приведи: reinterpret_cast<const std::uint8_t*>(s.data()).
    //    Тут приведення доречне: ми не тлумачимо число, а дивимось на байти рядка.
}

}  // namespace pouw
```

node/src/block.cpp · тіла написати самому

```
#include "block.h"
#include <stdexcept>
#include "sha256.h"

namespace pouw {
namespace {

void put_u32(std::vector<std::uint8_t>& out, std::uint32_t v) {
    // 1. Додай чотири байти в порядку (v >> 0), (v >> 8), (v >> 16), (v >> 24),
    //    кожен звузивши до std::uint8_t через static_cast.
    // 2. Жодного memcpy і reinterpret_cast: зсув працює зі ЗНАЧЕННЯМ числа, а не з його
    //    поданням у пам'яті, тому результат однаковий на будь-якій машині.
}

void put_u64(std::vector<std::uint8_t>& out, std::uint64_t v) {
    // 1. Те саме вісім разів, циклом по i = 0..7, зсув на 8 * i.
}

void put_i64(std::vector<std::uint8_t>& out, std::int64_t v) {
    // 1. static_cast до std::uint64_t: це визначена операція за модулем 2^64,
    //     від'ємні числа
    //    дають саме доповняльний код, який чекає Python. Далі put_u64.
}

void put_hash(std::vector<std::uint8_t>& out, const Hash& h) {
    // 1. out.insert(out.end(), h.begin(), h.end()).
}
```

```
}  // namespace

std::vector<std::uint8_t> serialize(const BlockHeader& h) {
    // 1. Вектор, одразу reserve(HEADER_BYTES): інакше кілька перевиділень на рівному
    //     місці.
    // 2. Виклич put_* рівно в порядку таблиці: version, height, prev_hash, task_id,
    //     round,
    //    miner_id, timestamp, w_start, w_end, ckpt_root, K, k, metric_micro.
    // 3. Перевір out.size() == HEADER_BYTES, інакше std::logic_error: це помилка
    //     твого коду,
    //    а не вхідних даних.
}

Hash block_hash(const BlockHeader& h) {
    // 1. Два рядки: серіалізувати і віддати в sha256. Більше тут нічого бути не має.
}

Hash ckpt_root_of(const std::vector<Hash>& ckpts) {
    // 1. Порожній список -> std::invalid_argument: блок без точок не має сенсу, а
    //     мовчазний
    //    хеш від нічого приховав би баг.
    // 2. Склей усі 32-байтові хеші підряд (reserve(32 * ckpts.size())) і поверни
    //     sha256.
}

}  // namespace pouw
```

ПІДКАЗКИ

Рівень 1. Почни з `hex.cpp`: він ні від чого не залежить і дає одразу зелений тест. Далі `sha256.cpp` (три рядки корисного коду), і лише потім `block.cpp`.

Рівень 2. Таблиця для `to_hex`: `static const char* D = "0123456789abcdef";`, далі `D[b >> 4]` і `D[b & 0x0F]`. У `put_u32` потрібен явний `static_cast<std::uint8_t>(v >> 8)`: без нього компілятор має рацію, коли попереджає про втрату.

Рівень 3. Каркас `put_u64`:

```
for (int i = 0; i < 8; ++i) {
    out.push_back(static_cast<std::uint8_t>(v >> (8 * i)));
}
```

Той самий цикл із межею 4 — це `put_u32`.

Тести

Набирай як є. Це не робота, це доказ.

`node/tests/test_block.cpp`

```
#include <cstdio>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>
#include "block.h"
#include "sha256.h"

using namespace pouw;
static int failed = 0;
```

```
#define CHECK(cond) do { if (!(cond)) { \
    std::printf("FAIL %s:%d   %s\n", __FILE__, __LINE__, #cond); ++failed; } } while (0)

// Той самий заголовок, від якого Python рахував золотий вектор у Кроці 1.
static BlockHeader sample_header() {
    BlockHeader h;
    h.version = 1;  h.height = 0;  h.round = 0;  h.timestamp = 0;
    h.prev_hash = from_hex(std::string(64, '0'));
    h.task_id   = from_hex(std::string(64, '1'));
    h.miner_id  = from_hex(std::string(64, '2'));
    h.w_start   = from_hex(std::string(64, '3'));
    h.w_end     = from_hex(std::string(64, '4'));
    h.ckpt_root = from_hex(std::string(64, '5'));
    h.K = 500;  h.k = 25;  h.metric_micro = 912300;
    return h;
}

static std::string read_file(const std::string& path) {
    std::ifstream in(path);
    std::stringstream ss;  ss << in.rdbuf();  return ss.str();
}

// Груба вибірка 64 hex-символів після ключа. Це тест, а не парсер JSON.
static std::string json_hex_field(const std::string& js, const std::string& key) {
    std::size_t p = js.find("\"" + key + "\"");
    if (p == std::string::npos) return "";
    p = js.find(':', p);   if (p == std::string::npos) return "";
    p = js.find('"', p);   if (p == std::string::npos) return "";
    return js.substr(p + 1, 64);
}

int main(int argc, char** argv) {
    // 1. Сам SHA-256: дві всесвітньо відомі константи.
    CHECK(to_hex(sha256(std::string("abc"))).rfind("ba7816bf", 0) == 0);
    CHECK(to_hex(sha256(std::string(""))).rfind("e3b0c442", 0) == 0);

    // 2. hex туди й назад.
    const std::string s(64, 'a');
    CHECK(to_hex(from_hex(s)) == s);
    CHECK(to_hex(from_hex(std::string(64, 'A'))) == std::string(64, 'a'));

    // 3. Довжина канонічного подання.
    const BlockHeader h = sample_header();
    CHECK(serialize(h).size() == 236);

    // 4. Будь-яка зміна будь-якого поля змінює block_hash.
    const Hash base = block_hash(h);
    BlockHeader t = h; t.height = 1;    CHECK(block_hash(t) != base);
    t = h; t.K = 501;                   CHECK(block_hash(t) != base);
    t = h; t.k = 26;                    CHECK(block_hash(t) != base);
    t = h; t.timestamp = 1;             CHECK(block_hash(t) != base);
    t = h; t.metric_micro = 912301;     CHECK(block_hash(t) != base);
    t = h; t.w_end[31] = 0x45;          CHECK(block_hash(t) != base);
    t = h; t.task_id[0] = 0x12;         CHECK(block_hash(t) != base);

    // 5. ckpt_root — це SHA-256 від склеєних хешів, без жодних роздільників.
    const std::vector<Hash> two = {h.w_start, h.w_end};
    std::vector<std::uint8_t> cat;
    cat.insert(cat.end(), h.w_start.begin(), h.w_start.end());
    cat.insert(cat.end(), h.w_end.begin(), h.w_end.end());
    CHECK(ckpt_root_of(two) == sha256(cat));

    // 6. Головний тест кроку: те саме число, що порахував Python.
    const std::string path = (argc > 1) ? argv[1] : "docs/golden.json";
```

```
    const std::string golden = json_hex_field(read_file(path), "block_hash");
    CHECK(golden.size() == 64);
    CHECK(to_hex(block_hash(h)) == golden);

    if (failed == 0) std::printf("test_block: OK\n");
    else             std::printf("test_block: %d checks FAILED\n", failed);
    return failed == 0 ? 0 : 1;
}
```

Файл золотих векторів має містити ключ `block_hash`, і саме один раз:

docs/golden.json — форма

```
{
  "header": {
    "version": 1, "height": 0, "round": 0, "timestamp": 0,
    "prev_hash": "0000...0000",  "task_id": "1111...1111",
    "miner_id": "2222...2222",   "w_start": "3333...3333",
    "w_end":    "4444...4444",   "ckpt_root": "5555...5555",
    "K": 500, "k": 25, "metric_micro": 912300
  },
  "block_hash": "<64 hex, порахований Python-ом у Кроці 1>"
}
```

УВАГА

Значення `block_hash` не можна взяти з цього гайда, з інтернету чи з голови. Воно породжується твоїм Python-кодом у Кроці 1 і є доказом лише тому, що дві незалежні реалізації на ньому зійшлись. Якщо спершу порахувати його в C++ і записати у `golden.json`, тест перевірятиме, що C++ дорівнює C++, і не викриє нічого. Значення в заголовку — повні 64 символи (`"00" * 32` тощо), крапки вище поставлені лише для стислості гайда.

Твоє завдання

ТВОЄ ЗАВДАННЯ

1. Набери три заголовки й новий `CMakeLists.txt`.
2. Напиши тіла в `hex.cpp`, `sha256.cpp`, `block.cpp`.
3. Набери `test_block.cpp` як є.
4. Перед першим запуском додай у `main.cpp` тимчасовий друк `sizeof(pouw::BlockHeader)`, але **спершу** порахуй це число на папері (див. «Передбач»).
5. Збери й запусти: `cmake -B build -DCMAKE_BUILD_TYPE=Release && cmake --build build -j`, далі `ctest --test-dir build --output-on-failure`.
6. Доведи тест до зеленого стану, включно із золотим вектором.
7. Коміт: `feat(node): canonical block serialization and sha256`.

Перевірка

ПЕРЕДБАЧ ДО ЗАПУСКУ

Запиши відповіді **до** запуску:

1. Скільки покаже `sizeof(BlockHeader)`? Порахуй по полях, з урахуванням вирівнювання на 8 для 64-бітних полів. Де саме сидить заповнення і скільки його?
2. Чи буде `sizeof(BlockHeader) == 236`? Якщо ні — це помилка в структурі?
3. Тест 4 змінює `w_end[31]` з `0x44` на `0x45` — один біт. Скільки приблизно символів у hex зміниться?
4. Що станеться з тестом 6, якщо в `put_u32` писати байти від старшого до молодшого?

ЕТАЛОН ДЛЯ САМОПЕРЕВІРКИ

1. `sha256("abc")` починається з `ba7816bf`, `sha256("")` — з `e3b0c442`. Це не наші числа, а константи стандарту: не збіглись — зламано найпростішу ланку.
2. `serialize(...).size()` дорівнює рівно 236, не «приблизно».
3. `sizeof(BlockHeader)` на звичайному 64-бітному ABI дорівнює 240: чотири байти заповнення між `version` і `height`, решта полів лягає без дір. Саме через них `static_assert(sizeof(BlockHeader) == 236)` не скомпілювався б — і це правильно.
4. `ctest` показує `100% tests passed`, і серед перевірок є звірка із золотим вектором.
5. Точного значення `block_hash` еталон не наводить навмисно: правильним є не конкретне число, а **рівність** твого C++ і твого Python.

САМОПЕРЕВІРКА

1. Чому `static_assert(sizeof(BlockHeader) == 236)` — погана перевірка, навіть якби вона проходила? Назви дві речі, яких вона не побачить.
2. Чому зсув `v >> 8` не залежить від порядку байтів машини, а `memcpy` залежить?
3. Ти додав у структуру поле `nonce`. Які тести впадуть? Які **мали б** упасти, але не впадуть, якщо не чіпати серіалізацію?
4. Навіщо `ckpt_root` окремо, якщо список точок і так лежить у блоці?
5. Чому `sha256` кидає виняток замість того, щоб повернути нульовий хеш при збої OpenSSL?

Зламай навмисне

ЗЛАМАЙ НАВМИСНЕ

Експеримент 1. memcpy замість руки. Тимчасово заміни тіло `serialize` на `std::vector<std::uint8_t> out(sizeof(h)); std::memcpy(out.data(), &h, sizeof(h)); return out;`

Що сказав тест на довжину? Далі познач структуру як `__attribute__((packed))`, зберись знову і подивись на золотий вектор. Збігся чи ні? Якщо збігся — поясни, чому такий код усе одно неправильний: назви три речі, від яких він тепер мовчки залежить, і згадай, що перевірка проходить лише на **твоїй** машині з **твоїм** компілятором.

Експеримент 2. Слабкий тест. Поміняй місцями виклики `put_u32` для `K` і `k`. Довжина лишилась 236, тест 3 зелений. Які тести впали? А тепер уяви, що тесту із золотим вектором немає: як довго ця помилка прожила б?

Експеримент 3. Зникле поле. Прибери з `serialize` рядок для `ckpt_root`, а замість нього допиши `put_hash(out, h.w_end)` ще раз. Довжина знову 236. Який тест це ловить? Тепер **посили тестовий набір**: додай перевірку, яка бере список змін «по одному полю за раз» і вимагає, щоб кожна змінювала `block_hash`. Таку перевірку неможливо обманути збігом довжини.

У ТЕКСТ РОБОТИ

Крок дає підрозділ «Структура блоку і канонічне подання» розділу 3. Таблиця з Кроку 1 стає рисунком, а тут додається аргументація: чому серіалізація ручна, чому `sizeof` не дорівнює 236 і чому доказом узгодженості двох реалізацій є золоті вектори, а не читання коду. Це один із найпростіших для пояснення на захисті фрагментів — не пропусти його.

Крок 8. Ланцюг і правила валідації

Мета кроку: структура `Chain`, яка приймає лише правильні блоки, зберігається на диск, відновлюється з диска і переходить на довшу гілку. Без мережі й без навчання.

Ідея

Ланцюг — це пам'ять мережі. Але цікавий не сам список блоків, а те, **кого в нього не пускають**. Правила прийняття блоку і є протокол; решта коду — обслуга.

Головне рішення кроку — розділити перевірку на дві.

	структурна перевірка	семантична перевірка
питання	чи блок складений за правилами	чи майнер справді навчав
де живе	C++, цей крок	Python, Крок 10, за інтерфейсом `verifier.h`
ціна	мікросекунди	перерахунок $q \cdot k$ кроків навчання
детермінізм	повний	з порогом і похибкою
що ловить	биті, суперечливі, чужі блоки	ліниве повторення, часткову роботу, підробку траєкторії

ЧОМУ САМЕ ТАК

Вартість. Структурно неправильний блок ліпиться безкоштовно. Якщо вузол на кожен такий блок запускатиме Python і перераховуватиме кроки навчання, один зловмисник із циклом `while true` зупинить мережу. Дешеві перевірки завжди йдуть першими.

Відтворюваність. Структурна перевірка — чиста функція від двох блоків: її можна прогнати мільйон разів, у тестах, під час завантаження з диска, на іншій машині. Семантична залежить від заліза, порогу й приватного вибору переходів (P5) і чистою не є принципово.

Незалежність від Python. Тести ланцюга мають працювати без torch, інакше кожен `ctest` тягтиме півтори секунди імпортів і кілька гігабайтів залежностей.

Нові слова

слово	що означає
`структурна валідація`	Перевірка, що блок складений за правилами і узгоджений із батьком. Дешева й детермінована.
`семантична валідація`	Перевірка, що заявлена робота справді виконана. Дорога, з порогом, живе в Python.
`чиста функція`	Результат залежить лише від аргументів: не читає годинник, не читає файли, не має стану.

слово	що означає
`реорганізація (reorg)`	Перехід вузла на іншу, довшу гілку: частина блоків перестає бути головним ланцюгом.
`append-only log`	Файл, у який лише дописують у кінець. Старі записи не переписуються, тому пошкодження локальне.
`std::unordered_map`	Хеш-таблиця. Потрібна для відповіді «чи бачив я вже цей блок» за сталий час.
`абстрактний клас`	Клас із чисто віртуальними методами (`= 0`): не створюється сам, задає інтерфейс нащадкам.
`std::ios::binary`	Режим потоку, у якому байти не перетворюються. Для бінарного формату обов'язковий.

Контракт

node/include/chain.h

```
#pragma once
#include <cstdint>
#include <string>
#include <unordered_map>
#include <vector>
#include "block.h"

namespace pouw {

struct Block {
    BlockHeader header;
    std::vector<Hash> ckpt_hashes;   // C_0 .. C_m, рівно K/k + 1 штук
};

// Хешування Hash для unordered_map: значення вже випадкове, вигадувати нічого не треба.
struct HashHasher {
    std::size_t operator()(const Hash& h) const noexcept;
};

inline constexpr std::int64_t MAX_FUTURE_SEC = 600;    // наскільки вперед приймаємо блок
inline constexpr std::int64_t MAX_BLOCK_SEC = 86400;   // скільки максимум «триває» блок

// Структурна перевірка блоку відносно батька. Порожній вектор = блок валідний.
// Чиста функція: не читає годинник, не читає диск, не викликає Python.
std::vector<std::string> validate_structure(const Block& b, const Block& parent);

// Окремі правила для генезису: у нього немає батька і лише одна контрольна точка.
std::vector<std::string> validate_genesis(const Block& b);

// Детермінований генезис: усі вузли будують його з тих самих параметрів задачі
// і зобов'язані отримати однаковий block_hash.
Block make_genesis(const Hash& task_id, const Hash& w0, std::uint32_t K, std::uint32_t k);

class Chain {
public:
    // true = після виклику блок є в ланцюгу (у тому числі якщо вже був).
    // false = відхилено; причина — у *why, якщо why != nullptr.
    bool add(const Block& b, std::string* why = nullptr);

    const Block& tip() const;            // кидає std::runtime_error на порожньому ланцюгу
    std::uint64_t height() const;        // висота вершини; 0 для самого лише генезису
```

```
    bool empty() const;
    bool has(const Hash& block_hash) const;

    // Гілка = блоки висот 1..H за зростанням (генезис у всіх спільний).
    // Замінює поточний ланцюг, лише якщо гілка СТРОГО довша і вся валідна.
    bool try_replace(const std::vector<Block>& branch, std::string* why = nullptr);

    const Block& at(std::uint64_t height) const;
    std::vector<Block> blocks_from(std::uint64_t from_height) const;

    bool save(const std::string& path) const;
    bool load(const std::string& path);

private:
    // check_clock = false під час відтворення з диска: учорашній блок не «з
    //     майбутнього».
    bool append_checked(const Block& b, bool check_clock, std::string* why);

    std::vector<Block> blocks_;
    std::unordered_map<Hash, std::uint64_t, HashHasher> index_;
};

}  // namespace pouw
```

Правила структурної валідації

Кожне правило закриває конкретну помилку або атаку. Якщо не можеш назвати, яку саме, — правило зайве.

правило	що ловить
`version == 1`	блок від несумісної версії протоколу
`height == parent.height + 1`	дірки в ланцюгу і спроби вставити блок збоку
`prev_hash == block_hash(parent.header)`	переписування історії: підміну батька
`task_id == parent.task_id`	блок із чужої задачі навчання в цьому ланцюгу
`K == parent.K` і `k == parent.k`	P3: майнер оголошує собі менше кроків, ніж вимагає задача
`k > 0`	ділення на нуль у наступному правилі: падіння вузла з одного повідомлення
`K % k == 0`	розбивку, за якої остання точка не збігається з кінцем роботи
`ckpt_hashes.size() == K/k + 1`	пропуск контрольних точок: механізму B нема що перевіряти
`ckpt_hashes.front() == w_start`	розрив між заявленим початком і першою точкою
`ckpt_hashes.back() == w_end`	підміну кінцевого стану вже після побудови траєкторії
`w_start == parent.w_end`	P1: майнер обрав собі зручний стартовий стан
`ckpt_root == ckpt_root_of(ckpt_hashes)`	підміну списку точок після хешування заголовка
`timestamp >= parent.timestamp`	час, що йде назад: ламає будь-які часові вимірювання

правило	що ловить
`timestamp <= parent.timestamp + MAX_BLOCK_SEC`	сміттєві мітки часу на кшталт 10^{18}
`metric_micro` у `[0, 1'000'000]`	заявлену точність 300 %, яка ламає порівняння метрик

УВАГА

Порядок двох правил критичний: `k > 0` перевіряється **до** `K % k == 0`. Інакше зловмисний вузол надсилає блок із `k = 0`, і кожен, хто його перевірить, падає з `SIGFPE`. Це не причіпка: «поділи і подивись» на керованих даних — класична вразливість. Запиши її в модель загроз як приклад того, що навіть дешева перевірка має бути написана в правильному порядку.

ЧОМУ САМЕ ТАК

Чому `validate_structure` не читає годинник, хоча правило про час у списку є?

Порівняння з батьком детерміноване: воно залежить лише від двох блоків і дає той самий результат завжди. Порівняння з поточним часом залежить від того, **коли** його виконують. Якби друга частина сиділа у `validate_structure`, то `load()` після перезапуску почав би відкидати власні вчорашні блоки. Тому годинникова перевірка живе в `Chain::add`, а `load` її не виконує: це різниця між «блок правильний» і «блок правильний **зараз**».

Шов для семантичної перевірки

node/include/verifier.h

```
#pragma once
#include <string>
#include "chain.h"

namespace pouw {

enum class Verdict {
    Accept,   // перевірка пройдена
    Reject,   // перевірка не пройдена: блок відхиляємо
    Error,    // перевірка не відбулась (воркер упав, немає ваг у сховищі)
};

struct VerifyResult {
    Verdict verdict = Verdict::Error;
    std::string reason;
    double distance = 0.0;    // відстань між перерахунком і заявленим (механізм B)
    double threshold = 0.0;   // поріг, з яким порівнювали
    double seconds = 0.0;     // скільки коштувала перевірка
};

// Семантична перевірка: чи справді виконано K кроків навчання.
// Реалізацію (запуск worker.py) пише Крок 10; тут лише інтерфейс.
class Verifier {
public:
    virtual ~Verifier() = default;
    virtual VerifyResult verify(const Block& b, const Block& parent) = 0;
};

// Заглушка на Кроки 8–9: приймає все, нічого не рахує.
class AlwaysAccept : public Verifier {
```

```
public:
    VerifyResult verify(const Block& b, const Block& parent) override;
};

}  // namespace pouw
```

УВАГА

`Verdict::Error` — це не `Reject`. Якщо Python упав або ваг немає у сховищі, вузол не має права оголосити блок недійсним: він просто **не зміг** перевірити. Інакше досить зламати чужий воркер, щоб чесні блоки почали відхилятись. На `Error` блок відкладається і перевіряється пізніше, а подія йде в лог. Ця відмінність уже закладена в контракт Кроку 1 полями `ok` і `verdict`.

ЧОМУ САМЕ ТАК

Чому `Chain` не викликає `Verifier` сам? Бо тоді `load()` запускав би навчання для кожного блоку — сотні секунд на старті; тести ланцюга залежали б від torch; кожен дубль блоку з мережі коштував би перерахунок.

Порядок роботи вузла натомість такий: структурна перевірка (безкоштовно) → семантична через `Verifier` (дорого, один раз на блок) → `chain.add`. Хто і в якому порядку це робить, вирішує головний цикл вузла — і саме тому туди можна буде вставити чергу, таймаут або fraud proof, не чіпаючи ланцюг.

Скелет

node/src/chain.cpp · тіла написати самому

```
#include "chain.h"
#include <chrono>
#include <fstream>
#include <stdexcept>
#include "sha256.h"

namespace pouw {

std::size_t HashHasher::operator()(const Hash& h) const noexcept {
    // 1. Значення вже криптографічно випадкове: склади std::size_t із перших 8 байтів
    //    (зсуви, як у Кроці 7). Ніяких власних «хеш-функцій».
}

std::vector<std::string> validate_structure(const Block& b, const Block& parent) {
    // 1. Заведи vector<string> why.
    // 2. Правила з таблиці по одному: кожне — окремий if, кожне додає свій рядок.
    //    НЕ виходь після першої помилки: у логах хочеться бачити всі причини одразу.
    // 3. Порядок: version, height, prev_hash, task_id; потім K і k (спершу k > 0!);
    //    потім розмір і межі списку точок; потім w_start / w_end / ckpt_root;
    //     останніми час
    //    і метрика.
    // 4. Формулюй причини так, щоб їх шукав grep: "prev_hash mismatch",
    //    "ckpt count 5 != 21", "w_start != parent.w_end".
}

std::vector<std::string> validate_genesis(const Block& b) {
    // 1. height == 0, prev_hash == zero_hash(), w_start == w_end.
```

```
    // 2. Рівно одна контрольна точка, і вона дорівнює w_start.
    // 3. ckpt_root == ckpt_root_of(b.ckpt_hashes).
    // 4. k > 0 і K % k == 0: параметри задачі мають бути осмислені вже тут — з них по
    //     індукції
    //    живуть усі наступні блоки.
}

Block make_genesis(const Hash& task_id, const Hash& w0, std::uint32_t K, std::uint32_t k) {
    // 1. version = 1, height = 0, prev_hash = zero_hash(), round = 0, miner_id =
    //     zero_hash(),
    //    timestamp = 0, w_start = w_end = w0, metric_micro = 0.
    // 2. ckpt_hashes = { w0 }, ckpt_root = ckpt_root_of(ckpt_hashes).
    // 3. Жодного поточного часу: генезис має бути побайтово однаковий у всіх вузлів,
    //    інакше мережа розпадеться на три окремі ланцюги.
}

bool Chain::append_checked(const Block& b, bool check_clock, std::string* why) {
    // 1. Порахуй h = block_hash(b.header) один раз і далі користуйся ним.
    // 2. has(h) -> нічого не роби, поверни true: дубль не є помилкою.
    // 3. Ланцюг порожній -> validate_genesis; інакше спершу перевір, що блок продовжує
    //    вершину (height == height() + 1), причина "not extending tip", і аж тоді
    //    validate_structure(b, tip()).
    // 4. check_clock -> timestamp не більше ніж на MAX_FUTURE_SEC попереду
    //    std::chrono::system_clock::now().
    // 5. Склей причини через "; " у *why і поверни false.
    // 6. Інакше blocks_.push_back(b), index_[h] = висота, поверни true.
}

bool Chain::add(const Block& b, std::string* why) {
    // 1. Один рядок: append_checked(b, true, why).
}

bool Chain::try_replace(const std::vector<Block>& branch, std::string* why) {
    // 1. Порожній ланцюг -> відмов: гілка добудовується до наявного генезису.
    // 2. branch.size() <= height() -> відмов, причина "not longer": рівна довжина НЕ
    //     витісняє
    //    поточну гілку (правило «перший побачений перемагає»).
    // 3. Побудуй тимчасовий Chain: поклади в нього blocks_[0], далі по черзі всі
    //     блоки гілки
    //    через append_checked(..., false, ...).
    // 4. Хоч один не пройшов -> нічого не міняй, поверни false із причиною і номером
    //     висоти.
    // 5. Усі пройшли -> забери в тимчасового blocks_ та index_ (std::move), поверни
    //     true.
}

bool Chain::save(const std::string& path) const {
    // 1. Відкрий у std::ios::binary | std::ios::trunc.
    // 2. Запиши 8 байтів магії "POUWCHN1".
    // 3. На кожен блок: payload = serialize(header) + u32(кількість точок) + точки по
    //     32 байти;
    //    спершу u32 довжини payload, потім сам payload.
    // 4. true лише якщо потік лишився добрим ПІСЛЯ закриття: помилка запису часто
    //    виявляється аж на flush.
}

bool Chain::load(const std::string& path) {
    // 1. Файлу немає — не помилка: поверни false мовчки, вузол почне з генезису.
    // 2. Звір магію. Не збіглась -> false: це чужий файл, краще впасти одразу.
    // 3. Очисти blocks_ і index_.
    // 4. Цикл: u32 довжини; кінець файлу рівно на межі запису — нормальне завершення;
    //    менше байтів, ніж обіцяно, — обірваний запис, повідом і зупинись.
    // 5. Розбери payload: 236 байтів заголовка (зворотні до put_* читалки), u32
    //     кількості,
```

```
    //    точки. Звір: довжина запису має дорівнювати 240 + 32 * кількість.
    // 6. Кожен блок додавай через append_checked(b, false, &why), а не push_back:
    //    файл на диску не є довіреним джерелом.
    // 7. Перший блок, що не пройшов, зупиняє відтворення: у ланцюгу лишається валідний
    //    префікс, функція повертає false і пише номер запису й причину.
}

}  // namespace pouw
```

формат store/chain.bin

```
8 байтів     магія "POUWCHN1"
далі по одному запису на блок:
    u32      довжина payload (little-endian)
    payload: 236 байтів канонічного заголовка
             u32          кількість контрольних точок
             32 * n       самі точки, підряд
інваріант:   довжина payload == 240 + 32 * n
```

ЧОМУ САМЕ ТАК

Чому бінарний append-only формат, а не JSON Lines? За JSONL є вагомий аргумент: файл читається очима. Але ціна висока. З'явився б **другий** шлях серіалізації заголовка, який рано чи пізно розійшовся б із канонічним, а розходження виявилось би як «ланцюг після перезапуску не збігається». До того ж на цьому кроці JSON-бібліотеки в проєкті ще немає: вона приходить у Кроці 9 і потрібна там для мережі.

Бінарний формат перевикористовує вже перевірений золотим вектором `serialize`, а довжина запису робить його самоописним: пошкодження виявляється на тому ж записі, а не через десять. Читабельність повертається однією підкомандою `node dump-chain`. Для протоколу з тисячами блоків на секунду вибір був би іншим, але в нас блок — це секунди навчання.

ПІДКАЗКИ

Рівень 1. Почни з `validate_structure`: це чиста функція, її можна писати й тестувати без будь-чого іншого. `save` / `load` роби останніми — вони найнудніші й найбагатші на дрібні помилки.

Рівень 2. Для читання з бінарного потоку заведи зворотні до `put_*` функції: `std::uint32_t take_u32(const std::uint8_t*& p)`. Вони теж працюють зсувами, але кожен байт приводь до `std::uint32_t` **до** зсуву, інакше `p[3] << 24` переповнить `int`.

Рівень 3. Каркас однієї перевірки, щоб задати стиль решті:

```
if (b.header.prev_hash != block_hash(parent.header)) {
    why.push_back("prev_hash mismatch: expected " +
                  to_hex(block_hash(parent.header)) + ", got " +
                  to_hex(b.header.prev_hash));
}
```

Текст причини — це те, що ти читатимеш о другій ночі. Не економ на ньому.

Тести

node/tests/test_chain.cpp — що має бути перевірено

```
// Набери сам за зразком test_block.cpp (той самий CHECK). Мінімальний перелік:
//  1. Генезис від make_genesis приймається порожнім ланцюгом; height() == 0.
//  2. Два виклики make_genesis з однаковими аргументами дають однаковий block_hash.
//  3. Коректний блок-нащадок приймається; height() == 1; tip() — це він.
//  4. Той самий блок удруге повертає true і НЕ збільшує height().
//  5. На кожне правило з таблиці — свій зіпсований блок, і перевірка, що
//     validate_structure повернула рівно одну причину, і саме ту.
//  6. Блок із k = 0 не роняє процес (тест на порядок перевірок).
//  7. save() у тимчасовий файл, новий Chain, load(), збіг tip() і height().
//  8. Гілка довжиною 2 витісняє ланцюг довжиною 1; гілка рівної довжини — ні.
```

Пункт 5 — найважливіший. Заведи допоміжну функцію, яка робить валідний блок-нащадок, і далі псуй у копії по одному полю. Тоді тест читається як сама таблиця правил, і додати правило пізніше буде дешево.

Твоє завдання

ТВОЄ ЗАВДАННЯ

1. Набери `chain.h` і `verifier.h`.
2. Напиши `chain.cpp` за скелетом у порядку: `validate_structure`, `validate_genesis`, `make_genesis`, `append_checked`, `add`, `try_replace`, `save`, `load`.
3. Напиши `verifier.cpp` із заглушкою `AlwaysAccept` — два рядки.
4. Додай `chain.cpp` і `verifier.cpp` до `pouw_core`, а `test_chain` — другою тестовою ціллю.
5. Напиши `test_chain.cpp`: не менше ніж по одному тесту на правило.
6. Тимчасовий режим `node demo-chain` у `main.cpp`: генезис, три блоки-заглушки, `save`, `load` у новий об'єкт, друк висоти й хеша вершини.
7. Коміт: `feat(node): chain with structural validation and persistence`.

Перевірка

ПЕРЕДБАЧ ДО ЗАПУСКУ

1. Блок правильний у всьому, крім `ckpt_root` (там лишився корінь попереднього блоку). Скільки причин поверне `validate_structure` — одну чи більше?
2. Ти зіпсував один байт у заголовку третього блоку у файлі. На якому записі зупиниться `load` і з якою причиною? (Уважно: помилка спливе не обов'язково на зіпсованому записі.)
3. Два вузли додали різні блоки на висоті 5, потім один із них — блок на висоті 6. Що станеться з ланцюгом другого вузла, коли він побачить обидва?

ЕТАЛОН ДЛЯ САМОПЕРЕВІРКИ

Числа тут не потрібні — потрібні властивості:

1. `save` → `load` у новий об'єкт дає **той самий** `to_hex(block_hash(tip().header))` і ту саму висоту. Це найкоротший доказ симетричності бінарного формату.
2. Кожне правило має тест, який падає, якщо правило закоментувати. Правило, яке не можна зламати тестом, у коді не потрібне.
3. Блок із `k = 0` дає рядок причини, а не `SIGFPE`.
4. Рівна за довжиною гілка не витісняє поточну; довша на один блок — витісняє, і після цього вершини обох вузлів збігаються.
5. `ctest --test-dir build --output-on-failure` — усі цілі зелені.

САМОПЕРЕВІРКА

1. Назви три правила, які ламають атаку «лінивого повторення» ще до запуску Python. Чому їх усе одно недостатньо?
2. Чому `add` повертає `true` на повторно доданий блок? Що зламалося б у мережевому циклі Кроку 9 при іншому рішенні?
3. Чому `try_replace` перевіряє гілку повністю з нуля, а не лише нові блоки? Де цей підхід перестане працювати?
4. Ланцюг приймає будь-який структурно правильний блок за мікросекунди. Скільки коштує зловмиснику зробити такий блок, і що саме — у якому кроці — робить блок дорогим?

Найдовший ланцюг

ЧОМУ САМЕ ТАК

Правило вибору гілки: перемагає найдовша, за рівної довжини — побачена першою.

Чому довжина чесно вимірює роботу? Бо кожен блок — це рівно K кроків навчання, а K фіксоване задачею і перевіряється правилом `K == parent.K` уздовж усього ланцюга аж до генезису. Отже, ланцюг висоти H відповідає $H \cdot K$ крокам. Це прямий аналог найбільшої накопиченої складності в Bitcoin, тільки одиниця роботи — крок SGD, а не хешування.

Де правило зламалося б: **якби `K` міг відрізнятись від блоку до блоку** — тоді вигідно робити багато дешевих блоків (тому `K` — параметр задачі, а не вибір майнера, P3); **якби структурно валідний блок був дешевим** — а він саме такий: на Кроках 8–9 ланцюг не захищений узагалі, будь-хто складе тисячу коректних заголовків за секунду. Дорогим блок робить лише семантична перевірка з Кроку 10, і це треба чесно записати в роботу: консенсус тримається не на структурі блоку, а на тому, що кожен блок перевіряється перерахунком.

Зламай навмисне

ЗЛАМАЙ НАВМИСНЕ

Експеримент 1. Стартовий стан із минулого. Збери вручну блок на висоті 3, у якого `prev_hash` правильний (від блоку 2), але `w_start` узятий із блоку 1. Яке правило спрацювало? Тепер прибери це правило й скажи вголос, що зміг би зробити майнер, якби вузли не звіряли `w_start` з `w_end` батька. Це рівно та атака, яку закриває P1.

Експеримент 2. Рівна гілка. Побудуй дві гілки однакової довжини від спільного батька й додай їх в обох порядках: перемагає перша побачена, результат залежить від порядку. Потім додай один блок до другої гілки й подивись на реорганізацію. Скільки блоків «випало» з головного ланцюга і що з ними далі має робити чесний вузол?

Експеримент 3. Один битий байт. Збережи ланцюг із чотирьох блоків і зіпсуй рівно один байт: (а) у полі довжини запису; (б) усередині заголовка третього блоку; (в) в одному з хешів контрольних точок. Випиши три різні повідомлення від `load` і поясни, яке правило спіймало кожен випадок. Варіант (б) особливо цікавий: помилка виявляється не на зіпсованому блоці.

Експеримент 4. Німа перевірка. Закоментуй правило `ckpt_root == ckpt_root_of(...)`. Які тести впали? Якщо жоден — твій набір тестів слабший за таблицю правил, і це треба виправити зараз, а не в лютому.

У ТЕКСТ РОБОТИ

Найбагатший на текст крок усього C++-блоку. У розділ 3 іде підрозділ «Ланцюг і правила прийняття блоку» з таблицею правил **разом із колонкою «що ловить»**: у такому вигляді вона читається як зв'язок між розділом 2 (модель загроз) і реалізацією. Окремим абзацом — поділ на структурну й семантичну перевірку з аргументом про вартість: це те, що відрізняє протокол корисної роботи від звичайного PoW, і на захисті про це спитають.

Крок 9. Мережа: три вузли по TCP

Мета кроку: три процеси на одній машині обмінюються блоками, сходяться до однакової вершини, і вузол, запущений останнім, наздоганяє інших.

Ідея

Мережева частина не додає протоколу жодного правила. Усі правила вже написані в Кроці 8; мережа лише переносить байти і вирішує, кого про що спитати. Якщо здається, що тут потрібна ще одна перевірка блоку, — вона забута в Кроці 8.

Три речі, у яких уся складність:

1. **Межі повідомлень.** TCP — це потік байтів, а не потік повідомлень. Те, що надіслано одним `send`, приходить будь-якими шматками. Межі треба нести самому.
2. **Спільний стан.** Ланцюг один, а потоків кілька. Один м'ютекс — і жодного разу не «майже правильно».
3. **Наздоганяння.** Вузол, який підключився пізно або лежав, має отримати пропущене.

Нові слова

слово	що означає
`сокет`	Дескриптор з'єднання. Для ядра — просто число, як файловий дескриптор.
`bind / listen / accept`	Прив'язати сокет до порту, оголосити слухачем, узяти наступне вхідне з'єднання.
`SO_REUSEADDR`	Опція, яка дозволяє прив'язатись до порту із залишками TIME_WAIT від попереднього запуску.
`TIME_WAIT`	Стан, у якому ядро тримає щойно закрите з'єднання (у Linux 60 с), щоб спізнілі пакети не потрапили в нове.
`часткове читання`	`recv` має право повернути менше байтів, ніж просили. Це норма, а не помилка.
`префікс довжини`	Чотири байти перед тілом, які кажуть, скільки байтів збирати.
`SIGPIPE`	Сигнал при записі в закрите з того боку з'єднання. За замовчуванням убиває процес без повідомлення.
`EINTR`	`recv` перервано сигналом. Не помилка з'єднання: треба повторити виклик.
`std::lock_guard`	Замок, що знімається сам при виході з області видимості. Ручний `unlock` — джерело зависань.

Контракт

одне повідомлення

```
u32   довжина тіла, little-endian   (той самий порядок байтів, що й у блоці)
...   тіло: JSON у UTF-8, без завершального нуля
```

ЧОМУ САМЕ ТАК

Чому префікс довжини, а не «читати до `}`» чи «одне повідомлення на `recv`»? Бо TCP не зберігає межі: два `send` можуть приїхати одним `recv`, один `send` — трьома. Це не рідкісний випадок і не залежить від розміру: воно просто **іноді** так відбувається — звідси класичне «працює в мене на столі, ламається під навантаженням». Пошук `}` не рятує: дужки є всередині повідомлення, і рахувати їх — це вже писати парсер поверх потоку.

Чому префікс little-endian, а не «мережевий порядок» big-endian, як велить традиція? Бо в протоколі вже є одне правило порядку байтів — із Кроку 7. Два правила в одній системі — це гарантована плутанина на стику. Традиція тут програє одноманітності; головне — записати рішення в контракт.

УВАГА

Одразу став стелю: префікс більший за 1 МіБ — розривай з'єднання, не виділяючи пам'ять. Інакше чотири випадкові байти (або зловмисник) змусять вузол спробувати виділити чотири гігабайти. Це найдешевша атака на будь-який протокол із префіксом довжини, і закривається вона одним `if`.

тип	коли надсилається	реакція одержувача
`hello`	одразу після з'єднання, з обох боків	запам'ятати ім'я; його висота більша — надіслати `getblocks`, менша — надіслати `inv` своєї вершини
`inv`	після прийняття власного або чужого блоку — всім сусідам	блок невідомий і висота більша — `getblock`; відстаємо більш ніж на один — `getblocks`
`getblock`	у відповідь на `inv` про невідомий блок	надіслати `block`, якщо він є
`getblocks`	коли ми відстаємо	надіслати всі `block` від `from_height` до вершини, за зростанням
`block`	у відповідь на `getblock` або `getblocks`	покласти в чергу вхідних; головний цикл перевірить і додасть

схеми повідомлень

```
{"type": "hello", "name": "alice", "height": 12}
{"type": "inv", "block_hash": "<64 hex>", "height": 13}
{"type": "getblock", "hash": "<64 hex>"}
{"type": "getblocks", "from_height": 5}
{"type": "block",
 "header": {"version": 1, "height": 13, "prev_hash": "<64 hex>", "task_id": "<64 hex>",
            "round": 13, "miner_id": "<64 hex>", "timestamp": 1792000000,
            "w_start": "<64 hex>", "w_end": "<64 hex>", "ckpt_root": "<64 hex>",
```

```
           "K": 500, "k": 25, "metric_micro": 912300},
 "ckpt_hashes": ["<64 hex>", "<64 hex>", "..."]}
```

УВАГА

У повідомленні `block` немає поля `block_hash`, і це навмисно. Хеш завжди перераховується з канонічних 236 байтів після розбору JSON. Поле в повідомленні було б підказкою, якій не можна довіряти: зловмисник надішле правильний блок із чужим хешем, і вузол, що повірив полю, занесе його в індекс під неправильним ключем. Загальне правило: **усе, що можна порахувати, рахується, а не читається з мережі.** JSON тут — лише транспорт: він ніколи не хешується (Крок 1) і ніколи не є джерелом істини про блок.

потік	скільки	що робить	чого не робить
головний	1	майнить заглушку, розгрібає чергу вхідних блоків, додає в ланцюг	не читає сокети
`accept`	1	приймає вхідні з'єднання, заводить потік на кожне	не торкається ланцюга
читач сусіда	по одному на з'єднання	читає повідомлення, відповідає на запити, кладе блоки в чергу	не додає блоки в ланцюг сам

node/include/net.h

```cpp
#pragma once
#include <atomic>
#include <cstdint>
#include <memory>
#include <mutex>
#include <queue>
#include <string>
#include <thread>
#include <vector>
#include "chain.h"

namespace pouw {

bool send_all(int fd, const std::uint8_t* buf, std::size_t n);
bool recv_all(int fd, std::uint8_t* buf, std::size_t n);
// Одне повідомлення: 4 байти довжини (little-endian) + тіло.
bool send_msg(int fd, const std::string& json);
bool recv_msg(int fd, std::string& out);

inline constexpr std::uint32_t MAX_MSG_BYTES = 1u << 20;   // 1 МіБ

struct Peer {
    int fd = -1;
    int port = 0;
    std::string name;
    std::mutex send_mx;      // один сокет — один пишучий одночасно
};

class Node {
public:
    Node(std::string name, int port, std::vector<int> peer_ports);
    ~Node();
```

```
    void start();                       // слухач + з'єднання із сусідами
    void stop();
    void pump();                        // один оберт головного циклу
    void broadcast_inv(const Hash& block_hash, std::uint64_t height);

    Chain& chain() { return chain_; }          // тільки під chain_mx_!
    std::mutex& chain_mutex() { return chain_mx_; }

private:
    void serve();                                       // потік accept
    void connect_peer(int port);                        // вихідне з'єднання
    void peer_loop(std::shared_ptr<Peer> p);            // потік на з'єднання
    void handle(const std::string& json, const std::shared_ptr<Peer>& from);
    void send_to(const std::shared_ptr<Peer>& p, const std::string& json);

    std::string name_;
    int port_ = 0;
    std::vector<int> peer_ports_;
    int listen_fd_ = -1;
    std::atomic<bool> running_{false};

    std::mutex chain_mx_;
    Chain chain_;
    std::mutex q_mx_;
    std::queue<Block> incoming_;
    std::mutex peers_mx_;
    std::vector<std::shared_ptr<Peer>> peers_;
    std::vector<std::thread> threads_;
};

}  // namespace pouw
```

ЧОМУ САМЕ ТАК

Чому `Peer` під `std::shared_ptr`, а не просто у векторі? Бо в ньому є `std::mutex`, а м'ютекс не копіюється й не переміщується: вектор при зростанні перемістив би елементи, і потік, який тримає посилання на старий об'єкт, працював би з мертвою пам'яттю.

Чому в кожного `Peer` окремий `send_mx`? Бо в один сокет пишуть двоє: потік-читач (відповіді) і головний цикл (розсилка `inv`). Два `send` без замка перемішають байти всередині повідомлення, і той бік прочитає сміття правильної довжини. Такий баг ловиться днями, бо виникає лише при збігу в часі.

Чому один м'ютекс на весь ланцюг, а не структури без блокувань? Бо ланцюг чіпають кілька разів на секунду, а замок коштує десятки наносекунд: виграшу немає взагалі. Натомість лок-фрі-код помиляється тихо — пошкодженням пам'яті, яке виявляється за годину й не відтворюється. Предмет роботи — верифікація навчання, а не багатопотокові структури даних. Один замок через `std::lock_guard` — рішення, яке можна захистити на будь-якій комісії.

Скелет

node/src/net.cpp — нижній рівень · тіла написати самому

```
#include "net.h"
#include <arpa/inet.h>
#include <netinet/in.h>
#include <sys/socket.h>
#include <unistd.h>
#include <cerrno>
#include <cstring>
#include "json.hpp"      // nlohmann/json, один заголовок
#include "sha256.h"

namespace pouw {

bool send_all(int fd, const std::uint8_t* buf, std::size_t n) {
    // 1. Цикл, доки не надіслано всі n байтів.
    // 2. r = ::send(fd, buf + sent, n - sent, MSG_NOSIGNAL) — прапорець потрібен, щоб
    //     запис
    //    у закрите з'єднання повертав помилку, а не вбивав процес.
    // 3. r < 0 і errno == EINTR -> повторити той самий виклик, це не помилка.
    // 4. r <= 0 інакше -> false.
}

bool recv_all(int fd, std::uint8_t* buf, std::size_t n) {
    // 1. Той самий цикл із ::recv.
    // 2. r == 0 означає, що сусід коректно закрив з'єднання -> false.
    // 3. Саме ця функція виправляє часткові читання. Її відсутність — баг, який
    //     проявляється
    //    лише на великих повідомленнях і лише іноді.
}

bool send_msg(int fd, const std::string& json) {
    // 1. Перевір json.size() <= MAX_MSG_BYTES.
    // 2. Склади 4 байти довжини тими самими зсувами, що й put_u32 у Кроці 7 (так, це
    //    повторення; якщо муляє — винеси обидві в node/include/bytes.h).
    // 3. Надсилай префікс і тіло одним буфером: два send на маленьке повідомлення —
    //     це зайвий
    //    пакет і зайвий шанс на перемішування.
}

bool recv_msg(int fd, std::string& out) {
    // 1. Прочитай рівно 4 байти префікса через recv_all.
    // 2. Збери з них u32 (кожен байт приводь до std::uint32_t ДО зсуву).
    // 3. n == 0 або n > MAX_MSG_BYTES -> false: з'єднання недовірене.
    // 4. out.resize(n) і прочитай рівно n байтів через recv_all.
}

}  // namespace pouw
```

node/src/net.cpp — вузол · тіла написати самому

```
void Node::serve() {
    // 1. listen_fd_ = ::socket(AF_INET, SOCK_STREAM, 0); перевір на -1.
    // 2. Увімкни SO_REUSEADDR через ::setsockopt ДО bind — інакше після перезапуску
    //     протягом
    //    хвилини отримаєш EADDRINUSE через TIME_WAIT.
    // 3. sockaddr_in: sin_family = AF_INET, sin_addr = INADDR_ANY, sin_port =
    //     htons(port_).
    //    Тут htons доречний: це чужий контракт (ядра), а не наш.
    // 4. ::bind, ::listen(listen_fd_, 8). При помилці друкуй std::strerror(errno) —
    //     саме
    //    повідомлення ядра, а не своє «щось пішло не так».
```

```
    // 5. Цикл, доки running_: fd = ::accept(...); на кожне з'єднання заведи Peer у
    //     peers_
    //    (під peers_mx_) і потік peer_loop.
}

void Node::connect_peer(int port) {
    // 1. socket, sockaddr_in з inet_pton("127.0.0.1"), ::connect.
    // 2. Сусід ще не піднявся — нормальний стан, а не помилка: зачекай секунду і
    //     спробуй ще
    //    раз, у циклі, доки running_.
    // 3. Після з'єднання: заведи Peer, надішли hello зі своїм ім'ям і висотою, заведи
    //     потік.
}

void Node::peer_loop(std::shared_ptr<Peer> p) {
    // 1. Цикл: recv_msg; false — вийти, прибрати Peer зі списку.
    // 2. Розбери JSON у try/catch: биті дані від сусіда не мають ронити вузол.
    // 3. Віддай handle(json, p).
}

void Node::handle(const std::string& json, const std::shared_ptr<Peer>& from) {
    // 1. Розбери "type" і розгалужся за таблицею повідомлень.
    // 2. hello:     запам'ятай ім'я; порівняй висоти, надішли getblocks або inv.
    // 3. inv:       під chain_mx_ подивись has(hash) і height(); за потреби
    //     getblock/getblocks.
    // 4. getblock:  знайди блок, збери JSON, надішли.
    // 5. getblocks: під chain_mx_ візьми blocks_from(from_height) — КОПІЮ, — відпусти
    //     замок
    //               і аж потім надсилай. Тримати замок під час запису в сокет не можна:
    //               повільний сусід зупинить увесь вузол.
    // 6. block:     розбери у Block, поклади в incoming_ під q_mx_. Не додавай у
    //     ланцюг тут!
}

void Node::broadcast_inv(const Hash& h, std::uint64_t height) {
    // 1. Під peers_mx_ візьми копію списку сусідів, відпусти замок.
    // 2. Кожному надішли inv через send_to (усередині — lock_guard на p->send_mx).
}

void Node::pump() {
    // 1. Під q_mx_ забери з черги всі накопичені блоки в локальний вектор, відпусти
    //     замок.
    // 2. Для кожного: під chain_mx_ спробуй chain_.add(b, &why).
    // 3. Прийнято — рядок у лог і broadcast_inv. Саме тут, а не при отриманні: блок,
    //     який ми
    //    не прийняли, далі не поширюємо.
    // 4. Причина "not extending tip" і висота більша за нашу — ми відстали: надішли
    //    getblocks(from_height = 1), склади отримані блоки в std::map<uint64_t, Block>;
    //    коли буфер суцільний від 1 до H і H > height(), виклич chain_.try_replace.
    // 5. Будь-яка інша причина — рядок у лог і нічого більше. Ніяких винятків назовні:
    //    поганий блок від сусіда — очікувана подія, а не збій.
}
```

УВАГА

Правило, без якого мережа захлинеться: `inv` розсилається лише для блоку, який ми **щойно прийняли вперше**. Якщо розсилати все, що прийшло, три вузли утворять цикл, у якому один блок ходить по колу вічно. Перевірка `has(hash)` у `Chain` — не оптимізація, а умова збіжності.

ПІДКАЗКИ

Рівень 1. Роби знизу вгору і перевіряй кожен шар окремо. Спершу `send_all` / `recv_all` / `send_msg` / `recv_msg` і крихітна програмка «клієнт шле три рядки, сервер їх друкує» — без блоків і ланцюга. Мережеві баги інакше не локалізуються.

Рівень 2. Перетворення `Block` ↔ JSON зроби двома вільними функціями в `net.cpp`: `nlohmann::json to_json(const Block&)` і `Block block_from_json(const nlohmann::json&)`. У другій кожне поле діставай явно: `j.at("K").get<std::uint32_t>()` кине виняток на відсутньому полі — це саме те, чого ми хочемо, але ловити його треба в `peer_loop`.

Рівень 3. Каркас циклу, який виправляє часткове читання:

```
std::size_t got = 0;
while (got < n) {
    ssize_t r = ::recv(fd, buf + got, n - got, 0);
    if (r < 0 && errno == EINTR) continue;
    if (r <= 0) return false;
    got += static_cast<std::size_t>(r);
}
return true;
```

`send_all` — дзеркальний, з `MSG_NOSIGNAL` у прапорцях.

node/src/main.cpp · тіла написати самому

```
int main(int argc, char** argv) {
    // 1. Розбери аргументи: --name alice --port 7001 --peers 7002,7003
    //    (плюс необов'язкові --chain store/alice.chain і --mine-sec 5).
    //    Немає обов'язкового аргументу — друкуй підказку й повертай 2.
    // 2. Побудуй генезис: task_id = sha256("pouwchain-demo-task"),
    //    w0 = sha256("pouwchain-demo-w0"), K = 500, k = 25. Усі три вузли мають
    //     отримати
    //    ОДНАКОВИЙ генезис — ніякого часу й випадковості тут.
    // 3. --chain указано і файл є -> chain.load(...); інакше chain.add(genesis).
    // 4. signal(SIGINT, ...) на прапорець зупинки, щоб Ctrl+C завершував акуратно.
    // 5. node.start(); далі цикл: node.pump(); кожні mine_sec секунд — заглушка
    //     майнера;
    //    sleep 100 мс; раз на секунду друкуй стан: ім'я, висота, короткий хеш вершини,
    //    кількість сусідів.
    // 6. На виході: node.stop(), chain.save(...).
}
```

заглушка майнера на цей крок

```
Справжнього навчання ще немає (воно приходить у Кроці 10), але блоки мають з'являтись,
інакше нема чого розсилати. Заглушка будує структурно валідний блок так:

    w_start      = tip().header.w_end
    ckpt[0]      = w_start
    ckpt[i]      = sha256(ckpt[i-1] ‖ miner_id ‖ u64(i))      для i = 1 .. K/k
    w_end        = ckpt[K/k]
    ckpt_root    = ckpt_root_of(ckpt)
    metric_micro = метрика батька + 100        (аби бачити зростання в логах)
    timestamp    = поточний час

Інтервал між спробами — mine_sec плюс випадкова затримка 0..2 с на вузол, щоб вузли не
подавали блоки синхронно і щоб виникали справжні перегони за той самий батьківський блок.
```

УВАГА

Заглушка — тимчасовий код, і має бути позначена як тимчасова: окрема функція `stub_mine`, коментар із номером кроку, у якому вона зникає. У Кроці 10 на її місце стане виклик воркера. Найгірше, що може статись із таким кодом, — він переживе диплом і потрапить у розділ 3 як «реалізація майнінгу».

CMakeLists.txt — дельта Кроку 9

```
find_package(Threads REQUIRED)
# до pouw_core додається node/src/net.cpp
target_link_libraries(pouw_core PUBLIC OpenSSL::Crypto Threads::Threads)
```

`json.hpp` (nlohmann/json, ліцензія MIT, один заголовок) поклади в `node/include/` і закоміть разом із проєктом: одна залежність, яка не потребує пакетного менеджера, і збірка лишається відтворюваною через рік. Бібліотеку треба назвати в переліку використаних засобів у записці.

Твоє завдання

ТВОЄ ЗАВДАННЯ

1. Набери `net.h`, постав `json.hpp` у `node/include/`.
2. Напиши нижній рівень і перевір його окремою міні-програмою до того, як братись за вузол.
3. Напиши перетворення `Block` ↔ JSON і тест «блок → JSON → блок дає той самий `block_hash`». Він не потребує мережі й ловить половину майбутніх помилок.
4. Напиши `Node`: `serve`, `connect_peer`, `peer_loop`, `handle`, `broadcast_inv`, `pump`.
5. Перепиши `main.cpp` під CLI і додай заглушку майнера.
6. Запусти три вузли в трьох терміналах і доведи властивості з еталона.
7. Коміт: `feat(node): tcp p2p with length-prefixed json messages`.

```
./build/node --name alice --port 7001 --peers 7002,7003 --chain store/alice.chain
./build/node --name bob   --port 7002 --peers 7001,7003 --chain store/bob.chain
./build/node --name carol --port 7003 --peers 7001,7002 --chain store/carol.chain
```

Перевірка

ПЕРЕДБАЧ ДО ЗАПУСКУ

1. Ти запускаєш alice, чекаєш 30 секунд, потім bob. Скільки повідомлень якого типу побачить bob у перші дві секунди? Випиши послідовність.
2. Два вузли одночасно знайшли блок на висоті 7. Через скільки блоків вони гарантовано зійдуться до спільної вершини і від чого це залежить?
3. Ти вбиваєш carol через Ctrl+C. Що станеться в потоках alice, які читали з неї, і яке саме значення поверне `recv`?

ЕТАЛОН ДЛЯ САМОПЕРЕВІРКИ

Це властивості, а не числа: вони мають виконуватись на будь-якому запуску.

1. **Збіжність.** Три вузли, запущені в будь-якому порядку, через десяток секунд друкують однакову висоту й однаковий короткий хеш вершини. Хеші розійшлись і не сходяться — зламано `try_replace` або правило «перший побачений».
2. **Наздоганяння.** Вузол, запущений із затримкою в хвилину, доходить до спільної вершини без ручного втручання, і в його логах видно один `getblocks` і серію `block`.
3. **Стійкість.** Убивство одного вузла не зупиняє двох інших: висота продовжує зростати. Піднятий заново вузол наздоганяє й працює далі.
4. **Відсутність шторму.** Кількість `inv` за хвилину пропорційна кількості прийнятих блоків, а не зростає лавиною. Вивід «поїхав» — забуто правило «розсилаємо лише щойно прийняте».
5. **Чистий вихід.** Ctrl+C зберігає ланцюг; наступний запуск піднімає ту саму висоту з файлу.

САМОПЕРЕВІРКА

1. Поясни своїми словами, чому `recv` має право повернути менше, ніж просили, і чому це не помилка.
2. Чому блоки не додаються в ланцюг прямо в потоці-читачі? Назви дві причини, і одна з них має бути про Крок 10.
3. Що станеться, якщо два потоки одночасно викличуть `send` на одному сокеті, і чому цей баг важко відтворити?
4. У нас блокуючі сокети й потік на з'єднання. За якої кількості вузлів це перестане годитись і що роблять натомість?
5. Мережа зараз довіряє сусідам повністю. Перелічи, що зловмисний сусід може зробити з вузлом, і які з цих можливостей уже закриті правилами Кроку 8.

Зламай навмисне

ЗЛАМАЙ НАВМИСНЕ

Експеримент 1. Без префікса. Тимчасово надсилай голий JSON без чотирьох байтів довжини, а на прийомі роби один `recv` у буфер на 4096 байтів і друкуй прочитане. Запусти розсилку кількох `inv` поспіль. Знайди у виводі два повідомлення, що склеїлись в одному буфері, і одне, розрізане навпіл. Тепер спробуй полагодити це «розбиттям по `}`» і поясни, на якому повідомленні такий парсер зламається.

Експеримент 2. Одне читання замість циклу. Поверни префікс, але заміни в `recv_all` цикл одним викликом `recv`. Надішли блок із `K = 1000`, `k = 10` (це 101 хеш, кілька кілобайтів JSON). Що з'явилось у логах? Чому помилка **іноді** не відтворюється, якщо вузли на одній машині?

Експеримент 3. Зайнятий порт. Запусти два вузли з однаковим `--port`. Випиши точний текст `std::strerror(errno)` після `bind`. Далі перезапусти вузол одразу після Ctrl+C з `SO_REUSEADDR` і без нього — різниця видна лише в другому випадку. І окремо перевір твердження: чи дозволяє `SO_REUSEADDR` двом **живим** процесам слухати один порт? (Відповідь не «так»; поясни, яка опція це робить і чому вона нам не потрібна.)

Експеримент 4. SIGPIPE. Прибери `MSG_NOSIGNAL` із `send_all` і вбий сусіда під час розсилки. Вузол зникає без жодного повідомлення — навіть без рядка в лозі. Поясни, чому поведінка «за замовчуванням убити процес» узагалі існує, і поверни прапорець назад.

У ТЕКСТ РОБОТИ

Крок дає підрозділ «Мережева підсистема» розділу 3. Таблиця повідомлень іде рисунком (тип, коли надсилається, реакція), таблиця потоків — другим. Окремо опиши формат дроту з обґрунтуванням префікса довжини: коротко, конкретно і показує, що ти розумієш різницю між потоком байтів і потоком повідомлень. Сюди ж — чесне речення про модель загроз: у прототипі сусіди довірені, підписів немає (P11), і захист від зловмисного **сусіда** зводиться до структурної валідації та обмеження розміру повідомлення.

КОНТРОЛЬНА ТОЧКА · 18 ЖОВТНЯ 2026

Кроки 7–9 вважаються закритими, якщо одночасно виконано:

1. `ctest` зелений, і серед перевірок є звірка `block_hash` із золотим вектором Python — дві мови доведено згодні;
2. `Chain` має тест на **кожне** правило структурної валідації, і кожен тест падає, якщо правило прибрати;
3. ланцюг переживає перезапуск: `save` → `load` дає ту саму вершину, а битий байт у файлі дає зрозуміле повідомлення, а не мовчазне прийняття;
4. три процеси `node` на портах 7001–7003 сходяться до однакової вершини, вузол, запущений останнім, наздоганяє, а вбивство одного не зупиняє інших.

Чого на цю дату ще **немає** — і це нормально: навчання, верифікації, механізмів A–D і атак. Блоки зараз порожні всередині: майнер — заглушка, `Verifier` — `AlwaysAccept`. Мережа переносить структурно правильні, але беззмістовні блоки. Саме тому наступний крок — інтеграція: на місце заглушки стає виклик `worker.py`, а `AlwaysAccept` змінюється на перевірку механізмом B.

Якщо станом на 18 жовтня мережі немає, а Кроки 7–8 закриті — це прийнятне відставання: переходь до Кроку 10 з одним вузлом і поверни мережу наприкінці. Якщо ж не закритий Крок 7 (золотий вектор не збігається), зупинись і закрий саме його: з розбіжністю в хешах усе подальше виглядатиме як випадкові, невідтворювані помилки.

Крок 10. Інтеграція: вузол запускає воркер

Мета кроку: три вузли на одній машині навчають спільну модель, розсилають блоки і перевіряють чужі блоки механізмом B (плюс A), а лінивий вузол отримує відмову.

Ідея

C++-вузол знає про блоки, ланцюг і TCP, але не вміє навчати. Воркер уміє навчати й перевіряти, але не знає, що таке раунд і сусід. Крок 10 — це шов між ними, і майже всі баги цього тижня будуть саме на шві. Домовлятися вже не треба: формат `task.json` і `result.json` зафіксовано в Кроці 1. Лишилось три речі:

1. **Клієнт воркера** — клас, який складає `task.json`, запускає процес і читає відповідь. Уся робота з процесами живе в одному файлі.
2. **Цикл майнінгу** — взяти верхівку, навчити, зібрати блок, додати до себе, розіслати.
3. **Шлях перевірки** — спершу дешеві структурні правила в C++, потім дорога семантика через воркер, потім вердикт і рядок у лог.

Семантику сховай за інтерфейсом із `verifier.h`, який ти оголосив у Кроці 8 і лишив без реалізації: коли у фазі 2 з'являться механізми C і D, вони підставляться за тим самим інтерфейсом, а цикл майнінгу й мережевий код не зміняться жодним рядком.

> ЧОМУ САМЕ ТАК
> Чому окремий процес, а не вбудований інтерпретатор Python? Бо вбудований — це GIL, складна збірка і один аварійний сигнал на дві половини: падає торч — падає вузол. Ціна ізоляції — старт Python з торчем на кожен виклик, секунда-три; саме цю цифру ти чесно покажеш у накладних витратах.

Нові слова

слово	що означає
`popen`	Запускає команду в дочірньому процесі через оболонку і дає потік для читання виводу. Падіння дочірнього процесу не валить батька.
`код виходу`	Число, яке процес повертає ОС. 0 — успіх. Дістається через `WEXITSTATUS`.
`run-тека`	Тека на кожен виклик воркера: `task.json`, `result.json`, `stderr.txt`.
`таймаут`	Межа часу на операцію. Після неї процес убивають, а виклик вважають невдалим.
`сирітський блок`	Блок, батька якого ще немає в ланцюгу. Чекає в черзі, а не відкидається.
`std::random_device`	Джерело випадковості ОС. Із нього сідується генератор валідатора (P5).

Контракт

Заголовок клієнта набирай як є.

node/include/worker_client.h

```
#pragma once
#include <cstdint>
#include <string>
#include <vector>

// Гіперпараметри беруться із завдання (P3), а не з аргументів вузла.
struct TrainRequest {
    std::string task_id;          // 64 hex
    std::string prev_block_hash;  // 64 hex
    std::string miner_id;         // 64 hex
    uint64_t    round = 0;
    std::string w_start;          // хеш ваг із верхівки (P1)
    uint32_t    K = 500;
    uint32_t    k = 25;
    uint32_t    batch_size = 64;
    double      lr = 0.01;
    std::string dataset = "mnist";
    std::string attack_kind;      // "" = чесний вузол; "lazy", "partial"
    double      attack_param = 0.0;
};

struct TrainResult {
    bool        ok = false;       // воркер відпрацював без збою
    std::string error;            // заповнене лише при ok == false
    std::string w_end;
    std::vector<std::string> ckpt_hashes;  // K/k + 1 штук, кожен 64 hex
    int64_t     metric_micro = 0;
    uint32_t    steps = 0;
    double      seconds = 0.0;    // чистий час навчання за версією воркера
};

// Один запит обслуговує і B, і A: зайві поля не потрапляють у task.json.
struct VerifyRequest {
    std::string task_id;
    std::string prev_block_hash;
    std::string miner_id;         // майнера блоку, який перевіряємо (P2)
    uint64_t    round = 0;
    std::string ckpt_from;        // B: початок переходу
    std::string ckpt_to;          // B: кінець переходу
    uint32_t    step_from = 0;    // B: з якого кроку починається перехід
    uint32_t    k = 25;
    uint32_t    batch_size = 64;
    double      lr = 0.01;
    std::string dataset = "mnist";
    double      alpha = 1.0;      // B: відносний поріг (P6)
    std::string w_end;                    // A: на яких вагах міряти
    int64_t     claimed_metric_micro = 0; // A: що заявив майнер
    uint32_t    sample_size = 2000;       // A: розмір підвибірки
};

// Сирий результат виклику процесу. Не плутати з pouw::VerifyResult із verifier.h
// (Крок 8): та структура — вердикт вузла, ця — те, що відповів воркер.
struct VerifyResult {
    bool        ok = false;
    std::string error;
    std::string verdict;          // "accept" | "reject"; при ok == false порожній
    double      distance = 0.0;
    double      threshold = 0.0;
```

```
    double      seconds = 0.0;
};

class WorkerClient {
public:
    explicit WorkerClient(std::string python, std::string worker_py, std::string run_dir);

    TrainResult  train(const TrainRequest& req);
    VerifyResult verify_b(const VerifyRequest& req);
    VerifyResult verify_a(const VerifyRequest& req);

private:
    std::string python_;      // ".venv/bin/python"
    std::string worker_py_;   // "worker/worker.py"
    std::string run_dir_;     // "run/nodeA"
    uint64_t    call_no_ = 0; // лічильник викликів: run_dir_/00017/
};
```

ЧОМУ САМЕ ТАК

Хеші тут — `std::string` у hex, а не тип `Hash` з Кроку 7, бо межа з JSON говорить hex-ом. Нехай клієнт буде тупим перекладачем: рядок увійшов — рядок вийшов; перетворення в `Hash` робить `main.cpp` в одному місці. А `attack_kind` — поле запиту, а не окремий клас, бо атака є властивістю вузла: сценарій «три вузли, один лінивий» має бути різними аргументами запуску, а не іншою збіркою.

УВАГА

`ok == false` **ніколи** не означає «блок відхилено». Це означає «вердикту немає».

Уяви протилежне. Якщо збій воркера трактується як `reject`, майнер, який зуміє зробити так, щоб перевірка падала, відхилить чужі чесні блоки; якщо як `accept` — так само дешево проведе будь-яке шахрайство. Правильно: повторити виклик один раз, потім утриматись — не додавати блок і не рахувати спробу в статистику. У CSV такий рядок іде з `verdict = error` і при підрахунку detection rate виключається явно.

Скелет

`node/src/worker_client.cpp` · тіла написати самому

```
// Приватний помічник (static у цьому ж файлі):
// int run_and_wait(const std::string& cmd, std::string* err)
// 1. FILE* p = popen(cmd.c_str(), "r"); nullptr -> поверни -1.
// 2. Читай вивід через fgets до кінця, інакше труба переповниться.
// 3. int st = pclose(p); WIFEXITED(st) ? WEXITSTATUS(st) : -1.

WorkerClient::WorkerClient(...)   // std::move у поля; створи run_dir_

TrainResult WorkerClient::train(const TrainRequest& req)
// 1. dir = run_dir_ + "/" + номер виклику з ведучими нулями; створи теку; ++call_no_.
// 2. JSON: {"op":"train", поля req, "store_dir":..., "out": dir + "/result.json"};
//    при непорожньому attack_kind додай поле "attack" точно за контрактом Кроку 5:
//    {"kind":"lazy","sigma":0.001,"claim":"prev"} або
//     {"kind":"partial","honest_fraction":0.1}.
// 3. Запиши у dir + "/task.json".
// 4. cmd = python_ + " " + worker_py_ + " " + task_path + " 2> " + dir + "/stderr.txt".
```

```
// 5. rc != 0 -> ok=false, error = "exit N" + останні рядки stderr.txt (без них
//    налагоджувати неможливо). Вихід.
// 6. result.json немає або не парситься, або в ньому "ok" == false -> ok=false. Вихід.
// 7. Перевір відповідь, перш ніж повірити: ckpt_hashes.size() == K/k + 1; кожен рядок
//     64 hex;
//    front() == req.w_start; back() == w_end. Не збіглося -> ok=false: це баг воркера.
// 8. Заповни TrainResult.

VerifyResult WorkerClient::verify_b(const VerifyRequest& req)
// 1. Те саме, але "op":"verify_b", поля ckpt_from/ckpt_to/step_from/k
//    і "threshold_rule": {"kind":"relative","alpha": req.alpha} (P6).
// 2. "verdict" не рівний "accept" і не "reject" -> ok=false.
// 3. Перенеси distance, threshold, seconds.

VerifyResult WorkerClient::verify_a(const VerifyRequest& req)
// 1. Те саме з "op":"verify_a": w_end, claimed_metric_micro, sample_size.
```

ПІДКАЗКИ

Рівень 1. Не пиши три схожі функції. Винеси в приватний метод усе спільне: тека, запис `task.json`, запуск, читання `result.json`. Публічні три відрізняються лише тим, який JSON складають.

Рівень 2. Тека — `std::filesystem::create_directories`. Запис — `std::ofstream`, `j.dump(2)`. Читання — `nlohmann::json::parse` у `try/catch`: на битих даних воно кидає виняток. Код виходу — `WIFEXITED` / `WEXITSTATUS` з `<sys/wait.h>`.

Рівень 3. Ідіома запуску:

```
FILE* p = popen(cmd.c_str(), "r");
if (!p) return -1;
char buf[4096];
while (std::fgets(buf, sizeof(buf), p) != nullptr) { /* збирай */ }
int st = pclose(p);
return WIFEXITED(st) ? WEXITSTATUS(st) : -1;
```

ЧОМУ САМЕ ТАК

Чому `popen` допустимий і де його межі. Виклик один на раунд, вузлів три, усе на одній машині — пул процесів не потрібен. Але обмежень три, і два з них кусаються.

Оболонка. Команда йде через `/bin/sh`: пробіл у шляху розриває рядок на два аргументи, тому лапки навколо кожного шляху обов'язкові. Ін'єкції тут немає лише тому, що шляхи приходять із твого конфіга; щойно шлях почне приходити з мережі, потрібен `fork` + `execv`, який оболонки не бачить.

Таймауту немає. `pclose` блокує до кінця процесу: зависнув воркер — застиг вузол, назавжди й без повідомлення. Для прототипу досить префікса `timeout 300` і коду виходу 124 як `ok=false, error="timeout"`; повне рішення (`fork` + `waitpid(WNOHANG)` у циклі з дедлайном і `kill`) — сорок рядків, у MVP не потрібне, але відсутність нагляду за процесом іде в обмеження роботи.

Виклик блокує потік. Для циклу майнінгу це нормально, для потоку сокетів — ні: вхідний блок клади в чергу і перевіряй окремим потоком.

Скелет: цикл майнінгу

node/src/main.cpp — майнінг одного раунду · тіла написати самому

```
// bool mine_one_round(Chain& chain, WorkerClient& wc, const NodeConfig& cfg)
//  1. const Block& tip = chain.tip(); prev = block_hash(tip.header).
//  2. TrainRequest: w_start = tip.header.w_end (P1), round = tip.header.round + 1,
//     prev_block_hash = prev, miner_id = cfg.miner_id, K/k/lr/batch із завдання (P3),
//     attack_kind = cfg.attack.
//  3. t0 = steady_clock::now(); res = wc.train(req); wall_train = now() - t0.
//  4. !res.ok -> лог worker_error, return false. Вузол живий і далі перевіряє чужі
//     блоки.
//  5. Заголовок: version = 1; height = tip.header.height + 1; prev_hash; task_id;
//     round;
//     miner_id; timestamp; w_start; w_end = res.w_end; K; k;
//     metric_micro = res.metric_micro (копіювати як є, не перераховувати з float).
//  6. ckpt_root перерахуй сам: SHA-256 від конкатенації res.ckpt_hashes по 32 байти.
//  7. Block b{header, ckpt}; прожени через validate_structure(b, tip). Не пройшов —
//     це твій
//     баг: лог і вихід, у мережу такий блок не випускай.
//  8. chain.add(b); chain.save(); розішли сусідам inv (Крок 9); лог mine_ok.
```

ЧОМУ САМЕ ТАК

`metric_micro` копіюється, а не рахується в C++: правило `round(metric × 1e6)` застосовується рівно один раз — там, де живе float. Якби округлювали обидві сторони, вони колись розійшлися б на одиницю в останньому розряді, і два чесні вузли порахували б різні хеші блоку. `ckpt_root` навпаки перераховується, бо за поле заголовка відповідає той, хто заголовок складає: перерахунок коштує одне хешування, а довіра до чужого числа — вечір налагодження, бо локально блок виглядав би правильним, а сусіди його відхиляли б. Загальне правило шва: кожне перетворення робить рівно одна сторона.

Скелет: шлях перевірки вхідного блоку

node/src/main.cpp — реакція на чужий блок · тіла написати самому

```
// pouw::VerifyResult WorkerVerifier::verify(const Block& b, const Block& parent)
//     override
// WorkerVerifier — реалізація інтерфейсу Verifier з Кроку 8 замість AlwaysAccept;
// WorkerClient і параметри (q, alpha) — його поля.
//  1. Батька немає? Поклади b у чергу сиріт, попроси getblocks, вийди. Це не
//     відхилення.
//  2. validate_structure(b, parent) — дешево й повністю в C++: height, prev_hash,
//     w_start == parent.w_end (P1), розмір списку точок == K/k + 1, ckpt.front() ==
//     w_start,
//     ckpt.back() == w_end, ckpt_root збігається з перерахунком, timestamp у межах.
//     Не пройшов -> reject, причина в лог, Python не запускаємо взагалі.
//  3. Семантика. m = b.ckpt_hashes.size() - 1 (кількість переходів).
//     std::random_device rd; std::mt19937_64 gen(rd());          // P5: власна
//     випадковість
//     std::uniform_int_distribution<uint32_t> pick(0, m - 1);  -> q різних індексів j.
//  4. Для кожного j: VerifyRequest{ ckpt_from = ckpt[j], ckpt_to = ckpt[j+1],
//     step_from = j * k,
//     miner_id = b.header.miner_id (P2!), prev_block_hash = b.header.prev_hash,
//     round = b.header.round, alpha = cfg.alpha }; vr = wc.verify_b(req).
//     !vr.ok -> повтори один раз, потім Verdict::Error (не Reject!): блок
//     відкладається.
//     vr.verdict == "reject" -> Verdict::Reject одразу, решту переходів не перевіряй.
```

```
//  5. Один wc.verify_a на w_end і metric_micro блоку.
//  6. Усі accept -> Verdict::Accept; chain.add(b), розсилку й приклеювання сиріт робить
//     викликач, а не верифікатор (Chain не має залежати від torch — див. Крок 8).
//  7. Лог: окремий рядок на кожен виклик перевірки (distance, threshold, seconds).
```

УВАГА

Найспокусливіша помилка кроку — вивести перевірювані переходи з хешу блоку («так усі валідатори перевірять те саме»). Блок складає майнер: він перебирає `timestamp`, доки вибір не впаде на чесно пораховані переходи. При частковій роботі з десятою часткою чесних переходів і $q = 2$ це близько сотні хешувань, тобто безкоштовно. Тому сід валідатора береться з `std::random_device` (P5) і нічим не пов'язаний з умістом блоку.

Ціна — валідатори можуть дійти різних висновків: один вибрав чесний перехід, другий фальшивий. У прототипі кожен вирішує за себе; fraud proof (валідатор розсилає пару «блок, номер переходу», решта перевіряє лише його) іде у фазу 2 і описується в роботі як спроєктований, але не реалізований.

Контракт: три вузли однією командою

scripts/run3.sh

```
#!/usr/bin/env bash
set -euo pipefail

BUILD=./build/node
RUN=run
ATTACK="${ATTACK:-}"          # ATTACK="--attack lazy" ./scripts/run3.sh

rm -rf "$RUN"; mkdir -p "$RUN" store; : > "$RUN/pids"

start() {   # ім'я порт піри [додаткові прапорці]
  "$BUILD" --name "$1" --port "$2" --peers "$3" \
           --store store --run "$RUN/$1" \
           --python .venv/bin/python --worker worker/worker.py \
           --task docs/task.json ${4:-} > "$RUN/$1.log" 2>&1 &
  echo "$!" >> "$RUN/pids"
}

start nodeA 9001 "127.0.0.1:9002,127.0.0.1:9003"
start nodeB 9002 "127.0.0.1:9001,127.0.0.1:9003"
start nodeC 9003 "127.0.0.1:9001,127.0.0.1:9002" "$ATTACK"

trap 'kill $(cat "$RUN/pids") 2>/dev/null || true' EXIT
wait
```

УВАГА

Усі три вузли бачать одну теку `store/`. Це свідоме спрощення: ваги через мережу не передаються, посилання на них — просто хеш. Але отже, в прототипі не існує проблеми доступності даних: атака «я не публікую свої контрольні точки» не моделюється, а плагіат технічно тривіальний. Цей абзац майже дослівно йде в підрозділ «Обмеження» (рішення B2). Обмеження, назване автором, на захисті працює на тебе; знайдене комісією — проти.

Контракт: формат логу

Один рядок на подію, кома як роздільник. Порожнє поле — нічого між комами.

run/nodeA.csv — формат, числа умовні

```
ts,node,event,height,hash,verdict,seconds
1761950412.118,nodeA,mine_start,12,,,
1761950419.902,nodeA,mine_ok,12,4b1f9c2a,,7.784
1761950422.560,nodeB,verify_b,12,4b1f9c2a,accept,2.527
1761950423.104,nodeB,accept,12,4b1f9c2a,accept,3.072
1761950431.740,nodeB,reject,13,9ac30e11,reject,2.610
```

Події: `mine_start`, `mine_ok`, `worker_error`, `block_recv`, `struct_ok`, `struct_fail`, `verify_b`, `verify_a`, `accept`, `reject`, `orphan`, `reorg`.

ЧОМУ САМЕ ТАК

Колонки, а не вільний текст, бо через два тижні потрібні будуть числа, а писати регулярні вирази по власних логах — втрачений вечір. Окремий рядок на фазу, а не один на блок, бо різниця міток `mine_start` і `mine_ok` — це час навчання за годинником, а сума `seconds` по `verify_b` і `verify_a` — час перевірки. Ці два числа дадуть накладні витрати Кроку 11 без додаткового коду.

Твоє завдання

ТВОЄ ЗАВДАННЯ

1. Набери `worker_client.h` як є, напиши тіла `worker_client.cpp`, додай файл у `CMakeLists.txt`.
2. Додай вузлу аргументи: `--name`, `--port`, `--peers`, `--store`, `--run`, `--python`, `--worker`, `--task`, `--attack`, `--q`, `--alpha`.
3. Реалізуй цикл майнінгу і шлях перевірки; семантику сховай за `verifier.h` з Кроку 8.
4. Зроби логування у форматі вище; кожен вузол пише свій `run/<name>.csv`.
5. Напиши `scripts/run3.sh`, запусти три чесні вузли, дай набрати 20 блоків, подивись логи.
6. Запусти `ATTACK="--attack lazy" ./scripts/run3.sh` і знайди в логах рядок із причиною відмови.
7. Заведи `docs/limitations.md`: спільний `store/`, відсутність таймауту й підписів, одна машина.
8. Коміт: `feat(node): worker integration, mining loop and validation path`.

Перевірка

ПЕРЕДБАЧ ДО ЗАПУСКУ

1. При $K = 500$, $k = 25$, $q = 2$ теоретична частка перевірки — $q \cdot k/K$. Порахуй її. Тепер додай старт Python на кожен виклик: яку частку побачиш у логах за годинником і в скільки разів вона перевищить теоретичну?
2. Три однакові вузли на одній машині: хто вигравав більшість раундів і чому саме він?
3. Чи росте власний ланцюг лінивого вузла, якщо сусіди його відхиляють?

ЕТАЛОН ДЛЯ САМОПЕРЕВІРКИ

Точних чисел тут бути не може — вони залежать від заліза. Мають виконуватись властивості:

1. **Три чесні вузли.** Висота росте без зупинок; після кількох раундів усі три показують однакову верхівку (той самий `block_hash` на тій самій висоті), розбіжність — не більше одного блоку і лише поки триває розсилка.
2. Заявлена метрика прийнятих блоків росте від раунду до раунду — з коливаннями, але тренд угору видно. Стоїть на місці — ваги в ланцюгу не оновлюються: перевір, чи справді `w_start` наступного блоку дорівнює `w_end` попереднього.
3. Жодного рядка `worker_error` у здоровому прогоні. Хибних відхилень між чесними вузлами на одній машині немає взагалі: перерахунок переходу тут побітово точний.
4. **Лінивий вузол.** Його блоки отримують `reject` від обох сусідів, у логах — `verify_b` із відстанню, помітно більшою за поріг; у ланцюгу чесних вузлів блоків `nodeC` немає. Сам `nodeC` вважає свій ланцюг правильним — це не баг, а прояв того, що рішення кожен вузол приймає сам.

САМОПЕРЕВІРКА

1. Чому сід вибору переходів не можна виводити з хешу блоку? Оціни в хешуваннях вартість підбору при $q = 2$ і десятій частці чесних переходів.
2. Чому `ok == false` не є вердиктом? Наведи атаку на трактування «збій = reject» і на «збій = accept».
3. Чому `ckpt_root` вузол рахує сам, а `metric_micro` бере як є? Сформулюй спільне правило.
4. Чому структурна перевірка йде першою, а семантична другою? Як це пов'язано з роллю A як дешевого фільтра перед B?
5. Два вузли домайнували одночасно: що стається з роботою того, хто програв, і як це пов'язано з рішенням B1 («перегони») і метрикою «частка корисної роботи»?

Зламай навмисне

ЗЛАМАЙ НАВМИСНЕ

Експеримент 1. Неправильний шлях до Python. Запусти вузол із неіснуючим `--python`. Вузол помирає чи деградує? Правильно: щораунду `worker_error`, вузол не майнить, але

приймає й ретранслює чужі блоки. Якщо він при цьому їх відхиляє — у тебе `ok == false` перетворилось на вердикт.

Експеримент 2. Повільний воркер. Встав `time.sleep(60)` на початку `worker.py`. Що ламається першим: черга вхідних блоків, розсилка, чи вузол просто застигає в `pclose`? Заміряй, на скільки блоків він відстане і чи наздожене ланцюг. Це найкоротший шлях зрозуміти, навіщо таймаут.

Експеримент 3. Одночасний майнінг. Три вузли з однаковим K: подивись події `orphan` і `reorg`. Порахуй, скільки блоків відкинуто не через шахрайство, а через програш у перегонах. Це пряма оцінка частки корисної роботи — число знадобиться в Кроці 11.

Експеримент 4. Порожнє сховище. Запусти вузол із `--store store_empty`. Яка перевірка спрацює першою і що потрапить у лог: `reject` чи `worker_error`? Поясни, чому правильна відповідь друга і що сталося б із чесними блоками сусідів при першій.

У ТЕКСТ РОБОТИ

У розділ 3: схема взаємодії «вузол ↔ воркер», діаграма послідовності раунду, обґрунтування вибору «окремий процес плюс файли» з таблицею альтернатив, формат логу. У розділ 2 — правило P5 у реалізованому вигляді з оцінкою вартості підбору. В «Обмеження» — спільний `store/`, відсутність таймауту й нагляду за процесом, довірені ідентифікатори вузлів.

Крок 11. Перші цифри і звіт керівнику

Мета кроку: шість прогонів, п'ять заповнених таблиць, два графіки і дві сторінки тексту, надіслані керівникові до 1 листопада.

Ідея

Прототип працює — це ще не результат. Результат — це число, у якого є одиниці виміру, методика й розкид. З цього кроку проєкт перестає бути програмуванням і стає вимірюванням.

що	що доводить	чого не дає
мережевий прогін (Кроки 7–10)	що система працює наскрізно: процеси, TCP, окремі воркери	статистики: 30 раундів по 3 вузли — не вибірка
симулятор (Крок 6)	таблиці, ROC-криві, залежності від параметрів на сотнях раундів і 5 сідах	що воно працює в мережі

ЧОМУ САМЕ ТАК

Чому не зібрати всі цифри з мережі? Бо кожен виклик воркера коштує старт Python: сотня раундів на трьох вузлах — це години на один сід, а п'ять сідів і сітка параметрів перетворюють це на тижні. Чому тоді потрібен мережевий прогін? Бо симулятор не доводить, що реалізація існує: у розділі 3 має стояти речення «наскрізний прогін трьох вузлів виконано», підкріплене логами.

Нові слова

слово	що означає
`detection rate`	Частка виявлених спроб шахрайства серед усіх спроб цього типу.
`false rejection rate`	FRR — частка чесних блоків, відхилених помилково.
`накладні витрати`	Час перевірки / час навчання. Міряється двічі: у чистих секундах і за годинником.
`baseline`	Опорне значення. Тут — централізоване навчання тієї самої моделі.
`ROC-крива`	Графік «виявлення проти хибних відхилень» при різних порогах. Замінює одне число кривою.

Контракт: які прогони робимо

ID	сценарій	де	раундів	сідів
R1	3 чесні вузли, механізм B+A	мережа	≥ 30	1

ID	сценарій	де	раундів	сідів
R2	те саме, `nodeC` лінивий	мережа	≥ 30	1
R3	те саме, `nodeC` робить 10% кроків	мережа	≥ 30	1
S1–S3	ті самі три сценарії	симулятор	≥ 200	5

УВАГА

Мережа і симулятор мають дати узгоджені числа виявлення. Розходяться більше ніж на статистичну похибку — це не дрібниця, а знахідка: одна з реалізацій робить не те, що ти думаєш. Найчастіші причини — різне α, різне k, різний порядок даних або інший `miner_id`, через який сід за P2 виходить не той. Шукай причину, а не підганяй.

Контракт: що записуємо

Два файли CSV зафіксовано в Кроці 6, і мережевий прогін свого формату не вигадує: подієвий лог вузла з Кроку 10 зводиться в ті самі колонки. Інакше один скрипт зведення не працюватиме на обох джерелах, а порівняти мережу із симулятором стане нічим.

файл	зернистість	ключові колонки
`<config>-<seed>.csv`	рядок на раунд	`winner`, `winner_role`, `was_attack`, `n_candidates`, `n_rejected`, `accuracy`, `rewards`, `steps_real`, `steps_accepted`, `seconds_train`, `seconds_verify`
`<config>-<seed>-checks.csv`	рядок на перевірку	`candidate`, `candidate_role`, `validator`, `mechanism`, `k`, `q`, `verdict`, `distance`, `threshold`, `seconds_verify`
те саме з мережі	1. дві колонки	`wall_train_s`, `wall_verify_s` — час за годинником, зі стартом Python

Перетворення пише окремий скрипт `experiments/from_logs.py`: `run/<name>.csv` (Крок 10) на вході, ті самі два файли на виході. Роль вузла він бере з конфіга прогону, а не з логу: `winner_role` і `candidate_role` — це відома тобі правда, яка ніколи не потрапляє в код перевірки.

ЧОМУ САМЕ ТАК

Навіщо два виміри часу? Відношення чистих секунд — властивість **протоколу**: скільки обчислень коштує перевірка; воно й порівнюване з літературою. Відношення за годинником — властивість **цієї реалізації**: у ньому сидить старт Python, якого в протоколі немає. Показати лише перше — прикрасити; лише друге — обмовити власний протокол через деталь реалізації. У роботі стоять обидва, з підписом, що є що.

Якщо колонки в тебе називаються інакше — лишай свої. Єдина вимога: не перейменовувати заднім числом, бо скрипти графіків і підписи до таблиць зав'язані на назви.

Контракт: порожні таблиці

Шаблони для розділу 4. Заповнюються твоїми числами; вигадувати або «уточнювати» значення не можна навіть у чернетці.

Т1. Частка виявлення, % (вказати n спроб і кількість сідів).

атака	A	B, $q = 1$	B, $q = 2$	B+A
ліниве повторення				
часткова робота, 10%				

Т2. Хибні відхилення чесних вузлів, %.

пара «залізо / режим»	$k = 10$	$k = 25$	$k = 50$
CPU проти CPU, та сама машина			
CPU проти GPU			
float32 проти float64			
інша версія PyTorch			

Т3. Накладні витрати.

механізм	навчання, с	перевірка, с	частка, %	теоретично $q \cdot k/K$, %
A				
B, $q = 1$				
B, $q = 2$				
B+A				
B+A, за годинником (мережа)				

Т4. Точність за раундами, % (порівняння за кількістю кроків SGD).

раунд	3 чесні	3 вузли, 1 лінивий	централізоване навчання
0			
50			
100			
200			

Т5. Частка нагород атакувальника і корисна робота.

зловмисних вузлів, %	прийнято блоків атакувальника, %	очікувано без перевірки, %	частка корисної роботи, %
0			
10			
20			
33			

УВАГА

Кожна таблиця — з підписом «заповнити своїми числами» лише доти, доки вона порожня; у роботу йде вже з умовами вимірювання.

У T4 порівняння з централізованим навчанням ведеться **за кількістю кроків SGD**, а не за часом: мережа витрачає на ті самі кроки більше часу, це очевидно і нецікаво. Питання таблиці інше — чи дають K кроків у розподіленій системі з перевіркою той самий прогрес, що й K кроків централізовано. У T2 рядок «CPU проти CPU» має дати нуль; ненульове значення тут — не результат, а баг детермінованості.

Скелет: графіки

experiments/plots.py · тіла написати самому

```
# main():
# 1. argparse: --rounds і --checks (по кілька файлів), --outdir, --dpi (типово 150).
# 2. Прочитай обидва набори через csv.DictReader; pandas не обов'язковий.
# 3. figure_accuracy(rounds, outdir); figure_roc(checks, outdir).

# figure_accuracy(rounds, outdir):        # раундовий файл Кроку 6
# 1. Згрупуй за (конфіг без seed, round) -> значення accuracy по сідах; середнє
#    і стандартне відхилення (statistics.mean / pstdev).
# 2. Три криві: S1 (чесні), S2 (з лінивим), baseline (централізоване навчання).
# 3. plt.plot для середнього, plt.fill_between для смуги; різні типи ліній
#    (суцільна, штрихова, пунктир) — рисунок має читатись у чорно-білому друці.
# 4. Підписи осей українською, легенда, savefig(..., dpi=dpi, bbox_inches="tight").

# figure_roc(checks, outdir):             # файл перевірок Кроку 6
# 1. Для кожного alpha з калібрувальних прогонів порахуй пару (FRR, detection):
#    FRR — частка verdict == "reject" серед candidate_role == "honest",
#    detection — частка verdict == "reject" серед решти ролей.
# 2. Відсортуй точки за FRR і з'єднай лінією; окрема крива для кожного k.
# 3. Познач обраний робочий alpha маркером і підпиши його.
# 4. Осі: «хибні відхилення, %» і «виявлення, %», обидві від 0 до 100.
```

УВАГА

Жодного прикрашання: не викидай сід, що дав погану криву, не обрізай вісь, не згладжуй. Сід, який поводиться інакше за решту, — це або знахідка, або баг, і обидва цікавіші за гарний графік. Якщо після п'яти сідів смуга $\pm\sigma$ перекриває різницю між механізмами, чесний висновок такий: «на цій вибірці різниця статистично не значуща».

Контракт: звіт керівнику

Дві сторінки, не більше. Керівник не спеціаліст із блокчейну, тому перший абзац — нагадування постановки, а не архітектура.

розділ	обсяг	що всередині
Що зроблено	5–8 речень	постановка одним абзацом; що працює: три вузли, TCP, MNIST, механізми B і A, атаки

розділ	обсяг	що всередині
Як перевірено	абзац	золоті вектори, юніт-тести, збіг верхівки на трьох вузлах, тест побітового перерахунку
Перші цифри	2–3 таблиці	T1, T3 і крива точності; під кожною — умови вимірювання
Що далі	3–4 рядки	фаза 2: механізми C і D, атаки 2, 4, 5; КТ2 — 13 грудня
Відкриті питання	2–4 питання	конкретні вибори, де потрібна саме його думка

підпис під кожною таблицею

```
Виміряно 28.10.2026. Linux, CPU <модель>, 8 потоків, PyTorch 2.14, MNIST,
K = 500, k = 25, alpha = 1.0, 5 сідів, середнє ± стандартне відхилення.
```

ЧОМУ САМЕ ТАК

Чому «відкриті питання» — обов'язковий розділ? Бо рецензія на текст приходить за два тижні, а відповідь на конкретне питання — за день: «подивіться, будь ласка» дає мовчання, «чи достатньо трьох механізмів для розділу 4» дає рішення. Чому дата й машина біля кожного числа? Бо в лютому ті самі експерименти підуть на T4, числа зміняться, і без підпису буде не зрозуміти, яка таблиця свіжіша.

Правила чесної звітності

1. **Один прогін — не результат.** Мінімум п'ять сідів, у таблиці — середнє і стандартне відхилення. Якщо число справді одне (дороге вимірювання, хмара), так і пиши: «один запуск, порядок величини».
2. **Поріг калібрується на одних сідах, виявлення міряється на інших.** Підібрати α так, щоб виявлення на тих самих даних вийшло красивим, означає показати, що поріг можна підігнати, а не що механізм працює. Сіди 1–5 — калібрування, 6–10 — оцінювання.
3. **Негативний результат — це результат.** Очікування P6 прямо каже: з абсолютним порогом пізно в навчанні ліниве повторення може проходити B. Побачив — це окремий висновок роботи. Механізм із чесно виміряною межею застосовності цінніший за механізм, про який написано «працює».
4. **Число з іншої машини маркується.** Змішувати машини в одній колонці без застереження не можна.
5. **«Приблизно» не є одиницею виміру.** У таблиці стоять n, кількість сідів і дата. Немає їх — таблиця не готова.

Твоє завдання

ТВОЄ ЗАВДАННЯ

1. Прогони R1, R2, R3; логи зберігай у теках `run/R1`, `run/R2`, `run/R3`.
2. Прогони S1–S3 у симуляторі: не менш ніж 200 раундів і 5 сідів на сценарій.
3. Напиши `experiments/from_logs.py`: подієвий лог Кроку 10 у два файли формату Кроку 6.

4. Зведи CSV у таблиці T1–T5. Клітинки без даних лишай порожніми з позначкою «не міряно».
5. Напиши `experiments/plots.py` і збери два рисунки: точність за раундами і ROC.
6. Порівняй виявлення в мережі й у симуляторі; розбіжність більша за похибку — знайди причину.
7. Напиши звіт на дві сторінки за структурою вище і надішли керівникові до 1 листопада.
8. Запиши підсумок у `docs/`: які числа отримано, на якому залізі, якою датою.
9. Коміт: `exp(run): first end-to-end numbers and report`.

Перевірка

ПЕРЕДБАЧ ДО ЗАПУСКУ

Запиши відповіді **до** того, як подивишся в CSV:

1. Часткова робота: частка чесних переходів $f = 0.1$, перевіряється q випадкових; імовірність виявлення — $1 - f^q$. Порахуй для $q = 1$ і $q = 2$ і впиши очікуване в T1 олівцем.
2. Яким буде FRR у рядку «CPU проти CPU» таблиці T2?
3. У скільки разів накладні витрати за годинником перевищать чисті і від чого це залежить — від K, від k чи від q?
4. Чи наздожене крива «3 чесні вузли» централізоване навчання за однакової кількості кроків?

ЕТАЛОН ДЛЯ САМОПЕРЕВІРКИ

Замість чисел — властивості, які мають виконуватись:

1. Виявлення лінивого повторення механізмом B при $k \le 50$ близьке до повного, і це узгоджується із запасом між похибкою чесного перерахунку й відстанню лінивого блоку, який ти міряв при калібруванні δ.
2. Виявлення часткової роботи зростає з q за законом $1 - f^q$, і виміряні точки лягають на цю криву в межах похибки. Не лягають — вибір переходів нерівномірний або передбачуваний: шукай баг у генераторі валідатора.
3. FRR на одній машині в детермінованому режимі дорівнює нулю точно, а не «майже».
4. Накладні витрати в чистих секундах близькі до $q \cdot k/K$ плюс вартість A; за годинником — помітно більші, і різниця приблизно дорівнює кількості викликів воркера, помноженій на час старту Python.
5. Крива «3 чесні вузли» йде поруч із baseline за кількістю кроків, а крива «з лінивим вузлом» не гірша за чесну: його блоки відхиляються, тож у ланцюг потрапляють лише чесні. Гірша — відхилення спрацьовують не завжди, і це треба пояснити, а не списати на шум.

САМОПЕРЕВІРКА

1. Чому не можна міряти виявлення на тих самих сідах, на яких калібрувався поріг?
2. Що саме доводить мережевий прогін, чого не доводить симулятор, і навпаки?

3. Чому таблиця без кількості сідів, назви машини й дати марна — навіть якщо числа правильні?
4. Як дістати «частку корисної роботи» з логів Кроку 10, не пишучи нового коду?
5. Чому «механізм A не виявив плагіат» — це результат, а не привід переробляти A?

Зламай навмисне

ЗЛАМАЙ НАВМИСНЕ

Експеримент 1. Підганяння порогу. Підкрути α так, щоб виявлення стало стовідсотковим, і подивись, що сталося з FRR. Постав обидві точки на ROC-криву. Скільки чесних блоків ти щойно продав за красиве число в T1?

Експеримент 2. Один сід проти п'яти. Побудуй T1 по одному сіду, потім по п'яти із середнім і стандартним відхиленням. Порівняй розкид між сідами з різницею між механізмами. Якщо розкид більший — що саме вимірювала таблиця по одному сіду?

Експеримент 3. Порівняння за часом. Побудуй T4 не за кроками, а за витраченим часом. Наскільки зміниться висновок про наздоганяння baseline і яке з двох порівнянь чесне?

У ТЕКСТ РОБОТИ

Станом на 1 листопада написано більше роботи, ніж здається:

1. рішення P1–P11, життєвий цикл раунду, механізми A–D і моделі атак — кістяк **розділу 2**;
2. архітектура «C++-вузол ↔ Python-воркер», формат заголовка блоку, канонічні хеші, протокол повідомлень і формат логу — **розділ 3**; писати його треба в листопаді, поки пам'ятаєш, чому зроблено саме так;
3. T1–T5, дві криві й методика вимірювання — підрозділ «Методика» **розділу 4** плюс попередні результати E0 і E2;
4. `docs/limitations.md` — окремий підрозділ розділу 4 і частина моделі загроз у розділі 2.

КОНТРОЛЬНА ТОЧКА · 1 ЛИСТОПАДА 2026

Прототип готовий, якщо одночасно виконано:

1. три вузли на C++ обмінюються блоками по TCP локально;
2. вузли навчають CNN на MNIST, ланцюг росте, верхівка в усіх однакова;
3. працює механізм B, а як базова лінія — A;
4. реалізовано щонайменше одну атаку, і її видно в цифрах, а не лише в коді;
5. є перша таблиця: виявлення, FRR, накладні витрати, точність за раундами.

Не встигаєш — ріж обсяг, а не переноси дату: мережі немає — прототипом вважається симулятор плюс C++-ланцюг без P2P, а мережа переїжджає у фазу 2 замість механізму D; механізм D переходить в «опційні»; CIFAR-10 прибирається зовсім, лишаються MNIST і Fashion-MNIST; fraud proof (P5) лишається описом у розділі 2 без реалізації.

Що б не було зрізано, звіт керівникові надсилається 1 листопада в будь-якому разі — з чесним переліком того, що не встиглось, і датою, коли воно буде.

Фаза 2. Решта механізмів і атак (2 листопада — 13 грудня)

Кроки 0–11 доведені до рівня контрактів і скелетів: там сказано, який файл створити, які сигнатури набрати і що має статись у кожному тілі функції. Починаючи з цього розділу гайд навмисно стає грубішим: далі — мета, задачі, критерій готовності, пастки й вказівники, але **без скелетів коду**.

> Причина не в лінощах автора гайда: конструкція другої половини проєкту залежить від чисел, яких ще немає. Який поріг δ брати, чи потрібен D окремий контроль норми, скільки реплік тримати в C — це визначають перші вимірювання на твоєму залізі, а не здогад у вересні. Розділи фаз 2–4 дописуються до рівня кроків тоді, коли прототип запрацює і з'являться перші CSV.

Формат кожного тижня однаковий: **мета**, **задачі** і **критерій готовності**, за яким ти впізнаєш, що тиждень закритий. Критерій завжди перевірний: його можна або показати на екрані, або ні.

Що має з'явитися за шість тижнів

тиждень	дати	що робимо	год
8	02–08.11	механізм C: реплікація, commit-reveal, медіана	8
9	09–15.11	механізм D: косинус або спад втрат; варіант A+	8
10	16–22.11	атака 2 (плагіат) і атака 4 (отруєння міток)	8
11	23–29.11	атака 5 як абляція сили протоколу	8
12	30.11–06.12	конфіг експериментів, CSV, графіки; fraud proof (опційно)	8
13	07–13.12	запас; чернетка розділу 1; план розділу 2 з керівником	8

Паралельно щотижня 2–3 години йдуть на текст розділу 1. Це не «якщо лишиться час» — це частина плану, і саме вона рятує квітень.

Тиждень 8. Механізм C — реплікація

Мета. Одне завдання видається n вузлам, результати зіставляються, приймається узгоджений. Це найнадійніший механізм і найдорожчий: корисна робота ділиться на n.

Тут є тонкість, якої немає в A, B і D. Щоб результати реплік **можна було** порівнювати, усі мають навчатись на однакових даних в однаковому порядку. Отже, для C seed виводиться із завдання, а не з майнера:

механізм	правило seed	чому саме так
A, B, D	`seed = SHA256(prev_hash ‖ miner_id ‖ round)` (P2)	у кожного майнера свій порядок даних — це й ловить плагіат
C	`seed_C = SHA256(prev_hash ‖ task_id ‖ round)` (P8)	репліки мають дістати однакові батчі, інакше їхні ваги непорівнянні

> **УВАГА**
> Це прямий конфлікт із рішенням P2. Прив'язка seed до `miner_id` існує саме для того, щоб копіювальник не видав чужий результат за свій: у нього інший порядок даних, тож перерахунок за механізмом B його викриє. У механізмі C ця прив'язка знята за побудовою — усі репліки законно мають однакові дані, і скопійовані ваги стають **невідрізнимими** від чесно порахованих. Тобто механізм C, доданий наївно, власноруч відкриває атаку 2, яку закривала P2.

Виправлення — commit-reveal (P8). Раунд ділиться на дві фази:

1. **Зобов'язання.** До дедлайну кожна репліка публікує лише `commit = SHA256(w_end ‖ miner_id ‖ nonce)`. Це 32 байти, з яких нічого не витягнеш, а `nonce` робить перебір безглуздим.
2. **Розкриття.** Після дедлайну репліки публікують ваги і `nonce`; кожен перевіряє відповідність зобов'язанню.
3. **Агрегація.** Медіана (покоординатна, за Yin et al. 2018) або більшість тих реплік, що збіглися в межах δ.

Копіювальник до дедлайну не бачить нічого, що можна скопіювати. Це не вигадка під диплом: та сама проблема й те саме лікування є в Bittensor, де валідатори копіювали ваги інших валідаторів замість власного оцінювання, і мережа перейшла на commit-reveal — готовий абзац для розділу 1.

Задачі. Видати одне завдання $n = 3$ реплікам у симуляторі; реалізувати дві фази раунду, порівняння в межах δ і медіану; поміряти накладні витрати (мають вийти $\approx n$ разів); прогнати випадок, коли одна з реплік — копіювальник. Дедлайн у симуляторі — це **порядок**, а не час: досить правила «поки всі зобов'язання не зібрані, жодне розкриття не приймається».

Критерій готовності. Три репліки з однаковим `seed_C` дають ваги, що збігаються в межах δ; копіювальник із вимкненим commit-reveal проходить перевірку, а з увімкненим — ні. Обидва числа в CSV.

Тиждень 9. Механізм D — градієнтна узгодженість, і варіант A+

Мета. Найдешевший механізм із чотирьох і єдиний, який майже не залежить від заліза. D не звіряє ваги побітово — він перевіряє, що оновлення **спрямоване туди, куди веде градієнт**. Два варіанти на вибір, вони відрізняються кількома рядками:

варіант	що рахує	звідки взято
косинус	$\cos(\Delta W, -\nabla L)$ на кількох батчах із призначених майнерові	FLTrust
спад втрат	чи зменшує оновлення втрати на вибірці валідатора: $f(x) - f(x - \gamma u) - \rho\|u\|^2$	Zeno

Ціна — один прямий і зворотний прохід на кількох батчах, тобто частки секунди. Для порівняння: механізм B коштує $q \cdot k/K$ від раунду, механізм C коштує n раундів.

D толерантний до різного заліза тому, що оцінює **ефект** оновлення, а не його точні числа: різниця між CPU і T4 у шостому знаку зміщує косинус на величину, якої не видно, тоді як B цю саму різницю зобов'язаний відрізнити від шахрайства порогом. Очікування, яке перевірять E1–E2: у D частка хибних відхилень майже нульова на всіх парах пристроїв — якщо підтвердиться, це один із найкорисніших висновків роботи.

Тим самим тижнем робиться **A+** — дешеве посилення механізму A (P9): порівняти спад втрат на **призначених** майнерові батчах і на випадкових **непризначених**. Чесний вузол помітно більше покращує саме призначені; скопійована модель покращує і ті, й ті однаково (ідея з Templar/Gauntlet у Bittensor).

> **УВАГА**
> Дві пастки D, які варто зафіксувати до вимірювань. Перша: косинус не бачить масштабу — вузол, який зробив крихітний крок у правильному напрямку, дасть чудовий косинус, не виконавши роботи. Тому поряд із косинусом завжди міряй норму $\|\Delta W\|$ і клади обидва числа в CSV. Друга: D нічого не каже про конкретні ваги, тож вузол, який навчався сумлінно, а потім додав невеликий отруйний зсув, його пройде. Це не вада реалізації, а межа механізму, і в розділі 4 вона має бути названа прямо.

Критерій готовності. `verify_d` повертає число для сумлінного блоку і для кожної з реалізованих атак (1 і 3); поріг обраний тією самою процедурою, що й δ у P6; час перевірки поміряний.

Тиждень 10. Атаки 2 (плагіат) і 4 (отруєння міток)

Атака 2 — плагіат. Вузол копіює чужий результат — ваги, контрольні точки, заявлену метрику — і підставляє свій `miner_id`. Від симулятора для цього потрібен **пул** поданих у раунді результатів, з якого атакувальник бере чужий; якщо досі майнери оброблялися строго послідовно, це треба поміняти.

механізм	що має статись	чому
A	пропускає	метрика справжня — її ж порахував чесний вузол
B	ловить	перерахунок іде на **батчах копіювальника**, а вони інші: seed прив'язаний до `miner_id` (P2)
C	пропускає без commit-reveal, ловить із ним	див. тиждень 8
D	ловить частково	напрямок оновлення чужий і не узгоджений із призначеними батчами копіювальника

> **У ТЕКСТ РОБОТИ**
> Клітинка «атака 2 × механізм B» — це пряме експериментальне підтвердження рішення P2: у розділі 2 ти пишеш правило, у розділі 4 показуєш число. Такі пари «рішення → цифра» комісія цінує найбільше, бо видно, що правило не переписане з чужої статті, а перевірене.

Атака 4 — отруєння міток. Вузол навчається чесно, але на зіпсованих мітках: $y' = (y + 1) \bmod 10$ на частці p прикладів (рецепт Tolpegin et al.). Мета атаки інша, ніж у решти: не отримати нагороду дешево, а **погіршити спільну модель**. Тому міряти треба не лише виявлення, а й деградацію точності — ця атака піде в E3. Ключове питання, на яке треба відповісти **до** реалізації: звідки валідатор бере мітки, коли перераховує перехід у механізмі B?

припущення про дані	що робить B з отруєнням	ким ловиться
валідатор має власну копію канонічного набору	ловить: перерахунок іде на чистих мітках, ваги розійдуться	B
дані локальні для майнера, валідатор відтворює перехід на них же	пропускає за побудовою: та сама отрута дає той самий результат	A на чистих даних валідатора або D

> **УВАГА**
> Рішення прийняте і записане як P12 у `docs/design-decisions.md`: **основний варіант — перший**, канонічний публічний набір даних, зафіксований хешем у завданні, а порція майнера задається його ж seed (P2). Інакше руйнується весь механізм B: валідатор перераховує перехід саме тому, що має ті самі дані. Другий варіант (мітки локальні для майнера) лишається **одним** контрольним прогоном у E2 — і саме він дає цінний результат: щойно дані перестають бути публічними, перерахунок контрольних точок стає сліпим до отруєння, і ловити його можуть тільки A на чистій вибірці валідатора та D.
>
> Наслідок, який треба чесно записати: за основного варіанта B ловить отруєння майже завжди, бо зіпсовані мітки псують **кожен** перехід, а не один. Тому цінність атаки 4 не в клітинці «4 × B», а в кривій деградації точності, коли отруєння проходить (режим «тільки A») і в контрольному прогоні з локальними мітками.

Критерій готовності. `attack_plagiarism` і `attack_poison(p)` працюють у симуляторі, для обох є вердикти всіх наявних механізмів; для отруєння додатково є крива точності спільної моделі при $p \in \{0.1, 0.3, 0.5\}$ проти сумлінного прогону.

Тиждень 11. Атака 5 — підробка траєкторії як абляція

Мета. Найскладніша атака і найцінніший експеримент роботи. Головне — правильно поставити питання. Питання не «чи зможу я зламати свій протокол» (це змагання з дослідниками, які писали статті роками), а **«які саме правила протоколу закривають підробку»**. Тому атака 5 подається як абляція: сітка $2 \times 2 \times 2$ по трьох осях сили протоколу.

вісь	слабкий варіант	сильний варіант	рішення
learning rate	η заявляє майнер	η фіксоване в завданні	P3

вісь	слабкий варіант	сильний варіант	рішення
поріг	абсолютний δ	відносний $\delta = \alpha \cdot \|\Delta W\|$	P6
вибір переходів	передбачуваний (напр. з `block_hash`)	приватний вибір валідатора	P5

В основі — дві опубліковані атаки Fang et al. 2023, обидві прості й обидві без градієнтів другого порядку (близько 50 рядків кожна):

1. **Нескінченно малі оновлення.** Атакувальник інтерполює контрольні точки між стартом і ціллю кроками, меншими за δ, і заявляє $\eta \to 0$. Валідатор, перераховуючи перехід із заявленим η, майже не зрушує ваги — і його результат збігається з інтерпольованою точкою. Перевірка проходить, навіть якщо перевіряти **кожен** перехід.
2. **Blindfold top-Q.** Якщо правило вибору відоме (наприклад, перевіряють Q найбільших оновлень), атакувальник робить рівно Q справжніх великих кроків — саме їх і перевірять, — а решту інтерполює з меншою нормою, щоб вони не потрапили у вибірку.

Тепер видно, чому це абляція, а не одна атака. Перша живе з того, що η обирає майнер: із фіксованим η (P3) валідатор перераховує перехід із **правильним** кроком, і інтерпольована точка йому не відповідає. Друга живе з передбачуваності вибору: з приватним вибором (P5) атакувальник не знає, які переходи робити справжніми.

Обидві будуються навколо однієї заготовки — лінійної інтерполяції $C_j = W_s + (j/m)(W_{\text{tgt}} - W_s)$ між станом із ланцюга й ціллю. Ескіз варіантів 5a, 5b і 5c уже записаний у `docs/research/literature.md`, розділ 9; код авторів дивись як довідку, але пиши сам. А от атаку Zhang et al. (S&P 2022) не пиши: їй потрібні L-BFGS і градієнти другого порядку, а працює вона лише при «вигідних» налаштуваннях ($k = 100$, $\delta = 0.1$). Її місце — абзац в огляді і рядок в обмеженнях.

> ІДЕЯ
> Очікуваний результат: у клітинці «η фіксоване + відносний поріг + приватний вибір» обидві атаки мають провалитися. Це негативний результат — і він і є результатом. Речення «атака, яка ламає опублікований PoL, не працює проти протоколу з фіксованим η і приватними викликами, і ось сітка з восьми клітинок, яка показує, яке саме з трьох правил це робить» — сильніший внесок, ніж іще одна зламана схема. Формулюй його саме так і у висновках, і на захисті.

Критерій готовності. Усі вісім клітинок дають число (частку виявлення), і щонайменше дві з них відрізняються якісно: слабка конфігурація пробивається, сильна — ні.

Тиждень 12. Конфіг, CSV, графіки і fraud proof

Мета. Перетворити набір скриптів на **стенд**: один JSON-конфіг на вході, один CSV на виході, графіки будуються з CSV окремою командою. Без цього фаза 3 перетвориться на ручні запуски з числами в блокноті, і в лютому їх не буде як відтворити.

experiments/results/<run_id>.csv — схема Кроку 6 плюс метадані фази 3

```
# рядок на раунд — як зафіксовано в Кроці 6:
run_id, seed, round, winner, winner_role, was_attack, n_candidates, n_rejected,
verdicts, accuracy, rewards, steps_real, steps_accepted, seconds_train, seconds_verify,
# додається у фазі 3:
device, torch_version, commit, started_at
```

Схему колонок не вигадуй наново: вона вже зафіксована в Кроці 6 і складається з двох файлів — рядок на раунд і рядок на кожну перевірку. Фаза 3 лише дописує до них чотири колонки метаданих. Це не бюрократія: саме вони відрізняють число, яке можна захищати, від числа, про яке через три місяці невідомо, на чому воно отримане.

Fraud proof (опційно). Валідатор, який відхилив блок, розсилає не вердикт, а **доказ** — пару `(block_hash, j)`, де j — номер переходу, на якому знайдено невідповідність. Будь-хто інший перевіряє лише цей перехід (ціна — k кроків) і переконується сам. Це полегшена версія гри Truebit/Verde і пряма відповідь на проблему з P5: валідатори обирають переходи приватно, тож можуть дійти різних висновків, а fraud proof дає спосіб розв'язати суперечку дешево.

Критерій готовності. Команда «конфіг → CSV» відпрацьовує від початку до кінця на маленькому прогоні (2 раунди, 2 вузли), а скрипт графіків будує з цього CSV картинку. Fraud proof, якщо зроблений: третій вузол розв'язує суперечку двох, перерахувавши один перехід.

Тиждень 13. Запас, розділ 1, погодження з керівником

Тиждень свідомо порожній на код. Він іде на чернетку розділу 1 (вона збиралася з листопада по 2–3 години), погодження плану розділу 2 з керівником і добивання того, що з'їхало. Якщо все встигнуто — найкращий момент прогнати E0 і E1 раніше графіка.

КОНТРОЛЬНА ТОЧКА · 13 ГРУДНЯ 2026

КТ2 виконана, якщо одночасно:

1. працюють усі чотири механізми — A (і A+), B, C, D — і кожен повертає число, а не лише вердикт;
2. реалізовані всі п'ять атак, включно з абляційною сіткою для атаки 5;
3. експеримент запускається з JSON-конфіга і пише CSV із повними метаданими прогону;
4. є чернетка розділу 1 (15–18 сторінок, нехай сира) і погоджений план розділу 2.

Якщо не встигаю — ріжемо в такому порядку: fraud proof (лишається описом у розділі 2); варіант A+; варіант «спад втрат» у D (лишається косинус). Механізми C і D та атаки 2, 4, 5 не ріжуться: без них немає головної таблиці, а отже немає роботи.

УВАГА

Фаза 2 — місце, де дипломи гинуть від розростання задачі. Спокуса виглядає невинно: «додам ще один варіант агрегації», «спробую Krum замість медіани», «зроблю адаптивний поріг». Кожна така думка коштує тиждень і не додає жодного рядка в таблицю розділу 4. Критерій приймання в цій фазі — **«механізм працює і міряється»**, а не «механізм оптимальний»: неоптимальний механізм із чесно поміряними detection rate, FRR і накладними

> витратами — це результат, а оптимальний механізм без цифр — нуль. Ідеї покращень ідуть у `docs/ideas.md` і в абзац «напрями подальших досліджень».

Фаза 3. Експерименти (14 грудня — 28 лютого)

Фаза 3 не додає функціональності. Вона перетворює працюючий код на таблиці й графіки розділу 4 — і на цьому все. Головне правило фази: **після старту експериментів протокол не змінюється**. Якщо змінити поріг, порядок даних чи правило раунду посеред серії, половина зібраних чисел стане непорівнянною з другою половиною, і це виявиться в березні, коли переробляти вже нема коли.

План експериментів

ID	на яке питання відповідає	конфігурація	артефакт
E0	чи система взагалі працює і чи не втрачає навчання	3–5 сумлінних вузлів, без атак, 100+ раундів	крива точності проти централізованої базової лінії; частка корисної роботи
E1	який поріг δ брати і скільки чесних блоків він відкине	пари пристроїв $\times$ $k \in \{1, 10, 50, 100\}$, ≈ 20 стартових точок	розподіли похибок, вибране δ, ROC-криві
E2	який механізм яку атаку ловить і якою ціною	5 атак $\times$ {A, A+, B, C, D, B+D}	**головна таблиця роботи**: detection rate, FRR, накладні витрати
E3	що стається з мережею, коли зловмисників багато	симулятор, 10 вузлів, частка 0/10/20/30/50 %	деградація точності; частка нагород атакувальників
E4	як параметри перевірки міняють компроміс	розмір підвибірки (A); k і q (B); поріг абсолютний/відносний; n (C); поріг (D)	криві «виявлення проти витрат»
E5	які правила протоколу закривають підробку траєкторії	сітка $2 \times 2 \times 2$ для атаки 5	таблиця абляції на вісім клітинок
E6	чи висновки не прив'язані до MNIST	повтор E2 на Fashion-MNIST (CIFAR-10 — якщо встигаю)	друга копія головної таблиці
E7	скільки коштує справжня мережа і чи вона працює наскрізно	3 вузли C++ по TCP локально, десятки раундів	накладні витрати мережі; записане демо для захисту

Пастка в кожному експерименті

E0. Пастка — нечесна базова лінія. Централізоване навчання має отримати **ту саму сумарну кількість кроків SGD**, ті самі гіперпараметри і те саме правило порядку даних. Друга пастка: за правилом перегонів (B1) у ланцюг потрапляє один блок за раунд, тож робота решти $N - 1$

майнерів пропадає і частка корисної роботи виходить $\approx 1/N$. Це не баг реалізації, а властивість обраного правила — її треба показати і чесно обговорити, а не «виправити» напередодні здачі.

E1. Найважливіший експеримент після E2, бо з нього береться δ для всіх інших. Пастка перша: δ, відкалібрований на одному k або на одній парі пристроїв, не переноситься на інші — похибка чесного вузла росте з k на порядки. Пастка друга, тонша: якщо калібрувати δ на тих самих прогонах, на яких потім міряти FRR, число вийде заниженим — стартові точки для калібрування і для оцінювання мають бути різними.

> Орієнтир, звідки брати очікування. Рання проба автора на **іншій машині** й на синтетичних даних дала при $k = 10$ похибку чесного вузла на іншому пристрої $\approx 1.6 \cdot 10^{-6}$ проти відстані лінивого повторення $\approx 2.0 \cdot 10^{-3}$ — запас приблизно в тисячу разів; при $k = 100$ запас падав до одиниць разів.
>
> Твої числа будуть іншими: інше залізо, справжній MNIST, інша версія PyTorch. Переміряти треба на своєму ПК, на справжніх даних і щонайменше на п'яти seed. Пари пристроїв: CPU проти CUDA (або проти T4 на Kaggle), валідатор у float32 проти float64, різна кількість потоків CPU, детермінований режим проти недетермінованого.

E2. Головна таблиця роботи — і головне місце, де можна непомітно зробити порівняння беззмістовним. Пастка: підібрати кожному механізму свій поріг так, щоб він гарно виглядав. Порівнювати механізми можна лише в **одній робочій точці**: δ береться з E1 один раз і далі не чіпається, а залежність від порогу показують E4 і ROC-криві. Друга пастка: detection rate без FRR нічого не вартий, бо механізм, який відхиляє все підряд, має ідеальне виявлення. Три числа в кожній клітинці — виявлення, хибні відхилення, накладні витрати — і жодне не публікується окремо від двох інших.

E3. Пастка — вибір атаки. Частку зловмисників має сенс міряти на атаці, яка справді псує модель, тобто на отруєнні (4); ліниве повторення ловиться майже завжди, і крива деградації вийде плоскою незалежно від частки. Друга пастка: частка нагород атакувальників залежить від правила раунду (B1), тож це правило треба назвати поруч із числом, інакше воно непорівнянне ні з чим.

E4. Пастка — видати п'ять одновимірних зрізів за дослідження взаємодій. Якщо міняти по одному параметру за раз, ти отримуєш п'ять зрізів, а не сітку, і в тексті так і треба написати. Друга пастка: накладні витрати треба **міряти**, а не рахувати за формулою $q \cdot k/K$ — реальна перевірка включає запуск процесу Python, завантаження ваг зі сховища і серіалізацію, і на малій моделі це може виявитися більшим за самі обчислення.

E5. Пастка — сплутати «атака не працює» з «атака не реалізована». Нуль у клітинці означає одне з двох, і розрізнити їх можна лише контрольними перевірками: підроблений блок має проходити структурну валідацію, його контрольні точки мають хешуватися рівно в те, що заявлено в блоці, а заявлена метрика має бути досяжною. Додай ці перевірки в сам експеримент.

E6. Пастка рівно одна, зате вона губить весь експеримент: перенести δ з MNIST на Fashion-MNIST. Інша модель і інші дані означають інший масштаб оновлень, а отже, інший розподіл похибок чесних вузлів. Калібрування (скорочену версію E1) треба повторити для кожного нового датасета.

E7. Пастка — намагатися дістати з мережі наукові числа. Прогонів буде мало, шуму багато, і detection rate звідти братися не має: він рахується в симуляторі. Завдання E7 інше — показати накладні витрати самої мережі і довести, що система існує не лише як скрипт. І окремо: демо для захисту має бути **записане** заздалегідь, бо живий запуск трьох вузлів перед комісією — це ризик без жодної вигоди.

Правила, спільні для всіх експериментів

1. Щонайменше **5 seed** на кожну конфігурацію. Одне число без розкиду — це не вимірювання.
2. У таблицях і на графіках — **середнє ± стандартне відхилення**, ніколи не одне значення.
3. **Один рядок CSV на раунд**, а не на прогін: агрегувати завжди можна потім, розагрегувати — ніколи.
4. Кожен прогін записує **машину, пристрій, дату, версію PyTorch і хеш коміта**. Прогін без цих полів вважається неіснуючим і переганяється.
5. Результати **ніколи не правляться руками**. Виправлення вносяться в код або конфіг, після чого прогін повторюється. Один відредагований у таблиці символ знецінює всю серію.
6. Код заморожений з 28 лютого. Після цієї дати правляться лише помилки, які роблять числа хибними, і кожна така правка означає повторний прогін відповідної серії.

> **УВАГА**
> Правила Kaggle і Colab прямо **забороняють майнінг криптовалют** (а Colab — ще й P2P-обмін файлами). Твої C++-вузли з майнінгом блоків і TCP-обміном автоматичні антизловживальні системи цілком можуть розпізнати саме так, а наслідок — блокування акаунта разом із усіма результатами. Правило без винятків: **у хмарі запускається лише Python** — навчання і перевірка як пакетні задачі через «Save & Run All», з вивантаженням CSV собі. Мережа вузлів працює тільки локально.

Що ріжемо, якщо не встигаю

черга	що ріжемо	чим це компенсувати в тексті
1	E6 повністю або лише Fashion-MNIST без CIFAR-10	прямо записати в обмеження: висновки перевірені на одному датасеті
2	E7 як джерело чисел, лишається тільки демо	накладні витрати мережі оцінити аналітично й позначити як оцінку
3	звузити E4 до двох параметрів замість п'яти	решту параметрів зафіксувати й назвати значення

E0, E1, E2 і E5 не ріжуться за жодних обставин: це ядро роботи. E0 доводить, що система працює, E1 дає поріг, E2 — головна таблиця, E5 — основна наукова новизна.

> **КОНТРОЛЬНА ТОЧКА · 28 ЛЮТОГО 2027**
> **КТ3: експерименти завершено, код заморожено.**
> 1. усі обов'язкові експерименти прогнані, CSV лежать в `experiments/results/`;

2. побудовані всі таблиці й графіки, які підуть у розділ 4;
3. для кожного числа відомо, на якій машині, з яким seed і на якому коміті воно отримане;
4. є чернетка розділу 2.

У ТЕКСТ РОБОТИ

З цієї фази в роботу йде не лише розділ 4, а й підрозділ «методика експерименту»: конфігурація заліза, версії бібліотек, кількість seed, спосіб калібрування δ, означення всіх чотирьох метрик. Пиши його **в грудні**, коли запускаєш перші прогони, а не в квітні за спогадами.

Фаза 4. Текст роботи (листопад — червень)

> **УВАГА**
> Текст починається в листопаді, а не у квітні. Перша причина: рішення найлегше описати в тиждень, коли ти його приймаєш. У листопаді ти пам'ятаєш, **чому** seed прив'язаний до `miner_id` і які ще варіанти розглядались; у квітні лишається сам факт, і розділ 2 перетворюється на переказ коду — найгірший різновид дипломного тексту. Друга: рамки «у текст роботи» в кроках 0–11 існують саме для того, щоб їх копіювати, це вже написані абзаци. Третя, неформальна: у квітні експерименти **завжди** переповзають за графік, і тиждень, зайнятий дописуванням розділу 1, — це тиждень, який ти в себе вкрав.

Обсяг і графік

частина	сторінок	чернетка до	звідки береться
Вступ	4–5	18.04	пишеться останнім, з готових розділів
Розділ 1. Аналіз предметної області	15–18	13.12	`docs/research/literature.md`
Розділ 2. Проєктування протоколу	12–15	21.02	`docs/design-decisions.md` + рамки «у текст» кроків 1–6
Розділ 3. Програмна реалізація	12–15	21.03	рамки «у текст» кроків 1–11
Розділ 4. Експериментальне дослідження	12–15	11.04	CSV і графіки фази 3
Висновки	2	18.04	відповіді на сім задач зі вступу

Разом 55–70 сторінок без джерел і додатків, 20–25 джерел. Ядро списку вже відібране в `docs/research/literature.md`, розділ 11.

Розділ 1. Аналіз предметної області

Що вже є. Практично весь матеріал у `docs/research/literature.md`: лінія PoW → PoUW → Proof of Learning, атаки на PoL, порівняльна таблиця індустрійних систем (Gensyn, Bittensor, Prime Intellect, Nous, Truebit), джерела про недетермінованість навчання і розділ 10 з аналізом прогалини.

Що писати з нуля. Зв'язний текст — у файлі лежать нотатки, а не розділ. Пояснити PoW з нуля (керівник не спеціалізується на блокчейні, робота має бути самодостатньою); провести читача від асиметрії «дорого знайти — дешево перевірити» до її втрати в навчанні; закінчити постановкою задачі, яка **випливає** з прогалини. Окремо: у `literature.md` частина позицій має позначку **(не перевірено)** — перед подачею кожну таку позицію звірити з оригіналом, бо помилку в бібліографії на нормоконтролі помічають одразу.

Розділ 2. Проєктування протоколу

Що вже є. Рішення P1–P11 з обґрунтуванням «яку атаку закриває» — готовий кістяк підрозділу про правила протоколу. Таблиця заголовка блоку і правило канонічного хешу — з рамки «у текст» Кроку 1. Список «поза межами роботи» — готова модель загроз і готовий підрозділ обмежень.

Що писати з нуля. Формальний опис чотирьох механізмів однаковою мовою (вхід, що перераховується, критерій прийняття, вартість) — у нотатках вони описані різними словами. Життєвий цикл завдання у вигляді схеми. Формальні моделі п'яти атак: що атакувальник знає, що може змінювати, чого досягає. Плюс рішення B1 і B2 з обґрунтуванням обраного варіанта.

Розділ 3. Програмна реалізація

Що вже є. Найбільше з усіх розділів: рамки «у текст роботи» кроків 1–11 писалися саме під нього — контракт обміну, канонічне подання, структура блоку, ланцюг і вибір гілки, P2P, зв'язка вузла з воркером, тестування на золотих векторах.

Що писати з нуля. Схема архітектури (одна картинка з обома частинами й потоком даних); опис модулів із їхніми відповідальностями; підрозділ про тестування; інструкція відтворення — як зібрати, запустити демо і повторити експеримент. Ключові модулі коду йдуть у додатки.

Розділ 4. Експериментальне дослідження

Що вже є. Нічого готового: розділ пишеться з CSV фази 3. Єдине, що можна і треба написати заздалегідь, — методику, ще в грудні.

Що писати з нуля. Методика (залізо, версії, seed, калібрування δ, означення метрик); E0–E7 по підрозділу на експеримент, кожен за схемою «питання → конфігурація → таблиця або графік → що з цього випливає»; обговорення; обмеження. Підрозділ «обмеження» пишеться чесно: малі моделі й один-два датасети, локальна мережа без затримок і втрат, довірені ID вузлів без підписів, sybil-атаки й економіка стимулів поза межами роботи, змова реплік поза моделлю загроз, атака Zhang et al. відтворена лише спрощено. Це не послаблює роботу, а посилює: схована вада, яку знайде рецензент, коштує більше, ніж названа автором.

Вступ, висновки, новизна

Вступ пишеться останнім, за структурою ДСТУ 3008:2015: актуальність, об'єкт, предмет, мета, задачі, методи, новизна, практичне значення, апробація. Висновки — це відповіді на сім задач зі вступу, по абзацу на задачу, плюс рекомендації щодо застосовності. Формулювання новизни:

> Наскільки відомо автору, це перше пряме експериментальне порівняння чотирьох родин механізмів верифікації для PoUW на основі навчання нейронних мереж у межах єдиного блокчейн-прототипу та спільного набору моделей атак з оцінкою частки виявлених атак,

> частки хибних відхилень на різнорідному обладнанні та накладних витрат. Часткові порівняння існують [Choi et al.; Ural].

Застереження «наскільки відомо автору» і пряме посилання на часткові порівняння обов'язкові: претензія на абсолютну першість без них — найлегша мішень для рецензента.

Формальні вимоги, які треба тримати в полі зору

що	вимога	статус
оформлення	ДСТУ 3008:2015; список джерел — ДСТУ 8302:2015 (дехто дозволяє APA)	уточнити на кафедрі
набір	Times New Roman 14 пт, інтервал 1,5; поля 2/2/2,5/1,5 см	за вимогами спорідненої кафедри
нумерація	рисунки й таблиці в межах розділу («Рис. 3.2»); посилання у квадратних дужках	—
анотації	UA та EN, 1200–2000 знаків, 5–15 ключових слів	обов'язково
антиплагіат	оригінальність $\geq 70\%$; подання не пізніше ніж за 10 днів до ЕК	кафедра може встановити суворіший поріг
захист	14–27.06.2027; доповідь 7–10 хв, 10–15 слайдів	дати оцінні, уточнити на кафедрі
графічна частина	щонайменше 5 аркушів А4 зі штампом, нормоконтроль	уточнити, чи потрібна для СШІ

Дати весняного графіка — оцінка з точністю до тижня. Список питань до кафедри зібраний у `docs/research/formal-requirements.md`, розділ 7; поставити їх треба восени, а не в травні.

> КОНТРОЛЬНА ТОЧКА · 18 КВІТНЯ 2027
> **КТ4: повна чернетка керівнику** — розділи 1–4, вступ, висновки, анотації UA/EN, список джерел, додатки. Далі: практика за темою БКР (26.04–16.05) іде на правки керівника; 17.05–13.06 — вичитка, нормоконтроль, антиплагіат, рецензія, презентація і попередній захист.

Що робити, коли щось іде не так

Цей розділ — про шість симптомів, які в цьому проєкті трапляються майже гарантовано. Для кожного є перша гіпотеза: місце, з якого статистично вигідно починати. Правило одне — **перевіряй гіпотезу вимірюванням, а не роздумами**. У системі з двох мов, випадковості й порогів міркування «так не може бути» помиляється приблизно завжди.

симптом	перша гіпотеза	що перевірити
перевірка відхиляє чесні блоки на **тій самій** машині	десь лишилася недетермінованість	тест побітової рівності перерахунку; `DataLoader`, dropout, аугментація, моментум, кількість потоків
C++ і Python дають різні хеші	розбіжність канонізації, а не помилка хешу	золоті вектори; порядок байтів; сортування імен; `.contiguous()`; метадані safetensors
точність моделі не росте по раундах	стан не переноситься між блоками	чи `w_start` блоку $n+1$ дорівнює `w_end` блоку n; `lr`; чи не переініціалізується модель щораунду
атака не виявляється взагалі	атака нічого не підмінює або перевірка не туди дивиться	чи відрізняється `w_end` атакувальника від чесного; q, k, поріг; імовірність $1 - f^q$
відхиляється все, включно з чесними	поріг із іншого k або з іншої пари пристроїв	на якому k калібрувався δ; відносний чи абсолютний; сирий розподіл відстаней
експерименти не відтворюються	не зафіксовані seed, версія, коміт	колонки метаданих у CSV; чи чисте дерево git на момент прогону

Чесні блоки відхиляються на тій самій машині. Це не питання порогу: на одному пристрої перерахунок має збігатися **побітово**, тож почни з тесту Кроку 2. Якщо він падає, шукай джерело випадковості в такому порядку: `DataLoader` (його не має бути взагалі), dropout або аугментація, оптимізатор із моментумом, чий стан не потрапив у контрольну точку, невстановлений `torch.use_deterministic_algorithms(True)`, різна кількість потоків у процесах майнера й валідатора.

C++ і Python дають різні хеші. Не порівнюй кінцеві хеші — вони розходяться від будь-якої дрібниці; іди по золотих векторах Кроку 1. Не збігається хеш заголовка — проблема в бінарній серіалізації (порядок байтів, ширина полів, зайве поле); заголовок збігається, а хеш ваг ні — проблема на боці ваг (сортування імен, `.cpu().contiguous()`, тег типу, спроба хешувати файл safetensors замість канонічного подання). Шукай **перший різний байт**, а не різний хеш.

Точність не росте по раундах. Найчастіша причина — модель не продовжує навчання, а починає заново: перевір, чи дорівнює `w_start` блоку $n+1$ значенню `w_end` блоку n. Якщо рівність тримається, а точність стоїть — дивись `lr` і K: за правилом перегонів модель просувається на K кроків за раунд, а не на $N \cdot K$, тож порівнюй із базовою лінією на **тій самій** сумарній кількості кроків.

Атака не виявляється взагалі. Перш ніж чіпати поріг, переконайся, що атака взагалі щось міняє: хеш `w_end` атакувальника має відрізнятися від чесного. Далі арифметика: якщо підроблено частку переходів, а перевіряється q випадкових, імовірність пропустити $\approx f^q$, де f — частка чесних переходів; при $q = 1$ і 10 % чесних переходів пропуск кожного десятого разу очікуваний, а не помилковий.

Відхиляється все, включно з чесними. Майже завжди поріг узятий не звідти: відкалібрований для $k = 10$, а застосований до $k = 100$, або калібрований на парі CPU–CPU, а застосований до CPU–CUDA, або переплутані відносний і абсолютний варіанти. Дивись не на вердикти, а на сирий розподіл відстаней: скупчені нижче порога — справа в порозі; перемішані з відстанями атак — справа в калібруванні, і треба повертатися до E1.

Експерименти не відтворюються. Симптом виглядає як «учора було 0,91, сьогодні 0,87». Перша дія — подивитись у CSV: якщо колонки з seed, пристроєм, версією PyTorch і хешем коміта порожні, прогін непридатний і його треба повторити. Друга — перевірити, чи було дерево git чистим: число, отримане на незакомічених змінах, не належить жодному коміту й не відтворюється в принципі.

Коли просити допомоги і що приносити

Орієнтир простий: 40–60 хвилин на одному симптомі **без жодної нової гіпотези** — час зупинитись і описати проблему. Не «без результату», а саме без нових ідей, що перевіряти далі.

1. **Точна команда**, яку ти запустив, разом із конфігом, seed і гілкою.
2. **Точний вивід** — повністю, копією, не переказом. «Падає з якоюсь помилкою» — це не опис.
3. **Що ти очікував побачити** і чому саме це. Найчастіше помилка виявляється тут: очікування було хибним.
4. **Що вже перевірено** і що це дало — щоб не пропонували гіпотезу, яку ти відкинув годину тому.
5. **Найменший приклад**, на якому симптом ще відтворюється: один перехід замість тисячі раундів.

Половина проблем розв'язується під час самого написання цих п'яти пунктів: вимога сформулювати очікування явно ловить хибні припущення краще за будь-яке відлагодження. Тому пиши їх у файл навіть тоді, коли нікому не збираєшся надсилати.

Правила, яких варто триматися до захисту

1. Один крок за сеанс. Незавершений крок закінчується записом у `UNKNOWN.md`, а не «потім згадаю».

2. Жодного числа без seed, пристрою і хеша коміта. Число без метаданих не існує.
3. Тиждень без коду — це тиждень тексту, а не тиждень паузи.
4. Не міняти протокол після початку експериментів без запису в журнал змін і без повторного прогону зачеплених серій.
5. Спершу симулятор, потім мережа: мережеві баги не мають права зривати таблиці розділу 4.
6. Механізм готовий тоді, коли він міряється, а не тоді, коли він оптимальний.
7. Кожне рішення, яке могло бути іншим, одразу перетворюється на абзац — у рамку «у текст роботи» або в `docs/design-decisions.md`.
8. Негативний результат — теж результат. Цифри не підганяються під гіпотезу, гіпотези переписуються під цифри.

Додаток А. Шпаргалка команд

Усе, що набирається щодня. Команди виконуються з **кореня репозиторію** (там, де лежить `CMakeLists.txt`), якщо не сказано інакше: половина помилок з імпортами й шляхами походить саме з того, що команду запустили з іншої теки.

Середовище Python

```
python3.12 -m venv .venv          # створити середовище (один раз)
source .venv/bin/activate         # увійти; у кожному новому терміналі
deactivate                        # вийти
pip install -r requirements.txt   # поставити пакети записаних версій
pip freeze > requirements.txt     # записати встановлене назад у файл
python -c "import torch; print(torch.__version__)"   # перевірка
which python                      # має вести всередину .venv
```

> **УВАГА**
> `which python` показує, чи ти у своєму середовищі: якщо шлях не веде в теку проєкту, `source .venv/bin/activate` забуто. `pip freeze` виконуй лише всередині активованого середовища, інакше в `requirements.txt` потраплять системні пакети.

Збірка C++

```
cmake -B build -DCMAKE_BUILD_TYPE=Release  # згенерувати правила збірки
cmake -B build -DCMAKE_BUILD_TYPE=Debug    # те саме, але з символами для gdb
cmake --build build -j                     # зібрати все на всіх ядрах
cmake --build build --target node -j       # зібрати одну ціль
rm -rf build                               # перед чистою перезбіркою
ctest --test-dir build --output-on-failure # тести C++ (з'являються у Кроці 7)
ctest --test-dir build -R canonical        # лише тести з "canonical" у назві
```

Для вимірювань — завжди `Release` (серіалізація і хешування в `Debug` повільніші в рази), для налагодження падінь — `Debug`; тип збірки записуй поруч із цифрами.

Python: тести, воркер, симулятор

```
python -m pytest worker/tests -q        # усі тести воркера
python -m pytest worker/tests -x -q     # зупинитись на першому падінні
python -m pytest worker/tests -q -k seed   # тести з "seed" у назві
python -m pytest worker/tests -q -s     # не ховати print() у тестах
python worker/worker.py run/task.json   # воркер: train | verify_a | verify_b
cat run/result.json                     # що воркер відповів
python -m sim.simulate sim/configs/e0.json    # симулятор за конфігом
PYTHONPATH=. python worker/worker.py run/task.json  # якщо worker.* не імпортується
```

Форма `python -m` (а не `pytest` чи `python sim/simulate.py`) навмисна: вона додає поточну теку в шляхи пошуку модулів, і `import worker.canonical` працює без фокусів зі змінними оточення.

git

```
git switch -c step-03-train-block   # нова гілка під крок гайда
git status --short                  # дивитись ПЕРЕД кожним комітом
git add worker/training.py          # конкретні файли, не все підряд
git commit -m "feat(worker): train_block with checkpoints"
git switch main && git merge --no-ff step-03-train-block
git branch -d step-03-train-block   # гілку після злиття видалити
git log --oneline -10               # історія одним рядком на коміт
git rev-parse --short HEAD          # хеш коміту: у шапку кожного CSV
```

Формат повідомлення — `тип(область): що зроблено`. Типи: `feat`, `fix`, `test`, `docs`, `refactor`, `chore`, `exp`. Області: `node`, `worker`, `sim`, `docs`, `guide`. Добре: `fix(node): read length prefix as little-endian` — видно й помилку, й виправлення; `exp(sim): E1 calibration run, 5 seeds` — експеримент видно в історії. Погано: `update`, `fixed bug` — через місяць не означають нічого.

УВАГА

У git ніколи не потрапляють: `store/`, `experiments/results/`, `data/`, `build/`, `.venv/`, `run/`, файли `*.safetensors`. Якщо щось із цього вже в індексі — `git rm -r --cached store experiments/results` (прибирає з індексу, лишає на диску). Ваги однієї моделі — сотні мегабайтів; чистити історію потім набагато дорожче, ніж не додати зараз.

Запуск трьох вузлів

Прапорці зафіксовані в Кроках 9 і 10. Кожен вузол — окремий термінал, спільна тека `store/`, власна тека прогону.

```
# термінал 1
./build/node --name alice --port 7001 --peers 7002,7003 \
             --store store --run run/alice 2>&1 | tee run/alice.log

# термінали 2 і 3 — те саме з --name bob --port 7002 і --name carol --port 7003
# (у --peers кожного вузла перелічені два інші)

ss -ltnp | grep -E '700[123]'      # хто слухає порти: до запуску і після падіння
pkill -f build/node                # прибити вузли від попереднього прогону
tail -f run/alice.log              # дивитись лог наживо
grep -c "block accepted" run/*.log # скільки блоків прийняв кожен вузол
```

`2>&1 | tee run/alice.log` пише вивід і на екран, і у файл: після падіння на двадцятій хвилині прогону лишиться лог, а не спогади. Логи прогонів — чернетка розділу 4.

Прибирання

```
du -sh store experiments/results run build   # скільки займає кожна тека
tar czf ../results-2026-11-01.tgz experiments/results  # архів ПЕРЕД чисткою
rm -rf run/*      # логи й тимчасові task/result: видаляти вільно
```

```
rm -rf build      # збірка: відновлюється двома командами
rm -rf store/*    # ваги: лише разом із ланцюгом, див. попередження
```

УВАГА

`store/` і ланцюг видаляються **тільки разом**: зітреш сховище, лишивши ланцюг, — вузол не знайде `w_start`, жоден блок не перевіриться, а симптом виглядатиме як помилка протоколу. `experiments/results/` — результати дослідження, а не сміття: перед видаленням архівуй.

Додаток Б. Типові помилки та як їх читати

Головна навичка тут не «знати відповідь», а **визначити, хто саме скаржиться**: після цього до причини лишається один крок.

хто говорить	як виглядає повідомлення	де причина
CMake	`CMake Error at CMakeLists.txt:12 (...)`	опис проєкту, не код
препроцесор	`fatal error: ...: No such file or directory`	не встановлений або не підключений заголовок
компілятор	`error:` з файлом і номером рядка	синтаксис або типи в цьому рядку
лінкер	`undefined reference`, `multiple definition`	файл не в цілі або бібліотека не під'єднана
операційна система	`Segmentation fault`, `bind: Address already in use`	пам'ять або ресурси під час виконання
Python	трейсбек із `Error` в останньому рядку	останній рядок — що сталось, кадри вище — де

> Читай **першу** помилку, а не останню: решта зазвичай є її наслідком. У трейсбеку Python останній рядок каже, **що** сталось, а найнижчий кадр із твоїм файлом — де. У пошук копіюй лише стабільну частину повідомлення, без своїх шляхів і адрес пам'яті.

CMake і лінкування

1. Функція оголошена, але не знайдена.

вивід

```
/usr/bin/ld: CMakeFiles/node.dir/src/block.cpp.o: in function `block_hash(BlockHeader
const&)':
block.cpp:(.text+0x5a): undefined reference to `sha256(unsigned char const*, unsigned long)'
collect2: error: ld returned 1 exit status
```

Означає. Компілятор повірив заголовку, що функція десь є, а лінкер її не знайшов: код функції не потрапив у програму. **Перевір першим**, чи доданий `node/src/sha256.cpp` у `add_executable(node ...)` — це причина в дев'яти випадках з десяти. Далі: чи збігається сигнатура в заголовку й у реалізації байт у байт (`const&` проти значення — вже інша функція), чи не забутий `namespace`, чи не оголошена функція `static` у `.cpp`.

Різновид тієї самої помилки — `undefined reference to 'EVP_DigestInit_ex'`: це не твоя функція, а бібліотека, отже в CMake бракує `target_link_libraries(node PRIVATE OpenSSL::Crypto)`. Правило: чужа назва — не під'єднана бібліотека, своя — файл, не доданий у ціль.

2. Заголовка немає в системі.

вивід

```
node/src/sha256.cpp:3:10: fatal error: openssl/evp.h: No such file or directory
compilation terminated.
```

Означає. OpenSSL у системі, можливо, і є (утиліта `openssl` працює), але немає **файлів для розробника** — заголовків. **Перевір першим**: постав пакет із суфіксом `-dev`:

```
sudo apt install libssl-dev        # Debian/Ubuntu
sudo dnf install openssl-devel     # Fedora
ls /usr/include/openssl/evp.h      # перевірка: файл має існувати
```

Після встановлення пакета перегенеруй збірку (`cmake -B build ...`): вже згенеровані правила про новий заголовок не знають.

3. CMake не знаходить OpenSSL.

вивід

```
CMake Error at FindPackageHandleStandardArgs.cmake:230 (message):
  Could NOT find OpenSSL, try to set the path to OpenSSL root folder in the
  system variable OPENSSL_ROOT_DIR (missing: OPENSSL_CRYPTO_LIBRARY)
```

Означає. Те саме, що й попереднє, але на етап раніше: `find_package(OpenSSL REQUIRED)` не знайшов ані заголовків, ані бібліотеки. **Перевір першим** той самий `libssl-dev`; якщо пакет стоїть, OpenSSL лежить у нетиповому місці — `cmake -B build -DOPENSSL_ROOT_DIR=/шлях`. І обов'язково `rm -rf build` перед повторною спробою: невдалий результат пошуку кешується разом з усім іншим у `CMakeCache.txt`.

4. Одна функція визначена двічі.

вивід

```
/usr/bin/ld: CMakeFiles/node.dir/src/chain.cpp.o: in function `to_hex(...)':
chain.cpp:(.text+0x0): multiple definition of `to_hex(...)';
CMakeFiles/node.dir/src/block.cpp.o:block.cpp:(.text+0x0): first defined here
```

Означає. Тіло функції лежить у заголовку, і цей заголовок підключили два різні `.cpp` — кожен отримав свою копію. **Перевір першим**: або познач функцію `inline`, або лиши в заголовку саме оголошення, а тіло перенеси в `.cpp` (для цього проєкту правильніше друге).

C++ під час виконання

5. Падіння після розбору короткого hex-рядка.

вивід

```
Segmentation fault (core dumped)
```

Означає. Програма звернулась до пам'яті, якої в неї немає. Типовий сценарій: `from_hex` отримав рядок, коротший за 64 символи (обрізаний JSON, порожнє поле `"prev_hash"` у генезисі, не повністю скопійований хеш), сформував менше ніж 32 байти, а далі `std::memcpy(header.prev_hash, bytes.data(), 32)` прочитав за межею буфера.

Перевір першим не «де падає», а «звідки прийшли дані»: будь-який розбір hex має на вході перевіряти, що довжина рядка рівно 64 і символи з `[0-9a-f]`, і кидати виняток інакше. Знайти сам рядок допомагає `Debug`-збірка під `gdb` (`bt` після падіння), а причину, тобто місце виходу за межу, — збірка з `-fsanitize=address`.

6. Порт зайнятий.

вивід

```
bind: Address already in use
node: failed to bind 127.0.0.1:9001
```

Означає. Або попередній вузол ще живий (термінал закрили, процес лишився), або він завершився, але сокет у стані `TIME_WAIT` і ядро тримає порт ще хвилину-дві.

Перевір першим `ss -ltnp | grep 9001`: якщо процес є — `pkill -f build/node`. Якщо процесу немає, а порт зайнятий, це другий випадок, і лікується він у коді: `setsockopt(fd, SOL_SOCKET, SO_REUSEADDR, ...)` перед `bind`. Для прототипу, який перезапускається десятки разів на вечір, `SO_REUSEADDR` обов'язковий.

7. Прочитано менше, ніж просили.

вивід

```
[alice] expected 4 bytes of length prefix, recv returned 3
[alice] json parse error: unexpected end of input
```

Означає. Це не помилка, а нормальна поведінка TCP, якої не чекали: TCP — це **потік байтів**, а не послідовність повідомлень. Один `send` на 5000 байтів може приїхати трьома порціями, два `send` — злитись в один `recv`. Повернене значення — «скільки вдалося зараз», а не «скільки просили». **Перевір першим**, чи є в коді цикл читання до повного набору байтів:

правило читання з сокета

```
read_exact(fd, buf, n):
    прочитано = 0
    поки прочитано < n:
        r = recv(fd, buf + прочитано, n - прочитано, 0)
        якщо r == 0  -> пір закрив з'єднання, це не помилка, а кінець
        якщо r < 0   -> справжня помилка, дивитись errno
        прочитано += r
```

Спершу `read_exact` на 4 байти довжини, потім `read_exact` на саму довжину. Баг підступний тим, що на коротких повідомленнях не відтворюється місяцями, а на першому блоці з довгим списком хешів контрольних точок з'являється одразу.

8. Спроба виділити гігабайти.

вивід

```
terminate called after throwing an instance of 'std::bad_alloc'
  what():  std::bad_alloc
Aborted (core dumped)
```

Означає. Програма прочитала префікс довжини і повірила йому: або префікс прочитано в іншому порядку байтів (16 замість 268435456), або потік зсунувся і довжиною вважається шматок JSON. **Перевір першим** порядок байтів (у нас усюди little-endian) і наявність верхньої

межі: повідомлення довше за 8 МБ — привід розірвати з'єднання, а не виділяти пам'ять. Недовірений вхід не має права керувати розміром виділення.

Python і PyTorch

9. Не знайдено власний модуль.

вивід

```
Traceback (most recent call last):
  File "worker/tests/test_canonical.py", line 3, in <module>
    from worker.canonical import canonical_weight_hash
ModuleNotFoundError: No module named 'worker'
```

Означає. Python шукає пакет `worker` серед своїх шляхів і не бачить кореня репозиторію. **Перевір першим**, за порядком: чи запущено з кореня проєкту (`pwd`); чи є файли `worker/__init__.py` і `worker/tests/__init__.py` — без них тека не пакет; чи форма запуску `python -m pytest worker/tests`, а не `pytest worker/tests` (перша додає поточну теку в шляхи пошуку, друга — ні). Крайній засіб на один запуск — `PYTHONPATH=. python worker/worker.py`.

10. Тензори на різних пристроях.

вивід

```
RuntimeError: Expected all tensors to be on the same device, but found at least two devices,
cuda:0 and cpu!
```

Означає. Модель переїхала на GPU, а дані (чи навпаки) лишились на CPU. **Перевір першим**, чи є одне місце, де визначається `device`, і чи всі переміщення йдуть через нього: модель, батч зображень і батч міток. `print(next(model.parameters()).device, x.device)` показує розбіжність одразу.

Другий бік цієї помилки: перед канонічним хешем ваги **завжди** приводяться до CPU — `t.detach().cpu().contiguous()`. Забутий `.cpu()` дає не лише виняток, а й ризик іншого хешу, тобто мовчазної розбіжності між майнером і валідатором.

11. Немає детермінованої реалізації операції.

вивід

```
RuntimeError: index_add_cuda_ does not have a deterministic implementation, but you set
'torch.use_deterministic_algorithms(True)'. You can turn off determinism just for this
operation, or you can use the 'warn_only=True' option.
```

Означає. Ти попросив побітову відтворюваність, а обрана операція недетермінована (порядок паралельного підсумовування не фіксований). **Перевір першим** не як вимкнути детермінізм — він фундамент механізму B, — а чим замінити операцію: детермінований аналог, виконання цього шматка на CPU або, для CUDA, змінна оточення `CUBLAS_WORKSPACE_CONFIG=:4096:8` перед запуском. І пам'ятай межі гарантії: побітова відтворюваність обіцяна лише на **тій самій** машині, тій самій версії PyTorch і тому самому пристрої — саме тому поріг перевірки B калібрується, а не призначається.

12. Форми не сходяться при перетворенні масиву.

вивід

```
ValueError: could not broadcast input array from shape (784,) into shape (28,28)
```

Означає. Кількість чисел правильна, а форма — ні. Класика при ручному читанні MNIST: зображення лежить одним рядком із 784 чисел, а його кладуть у двовимірний осередок. **Перевір першим**, де саме форма змінюється: `.reshape(-1, 28, 28)` при завантаженні чи `.reshape(-1, 784)` перед повнозв'язним шаром; `print(arr.shape)` до і після кожного перетворення знаходить цей баг за дві хвилини.

Сусідній різновид уже всередині моделі — `shape '[-1, 1024]' is invalid` — означає, що після згорток розмір не такий, як ти порахував на папері. Не вгадуй: `print(x.shape)` перед `view`, або `nn.Flatten()` і `nn.LazyLinear`, щоб розмір порахувався сам.

13. Невідповідність типів у функції втрат.

вивід

```
RuntimeError: expected scalar type Long but found Float
```

Означає. `F.cross_entropy(logits, y)` вимагає `float32` для `logits` і цілі `int64` (`torch.long`) для міток, а мітки приїхали дробовими. **Перевір першим** `print(x.dtype, y.dtype)` перед викликом втрат: має бути `torch.float32` і `torch.int64`.

> **УВАГА**
> Найчастіше це вилазить не в навчанні, а в атаці №4 (label flipping). Вираз `(y + 1) % 10` над тензором, який десь раніше став `float`, тихо лишається дробовим, і `cross_entropy` падає лише на наступному кроці. Пиши `y = ((y + 1) % 10).long()` і перевіряй тип одразу після підміни міток.

Мовчазна помилка: C++ і Python дають різні хеші

Ця помилка не має повідомлення: обидві половини працюють, тести всередині кожної проходять, а разом вони не сходяться — вузол відкидає власний блок або `verify_b` завжди повертає `reject`. Шукай двійковим пошуком, від найпростішого до найскладнішого.

1. **Чи однакова сама SHA-256?** Обидві сторони рахують `sha256(b"abc")`. Правильна відповідь починається з `ba7816bf`. Не збіглося — проблема не в проєкті, а в підключенні бібліотеки.
2. **Чи однаковий заголовок блоку?** Візьми золотий вектор із `docs/golden.json`, надрукуй у обох мовах довжину і перші 16 байтів у hex. Довжина має бути рівно 236.
3. **Чи однаковий хеш ваг?** Візьми стан з одного тензора `{"w": torch.zeros(2, 3)}` і порівняй хеш, порахований обома сторонами.

Коли зрозуміло, **який** із трьох хешів різний, причина завжди в цьому списку:

що перевірити	як виглядає помилка	як переконатись
порядок байтів	числа збігаються при друці, байти — ні	`struct.pack("<I", 1)` має дати `01 00 00 00`, а не `00 00 00 01`

що перевірити	як виглядає помилка	як переконатись
сортування імен	хеш стрибає між запусками	`sorted(state)` у Python і те саме порівняння рядків у C++
`contiguous`	хеш змінюється після `transpose`	перед байтами завжди `.detach().cpu().contiguous()`
байт типу і виміри	різниця лише на багатовимірних тензорах	`dtype_tag = 1` і `ndim` — по одному байту, кожен вимір — `u32`
метадані	той самий стан дає різні хеші	хеш береться від тензорів, а не від байтів файлу `.safetensors`
hex проти байтів	довжина 64 замість 32	у хеш ідуть `digest()` (32 байти), у JSON — `hexdigest()`

УВАГА

Окремо — найтиповіша пастка C++: компілятор має право вставити порожні байти між полями структури, щоб вирівняти їх у пам'яті. Хеш від `sizeof(BlockHeader)` байтів дасть 240 або 248 байтів, і Python такого числа не отримає ніколи. Заголовок серіалізується полем за полем у буфер, і перед хешуванням перевіряється `buf.size() == 236`.

Розбіжність хешів небезпечніша за падіння: падіння видно одразу, а розбіжність виглядає як результат експерименту — і в таблицю FRR потрапляє число, яке нічого не міряє.

Додаток B. Словник понять

Словник для швидкого пригадування, а не заміна розділам гайда. Якщо поняття тут пояснене, а ти не можеш переказати його своїми словами — це рядок для `UNKNOWN.md`.

Блокчейн і протокол

слово	що означає
`блок`	Порція роботи з посиланням на попередню. У нас — результат K кроків навчання: хеші станів, хеші контрольних точок, заявлена метрика.
`заголовок блоку`	Фіксовані 236 байтів, від яких береться `block_hash`. Тіло блоку може бути будь-яким, але хешується саме заголовок.
`хеш-ланцюг`	Кожен блок містить хеш попереднього. Зміна одного старого байта змінює всі наступні хеші, тому історію не можна тихо переписати.
`генезис-блок`	Перший блок ланцюга, у якого немає попередника. У нас він задає стартовий стан моделі (правило P1).
`форк`	Дві версії ланцюга від спільного предка. Виникає, коли два майнери одночасно знайшли валідний блок.
`найдовший ланцюг`	Правило вибору між форками: перемагає гілка з більшою кількістю блоків. У нас кожен блок — це фіксовані K кроків, тому довжина пропорційна виконаній роботі.
`orphan`	Блок, який прийшов раніше за свого попередника. Не відкидається, а чекає в черзі, поки попередник з'явиться.
`консенсус`	Правила, за якими незалежні вузли без керівника сходяться на одній історії. Складається з правил валідності блоку і правила вибору гілки.
`PoW`	Proof of Work, доказ роботи: щоб додати блок, треба витратити обчислення. Перевірка коштує одну хеш-операцію — саме ця асиметрія і тримає Bitcoin.
`PoUW`	Proof of Useful Work: та сама ідея, але робота має цінність поза мережею. Тема цієї роботи; ціна — втрата дешевої перевірки.
`PoL`	Proof of Learning: доказ того, що навчання справді відбулося, через послідовність контрольних точок. Наш механізм B.
`майнер`	Вузол, який виконує роботу і подає блок. У нас — той, хто робить K кроків навчання на призначених йому батчах.
`валідатор`	Вузол, який перевіряє чужий блок одним із механізмів A–D і голосує за прийняття чи відхилення. Той самий вузол буває і майнером, і валідатором.
`раунд`	Один цикл «завдання, робота, перевірка, прийняття». Номер раунду входить у заголовок блоку і у формулу seed.
`nonce`	Число, яке додають до даних, щоб змінити хеш. У PoW його перебирають; у нас — використовується лише в схемі commit-reveal, щоб приховати зобов'язання.

слово	що означає
`commit–reveal`	Дві фази: спершу всі публікують `SHA256(результат ‖ miner_id ‖ nonce)`, потім розкривають сам результат. До дедлайну копіювати нічого, бо видно лише хеш (правило P8).
`fraud proof`	Компактний доказ, що блок неправильний: номер поганого переходу. Перевірити доказ дешево (k кроків), тому не потрібно, щоб усі перевіряли все.
`sybil`	Атака, у якій один учасник створює багато особистостей і отримує більшість голосів. У прототипі не розглядається: ID вузлів вважаються довіреними (правило P11).
`модель загроз`	Явний список того, від чого протокол захищає і від чого ні. Без нього будь-які цифри виявлення нічого не означають.

Навчання нейронних мереж

слово	що означає
`тензор`	Багатовимірний масив чисел — базовий тип PyTorch. Ваги моделі — це словник «ім'я тензора → тензор».
`батч`	Порція прикладів, оброблених разом за один крок. У нас $B = 64$ і фіксується завданням (правило P3).
`крок`	Одне оновлення ваг за одним батчем. Основна одиниця роботи в протоколі: блок містить рівно K кроків.
`епоха`	Повний прохід по всьому навчальному набору. У нас майже не використовується: роботу міряємо кроками, бо блок має бути коротким.
`градієнт`	Вектор, який показує, у який бік зростає функція втрат. Крок навчання рухає ваги в протилежний бік.
`learning rate`	Множник довжини кроку, η. Фіксований у завданні; якби майнер міг заявити $\eta \to 0$, він проходив би перевірку B, нічого не навчаючи.
`SGD`	Стохастичний градієнтний спуск: `w = w − eta ∗ grad`. Обраний навмисне через простоту й відтворюваність.
`momentum`	Додаткова «інерція» до SGD. У нас вимкнена: її стан довелося б класти в кожну контрольну точку, інакше перерахунок валідатора не збігся б.
`контрольна точка`	Збережений стан ваг кожні k кроків. Послідовність $C_0 \ldots C_m$ — це і є доказ навчання; у блоці лежать лише їхні хеші.
`seed`	Число, з якого генератор випадкових чисел відтворює ту саму послідовність. У нас `seed = SHA256(prev_hash ‖ miner_id ‖ round)`: майнер його не обирає.
`детермінізм`	Властивість «однаковий вхід дає однаковий вихід побітово». На одній машині досяжна, між різним залізом — ні. Уся складність механізму B саме тут.
`метрика`	Число, яким міряють якість моделі; у нас — частка правильних відповідей на тестовому наборі. У заголовку зберігається як ціле $\times 10^6$.

слово	що означає
`overfitting`	Модель запам’ятала навчальні приклади замість того, щоб узагальнити. Причина, чому метрика рахується на окремому тестовому наборі.
`label flipping`	Псування міток: `y' = (y + 1) mod 10` на частці прикладів. Основа атаки №4 — вузол працює чесно, але псує спільну модель.
`detection rate`	Частка спроб шахрайства, які механізм спіймав. Головна метрика роботи, рахується окремо для кожної пари «атака — механізм».
`FRR`	False rejection rate, частка чесних блоків, відхилених помилково. Ціна перевірки: чесний вузол на іншому залізі дає трохи інші ваги.
`накладні витрати`	Відношення часу перевірки до часу навчання. Для B це приблизно $q \cdot k/K$; дорога перевірка знецінює всю ідею корисної роботи.
`ROC`	Крива «виявлення проти хибних відхилень» при різних значеннях порогу. Показувати ROC чесніше, ніж одну точку: читач бачить компроміс, а не вдало підібраний поріг.
`поріг δ`	Максимальна допустима відстань між вагами майнера і перерахунком валідатора. Відносний і калібрований (правило P6); сам поріг — вектор атаки.

> Два поняття, які найлегше сплутати. **Detection rate** і **FRR** міряються на **різних** вузлах: перше — на зловмисних, друге — на чесних. Механізм, який відхиляє все підряд, має ідеальний detection rate і нікчемну цінність, і видно це лише за FRR. Тому в таблицях роботи вони завжди стоять поруч.

Додаток Г. Чек-листи

Чек-лист потрібен не тому, що ти щось забудеш, а тому, що без нього «попрацював три години» не перетворюється на «просунувся на крок».

Чек-лист сеансу

ТВОЄ ЗАВДАННЯ

Перед роботою (три хвилини):

1. `git switch main && git pull` — почати з актуального стану.
2. `git switch -c step-NN-назва` — окрема гілка під сьогоднішній крок.
3. Записати одним реченням, що саме сьогодні має запрацювати. Не «попрацювати над верифікацією», а «`verify_b` повертає `reject` на лінивому повторенні».
4. Відкрити `UNKNOWN.md` і подивитись, чи не блокує сьогоднішній крок щось із нього.

Після роботи (п'ять хвилин):

1. Тести зелені: `python -m pytest worker/tests -q` і `ctest --test-dir build --output-on-failure`.
2. Коміт із осмисленим повідомленням; `git status --short` перед ним.
3. `UNKNOWN.md` оновлено: нові слова додані, розібрані — закреслені.
4. Один рядок у журнал робіт.

docs/journal.md

```
| дата | год | що робив | де застряг | наступне |
|---|---|---|---|---|
| 2026-10-07 | 2,5 | verify_b | хеші C++/Python розійшлись | звірити байти K |
| 2026-10-09 | 3,0 | знайшов: K писався як u64 | — | атака 1 у симуляторі |
```

Колонка «де застряг» — найцінніша: наступний сеанс починається з неї замість п'ятнадцяти хвилин пригадування, а у квітні з цих рядків складається підрозділ «Тестування й налагодження» — помилки інтеграції це матеріал роботи, а не сором.

Чек-лист тижня

САМОПЕРЕВІРКА

1. Який крок гайда завершено **повністю** (тести, коміт, злиття в `main`)?
2. Яка найближча контрольна точка і скільки днів до неї?
3. Скільки годин вийшло цього тижня? Порівняй із планом (10 год до 1 листопада, 8 далі).
4. Що заблоковано і чим саме? Для кожної блокади — одна дія: спитати керівника, прочитати статтю, спростити задачу. Якщо в плані щось уже видно як нереальне, ріжемо обсяг, а не терміни: список того, що ріжеться першим, є у `docs/plan.md`.

Чек-лист перед контрольною точкою 1 листопада

КОНТРОЛЬНА ТОЧКА · 1 ЛИСТОПАДА 2026

Прототип зараховано, якщо виконано всі п'ять пунктів одночасно:

1. Три вузли на C++ обмінюються блоками по TCP локально, ланцюг у всіх трьох однаковий.
2. Вузли навчають модель на MNIST, і блоки з результатами додаються в ланцюг.
3. Працює механізм B (перерахунок переходу), як базова лінія — механізм A.
4. Реалізовано щонайменше одну атаку, і її видно в цифрах, а не на словах.
5. Є перші числа: detection rate, FRR, накладні витрати, точність за раундами проти централізованого навчання.

Якщо мережі немає — прототипом вважається симулятор плюс C++-ланцюг без P2P, а мережа переїжджає у фазу 2. Рішення про скорочення приймається **до** дати, а не після.

Чек-лист перед кожним експериментом

УВАГА

Експеримент, умови якого не записані, доведеться переробляти. Це не акуратність, а економія тижнів у лютому.

1. Seed зафіксовані й перелічені явно; їх не менше п'яти, у результат іде середнє і стандартне відхилення.
2. Записано машину: процесор, кількість потоків, пристрій (CPU чи GPU), версія PyTorch, тип збірки C++.
3. Записано коміт: `git rev-parse --short HEAD`. Робоче дерево чисте — інакше коміт нічого не означає.
4. Конфіг збережено **поруч із CSV**, а не лише в `sim/configs/`: конфіг у репозиторії завтра зміниться, а копія біля результату — ні.
5. Ім'я файлу результату містить експеримент і дату: `experiments/results/e2-2026-12-05.csv`.

Поруч із кожним CSV лягає файл `*.meta.json` із полями `exp`, `commit`, `dirty`, `machine`, `torch`, `build_type`, `seeds`, `config`. Вісім рядків, які через три місяці відрізняють результат від набору чисел.

Чек-лист перед показом керівнику

САМОПЕРЕВІРКА

Зустріч — це п'ять пунктів у такому порядку, а не переказ того, що відбувалось:

1. **Що зроблено** — одне речення на кожен пункт, без коду на екрані.
2. **Як перевірено** — які тести, скільки прогонів, на чому саме.
3. **Цифри з умовами вимірювання** — не «виявлення 94%», а «94% на атаці 1, k = 10, відносний поріг $\alpha = 1$, п'ять seed, CPU».

4. **Що далі** — наступний крок і дата.
5. **Відкриті питання** — сформульовані так, щоб на них можна було відповісти «так» або «ні», або вибрати з двох варіантів.

Керівник не спеціалізується на блокчейні, тому один слайд чи абзац на початку має пояснювати контекст: що таке раунд, хто такий валідатор, який механізм зараз обговорюємо.

Додаток Д. Як користуватися розв'язками

Розв'язки лежать в окремому файлі `pouwchain-solutions.pdf` навмисно: щоб їх не можна було побачити випадково, скосивши око.

Правило те саме, що й у `toychain`, і воно не про чесність, а про те, як влаштована пам'ять.

1. **Спробуй сам** — щонайменше 20–30 хвилин на функцію. Застряг на місці, а не думаєш — це вже сигнал, але не привід відкривати розв'язок.
2. **Прочитай підказки по одній.** Рівень 1 каже, куди дивитись; рівень 2 — яка функція чи формула; рівень 3 — майже готовий рядок. Між рівнями роби ще одну спробу.
3. **Відкрий розв'язок і прочитай РІВНО ОДИН фрагмент** — той, що стосується місця, де ти стоїш. Не гортай далі, не читай «як там зроблено загалом».
4. **Закрий файл.** Фізично, не згорни вікно.
5. **Напиши код з пам'яті.** Не з екрана поруч, не копіюванням.
6. **Не вийшло з пам'яті — значить, ти не зрозумів, а впізнав.** Повернись до кроку 1 з тим самим фрагментом.
7. **Запиши в `UNKNOWN.md`, що саме довелося підглянути.** Через тиждень перевір себе на цьому місці ще раз.

> УВАГА
> **Диплом, захищений із кодом, якого ти не можеш пояснити, гірший за диплом із меншим обсягом коду.**
>
> Комісія не читає репозиторій. Вона ставить питання «а чому тут саме так» і дивиться, скільки секунд ти думаєш. Три функції, про кожну з яких ти можеш говорити п'ять хвилин, дають кращий захист, ніж тридцять переписаних, про які сказати нічого. І це не риторика: обсяг роботи тут і так достатній, ризик лежить рівно в протилежному боці.

> Окремий випадок — код із чужих репозиторіїв (PoL, атаки Fang, label flipping). Ліцензії там або відсутні, або неясні, тому в проєкт він не переноситься взагалі: читається як довідка, а пишеться своє. Це не лише юридичне питання — переписане своїми руками ти зможеш пояснити, вставлене не зможеш.

Додаток Е. Питання, до яких треба бути готовим на захисті

Комісія не спеціалізується на блокчейні. Тому найважчі питання будуть не технічні, а прості: «навіщо», «а чим це відрізняється», «скільки коштує». На кожне має бути відповідь на 30–60 секунд плюс місце в роботі, куди можна показати.

питання	звідки береться відповідь
Чим ваша робота відрізняється від федеративного навчання?	Розділ 2, модель загроз: там сервер довірений і агрегує оновлення, тут довіреного центру немає — тому й потрібна верифікація роботи, а не лише агрегація.
Чому не можна просто перевірити точність натренованої моделі?	Розділ 4, рядок механізму A в головній матриці: A пропускає плагіат і часткову роботу, бо метрика не доводить, що ваги отримані саме цим вузлом.
Звідки взявся поріг δ і що буде, якщо його змінити?	Розділ 2, правило P6, і експеримент E1: поріг не призначений, а відкалібрований за розподілом похибок чесних вузлів; ефект зміни — ROC-крива, а не одне число.
Чому чесний вузол може бути відхилений?	Розділ 1, недетермінованість навчання: два запуски на різному залізі не дають побітово однакових ваг. Це метрика FRR, і в таблицях вона стоїть поруч із виявленням.
Що заважає вузлу просто скопіювати чужу роботу?	Розділ 2, правило P2: `seed = SHA256(prev_hash ‖ miner_id ‖ round)`, тому в копіювальника інший порядок даних, і скопійовані контрольні точки не витримують перерахунку.
Скільки коштує перевірка?	Розділ 4, стовпчик накладних витрат: для B це приблизно $q \cdot k/K$ від часу раунду, для C — n-кратна вартість. Числа виміряні, а не оцінені.
Чому дві мови — C++ і Python, а не одна?	Розділ 3, архітектура: мережа й консенсус на C++, навчання на PyTorch, зв'язка через окремий процес і JSON-файли. Одна мова коштувала б місяці без наукової віддачі.
Що саме нового у вашій роботі?	Вступ, новизна: пряме порівняння чотирьох родин механізмів на одному стенді зі спільним набором з п'яти атак. Окремі механізми в літературі описані, прямого порівняння за detection rate, FRR і накладними витратами не знайдено.

питання	звідки береться відповідь
Які обмеження ви самі бачите?	Розділ 4, підрозділ обмежень: малі моделі й MNIST, локальні вузли замість реальної мережі, немає підписів і економіки стимулів, sybil і змова поза моделлю загроз.
Чи можна це застосувати на практиці?	Висновки: за нинішніх накладних витрат і FRR підхід придатний там, де цінність навченої моделі перевищує вартість перевірки; загального заміщення PoW із цих цифр не випливає.
Як ви переконались, що ваша реалізація правильна?	Розділ 3, тестування: золоті вектори, на яких звіряються C++ і Python, тест побітової рівності перерахунку на одній машині, тести на незалежність хешу від порядку ключів.
Чому MNIST, а не велика сучасна модель?	Розділ 4, методика: масштаб моделі не є предметом дослідження, а малий масштаб дає сотні раундів і п'ять seed замість одного прогону. Перенесення висновків на великі моделі записане в обмеженнях.

Два дешевих прийоми. Перший: на кожне «чому» тримай напоготові контрприклад — що зламається, якщо зробити інакше (без прив'язки seed до `miner_id` проходить плагіат; з абсолютним порогом пізно в навчанні проходить ліниве повторення). Другий: про те, чого ти не робив, кажи прямо і показуй, де це записане в обмеженнях — «поза межами роботи, підрозділ 4.5» є повноцінною відповіддю, імпровізація на цьому місці — ні.

> **У ТЕКСТ РОБОТИ**
> Ці дванадцять питань — готовий план останнього слайда презентації і водночас перевірка повноти роботи. Якщо на якесь із них немає місця в тексті, куди можна показати пальцем, це не проблема захисту, а пропущений підрозділ. Перевір цей додаток ще раз у березні, коли розділи 3 і 4 будуть дописані.